CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

COLLEGE OF COMPUTER STUDIES

Software Test Document

for

GIT it!

Change History

Version	Date	Description
1.0	21 October 2026	Regression test of the main branch working tree (2f9958f plus uncommitted changes) and production smoke test.
1.1	21 October 2026	Run 2: fixes for INC-01 to INC-06 retested; Ask Git-It cases rewritten for the rebuilt tutor (3.14 and 3.15); INC-07, observations and A.4 added.
1.2	21 October 2026	INC-07 fixed; 3.13 retested on production after deployment; 3.14 and 3.15 retested; observations O-4 and O-5 added.
2.0	21 October 2026	Run 3: full regression of the current working tree. All 63 Adventure tiers and 12 Challenge Gate trials played in the browser; learner, admin and production features added; screenshots embedded in each test case.
2.1	30 October 2026	A.3: restored the incident reports for INC-01 to INC-07 from runs 1 and 2, which version 2.0 had left out, with each corrective action, its commit and the retest results from runs 2 and 3. Added the defect log (Table A-5) and the run 1 screenshots of four defects (Figures 75 to 78).
3.0	30 October 2026	Run 4: performance test of the production deployment against NFR-PERF-01 to NFR-PERF-09 (cases 3.31 to 3.38, Table 2-2 and Appendix A.4). Five cases passed and three failed. Added INC-08 and INC-09 (closed) and INC-10 (open) to the defect log, and observations O-6 and O-7.
3.1	30 October 2026	Run 5: retest of cases 3.31, 3.32, 3.35 and 3.36 after two INC-10 corrective actions (database moved to Singapore, 8 server threads; Table 2-3 and Appendix A.5). The three failed cases

		improved but still fail; 3.35 passed again. INC-10 updated and still open.
3.2	30 October 2026	Run 6: final performance test after the INC-10 corrective actions (commits 6e9ed3c and d65fe22), against the targets revised in SRS 1.4 (Table 2-4, Appendix A.6). Every repeated case passed and INC-10 is closed. Result fields renamed Actual Result; Table 1-1 maps the testing and quality assurance requirements to this document; every incident report lists its fix commit.improved but still fail; 3.35 passed again. INC-10 updated and still open.

Table of Contents

1. Introduction 11

1.1. System Overview 11

1.2. Test Approach 11

1.3. Definitions and Acronyms 12

2. Test Plan 14

2.1 Features to be Tested 14

2.2 Features not to be Tested 14

2.3 Testing Tools and Environment 15

3. Test Cases 18

3.1 Backend Automated Regression Suite 18

3.1.1 Purpose 18

3.1.2 Inputs 18

3.1.3 Expected Outputs & Pass/Fail Criteria 18

3.1.4 Test Procedure 18

3.2 Backend Code Quality Gates 18

3.2.1 Purpose 18

3.2.2 Inputs 18

3.2.3 Expected Outputs & Pass/Fail Criteria 18

3.2.4 Test Procedure 18

3.3 Frontend Automated Suite and Build 19

3.3.1 Purpose 19

3.3.2 Inputs 19

3.3.3 Expected Outputs & Pass/Fail Criteria 19

3.3.4 Test Procedure 19

3.4 Architecture, Contract and Deployment Guards 19

3.4.1 Purpose 19

3.4.2 Inputs 19

3.4.3 Expected Outputs & Pass/Fail Criteria 19

3.4.4 Test Procedure 19

3.5 Sign In and Sign Out 20

3.5.1 Purpose 20

3.5.2 Inputs 20

3.5.3 Expected Outputs & Pass/Fail Criteria 20
3.5.4 Test Procedure 20
3.6 Account Registration 22
3.6.1 Purpose 22
3.6.2 Inputs 22
3.6.3 Expected Outputs & Pass/Fail Criteria 22
3.6.4 Test Procedure 22
3.7 Password Reset 24
3.7.1 Purpose 24
3.7.2 Inputs 24
3.7.3 Expected Outputs & Pass/Fail Criteria 24
3.7.4 Test Procedure 24
3.8 Onboarding and Companion Purchase 26
3.8.1 Purpose 26
3.8.2 Inputs 26
3.8.3 Expected Outputs & Pass/Fail Criteria 26
3.8.4 Test Procedure 26
3.9 Adventure Terminal Objectives and Exit 27
3.9.1 Purpose 27
3.9.2 Inputs 27
3.9.3 Expected Outputs & Pass/Fail Criteria 27
3.9.4 Test Procedure 27
3.10 Progress Persistence 29
3.10.1 Purpose 29
3.10.2 Inputs 29
3.10.3 Expected Outputs & Pass/Fail Criteria 29
3.10.4 Test Procedure 29
3.11 Access Control and Routing 30
3.11.1 Purpose 30
3.11.2 Inputs 30
3.11.3 Expected Outputs & Pass/Fail Criteria 30
3.11.4 Test Procedure 30
3.12 Responsive Layout at Phone Width 32
3.12.1 Purpose 32
3.12.2 Inputs 32
3.12.3 Expected Outputs & Pass/Fail Criteria 32

3.12.4 Test Procedure 32
3.13 Production Smoke Test 33
3.13.1 Purpose 33
3.13.2 Inputs 34
3.13.3 Expected Outputs & Pass/Fail Criteria 34
3.13.4 Test Procedure 34
3.14 Ask Git-It: General Git Help 37
3.14.1 Purpose 37
3.14.2 Inputs 37
3.14.3 Expected Outputs & Pass/Fail Criteria 37
3.14.4 Test Procedure 37
3.15 Ask Git-It: Scenario Answer Protection 40
3.15.1 Purpose 40
3.15.2 Inputs 40
3.15.3 Expected Outputs & Pass/Fail Criteria 40
3.15.4 Test Procedure 40
3.16 Repository Diagrams 41
3.16.1 Purpose 41
3.16.2 Inputs 41
3.16.3 Expected Outputs & Pass/Fail Criteria 41
3.16.4 Test Procedure 41
3.17 Project Files Panel 44
3.17.1 Purpose 44
3.17.2 Inputs 44
3.17.3 Expected Outputs & Pass/Fail Criteria 44
3.17.4 Test Procedure 44
3.18 Workspace Aids 45
3.18.1 Purpose 45
3.18.2 Inputs 45
3.18.3 Expected Outputs & Pass/Fail Criteria 45
3.18.4 Test Procedure 45
3.19 Run Control: Failure, Retry and Replay 47
3.19.1 Purpose 47
3.19.2 Inputs 47
3.19.3 Expected Outputs & Pass/Fail Criteria 47
3.19.4 Test Procedure 47

3.20 Easy Tier of Every Adventure Level 51
3.20.1 Purpose 51
3.20.2 Inputs 51
3.20.3 Expected Outputs & Pass/Fail Criteria 51
3.20.4 Test Procedure 51
3.21 Medium and Hard Tiers of Every Adventure Level 51
3.21.1 Purpose 51
3.21.2 Inputs 51
3.21.3 Expected Outputs & Pass/Fail Criteria 51
3.21.4 Test Procedure 51
3.22 Interactive Staging, Conflicts and Merge Tool 52
3.22.1 Purpose 52
3.22.2 Inputs 52
3.22.3 Expected Outputs & Pass/Fail Criteria 52
3.22.4 Test Procedure 52
3.23 Challenge Gates 53
3.23.1 Purpose 53
3.23.2 Inputs 53
3.23.3 Expected Outputs & Pass/Fail Criteria 53
3.23.4 Test Procedure 53
3.24 Squire's Drill 54
3.24.1 Purpose 54
3.24.2 Inputs 54
3.24.3 Expected Outputs & Pass/Fail Criteria 54
3.24.4 Test Procedure 54
3.25 Chapter Features 56
3.25.1 Purpose 56
3.25.2 Inputs 56
3.25.3 Expected Outputs & Pass/Fail Criteria 56
3.25.4 Test Procedure 56
3.26 Dashboard Views 60
3.26.1 Purpose 60
3.26.2 Inputs 60
3.26.3 Expected Outputs & Pass/Fail Criteria 60
3.26.4 Test Procedure 60
3.27 Shop and Loadout 62

3.27.1 Purpose *62*
3.27.2 Inputs *62*
3.27.3 Expected Outputs & Pass/Fail Criteria *62*
3.27.4 Test Procedure *62*
3.28 Account Settings *65*
3.28.1 Purpose *65*
3.28.2 Inputs *65*
3.28.3 Expected Outputs & Pass/Fail Criteria *65*
3.28.4 Test Procedure *65*
3.29 Admin Console *68*
3.29.1 Purpose *68*
3.29.2 Inputs *68*
3.29.3 Expected Outputs & Pass/Fail Criteria *68*
3.29.4 Test Procedure *68*
3.30 Keyboard Access *71*
3.30.1 Purpose *71*
3.30.2 Inputs *71*
3.30.3 Expected Outputs & Pass/Fail Criteria *71*
3.30.4 Test Procedure *71*
3.31 Command Confirmation Time *72*
3.31.1 Purpose *72*
3.31.2 Inputs *72*
3.31.3 Expected Outputs & Pass/Fail Criteria *72*
3.31.4 Test Procedure *73*
3.32 Concurrent Learner Workload *73*
3.32.1 Purpose *73*
3.32.2 Inputs *73*
3.32.3 Expected Outputs & Pass/Fail Criteria *73*
3.32.4 Test Procedure *74*
3.33 Immediate Terminal Feedback *74*
3.33.1 Purpose *74*
3.33.2 Inputs *74*
3.33.3 Expected Outputs & Pass/Fail Criteria *74*
3.33.4 Test Procedure *74*
3.34 Home Load Time *75*
3.34.1 Purpose *75*

3.34.2 Inputs 75
3.34.3 Expected Outputs & Pass/Fail Criteria 75
3.34.4 Test Procedure 75
3.35 Database Query Time 75
3.35.1 Purpose 75
3.35.2 Inputs 75
3.35.3 Expected Outputs & Pass/Fail Criteria 75
3.35.4 Test Procedure 76
3.36 Level Start Time 76
3.36.1 Purpose 76
3.36.2 Inputs 76
3.36.3 Expected Outputs & Pass/Fail Criteria 76
3.36.4 Test Procedure 76
3.37 Cold Start Recovery 77
3.37.1 Purpose 77
3.37.2 Inputs 77
3.37.3 Expected Outputs & Pass/Fail Criteria 77
3.37.4 Test Procedure 77
3.38 AI Tutor Response Time 77
3.38.1 Purpose 77
3.38.2 Inputs 77
3.38.3 Expected Outputs & Pass/Fail Criteria 77
3.38.4 Test Procedure 77
Appendix (Test Logs) 78
A.1 Test Execution Log 78
A.2 Test Results 79
A.3 Incident Report 88
INC-01 Wrong password shows a session-expired message 90
INC-02 Level title overlaps the mode label in the workspace header 91
INC-03 Backend fails ruff lint and format checks 92
INC-04 Ask Git-It chat covers page content and required controls 93
INC-05 Unknown addresses show a developer error screen 94
INC-06 Deployment settings guard fails under the CI environment 94
INC-07 Ask Git-It labels every Home question as a scenario 95
INC-08 API returns HTML server errors when Redis is unreachable 96
INC-09 Stale frontend test blocks CI and production deploys 96

INC-10 Command confirmation, level start and concurrent learners miss their targets ... 97
Observations ... 98
A.4 Performance Measurements ... 98
A.5 Retest Measurements (Run 5) ... 102
A.6 Final Measurements (Run 6) ... 105

1. Introduction

1.1. System Overview

GIT it! is a scenario-driven web application for learning Git. Players climb the Arcane Spire, a story map of chapters and levels. Each level runs Git commands in a simulated terminal beside a live repository graph and a project-file editor, and the backend checks the resulting repository state before it awards stars, GitCoins and progress.

The frontend is a React and Vite single-page application. The backend is a Django REST Framework API that verifies commands, evaluates objectives, grants rewards, seeds the curriculum and stores progress. Other features include Challenge Gates, Squire's Drills, the Chapter Book, a companion shop (the Armory), rank and mastery, an admin console and an AI tutor called Ask Git-It.

1.2. Test Approach

This document records Run 3, a full regression test carried out on 21 October 2026 on commit 91d474b plus the uncommitted working-tree changes of that day. It replaces the partial scope of runs 1 and 2 (versions 1.0 to 1.2) with a test of every active feature. Three methods were used:

- **Automated regression:** the backend and frontend test suites, linters, the production build and the repository guard scripts, run locally with the same commands as the CI pipeline.
- **Browser end-to-end testing:** a scripted Chromium session drove the real user interface on a local build. Every Adventure scenario was solved with its authored solution as the test oracle, and each result was checked against the run and completion records saved by the backend.
- **Production smoke test:** public pages, one test learner account and read-only admin views of https://git-it-34.vercel.app were checked without changing production data.

All 21 Adventure levels were played through Easy, Medium and Hard: 63 tiers and 149 ordered scenarios passed with three stars. All 12 Challenge Gate trials then passed with three stars. The local backend used a disposable SQLite database, migrated and seeded with the full curriculum, kept separate from any shared database.

Every Run 3 test case passed. The working tree tested here was not deployed during this run, so the production smoke test (3.13) covers the version that was live on 21 October 2026.

Version 3.0 adds Run 4, a performance test of the production deployment carried out on 30 October 2026 on commit efe4059, the version live at the time. Run 4 measured the performance requirements NFR-PERF-01 to NFR-PERF-09 of the SRS (version 1.3) with three methods: a scripted workload of up to 40 virtual learners, timing in a Chromium browser, and database statistics from pg_stat_statements. Its test cases are 3.31 to 3.38 and its measurements are in Appendix A.4. Five cases passed and three failed; the failures are recorded as INC-10.

Version 3.1 adds Run 5, carried out on 30 October 2026 after two corrective actions for INC-10: the database moved to a Supabase project in Singapore, the API's region, and the API's threads were raised from 2 to 8 (commit 6e9ed3c). Run 5 repeated cases 3.31, 3.32, 3.35 and 3.36 with the same methods (Table 2-3, Appendix A.5). With 40 learners joining gradually, throughput and error rate now meet their targets, and command confirmation and level start are several times faster. Cases 3.31, 3.32 and 3.36 still fail, so INC-10 stays open.

Version 3.2 adds Run 6, the final performance test, carried out on 30 October 2026 after the remaining corrective actions for INC-10 (commit d65fe22 made the battle images smaller) and

against the performance targets as revised in SRS version 1.4. Run 6 repeated cases 3.31 to 3.36 and 3.38 (Table 2-4, Appendix A.6). All of them passed, so INC-10 is closed. Each test case reports its Actual Result against the acceptance criteria in its section 3.n.3, run by run.

The course requires evidence of functional, integration, system, regression and performance testing, defect logs with corrective actions and retest results, and procedures with expected and actual results and acceptance criteria. Table 1-1 shows where this document provides each.

Required evidence	**Where in this document**
Functional testing	Cases 3.5 to 3.12 and 3.14 to 3.30: each feature exercised in the browser against its expected outputs (Run 3).
Integration testing	Case 3.1 (API endpoints and the database through the backend suite), 3.4 (the generated API contract between backend and frontend), 3.13 (production: Vercel frontend, Render API, Supabase database and Redis together) and 3.14 (the tutor with its AI provider).
System testing	Case 3.13, cases 3.20 to 3.23 (all 63 Adventure tiers and 12 Challenge Gate trials played end to end) and the performance cases 3.31 to 3.38 on production.
Regression testing	Run 3 repeated cases 3.1 to 3.30 on the full working tree; the automated suites 3.1 to 3.4; Runs 5 and 6 repeated the affected cases after each INC-10 change.
Performance testing: workload, response times, throughput and error rate	Cases 3.31 to 3.38; Tables 2-2 to 2-4; Appendices A.4 to A.6. Table A-19 gives the final status of each performance requirement, and Table A-20 the targets SRS 1.4 revised and why.
Defect logs, corrective actions and retest results	Appendix A.3: the defect log (Table A-5) and the report of each incident INC-01 to INC-10, with its fix commit and retest.
Procedures, expected and actual results, acceptance criteria	Every case: 3.n.3 Expected Outputs & Pass/Fail Criteria (the acceptance criteria) followed by its Actual Result, and 3.n.4 Test Procedure. Table A-2 lists the actual result of every case.

Table 1-1. Testing and quality assurance coverage.

1.3. Definitions and Acronyms

- **STD:** Software Test Document.
- **DRF:** Django REST Framework.
- **CI:** continuous integration, the GitHub Actions workflow in .github/workflows/ci.yml.
- **E2E:** end-to-end test that drives the real user interface.
- **Tier:** one difficulty (Easy, Medium or Hard) of a level, made up of ordered scenarios.
- **Scenario:** one repository task inside a tier, with its own objectives and command budget.
- **Command budget:** the maximum number of counted commands in a scenario. Mistyped commands also count.
- **Replay:** a repeat attempt that does not change saved progress.
- **Challenge Gate:** the end-of-chapter test, with one trial per difficulty.
- **GitCoins:** in-game currency earned from rewards and spent in the shop.
- **Companion:** the playable character bought in the Armory.

- **Percentile (p50, p95, p99):** the time within which 50, 95 or 99 percent of requests finished. p50 is the median.
- **Throughput:** completed requests per second across all virtual learners.
- **Error rate:** the share of requests that failed: HTTP 5xx, HTTP 429 from the hosting platform, or no response within 30 seconds.
- **Virtual learner:** a script thread that signs in and plays like a learner, submitting a command about every 10 seconds and reloading Home data about every 60 seconds.
- **Cold start:** the first request after the free API host has gone to sleep.
- **Acceptance criteria:** the pass/fail criteria in section 3.n.3 of a test case. A case passes when its actual result meets all of them.
- **Actual result:** what a run observed, with measured values, and whether it met the acceptance criteria (PASS or FAIL).

2. Test Plan

2.1 Features to be Tested

- Backend services, covered by the automated suite: command verification, evaluation, rewards, tier runs, curriculum seeding, authentication and admin APIs.
- Frontend components, API wrappers and the production build, covered by the automated suite.
- Code quality, architecture boundaries, API contract, generated targets and deployment configuration, covered by the guard scripts.
- Sign in, sign out, registration and the complete password reset.
- First-time onboarding and the first companion purchase.
- Adventure play: terminal objectives, stars, exit, failure, retry, replay and progress persistence.
- Every tier of all 21 Adventure levels (Easy, Medium and Hard), including git add -p, conflict editing and git mergetool.
- All four Challenge Gates and their 12 trials.
- Repository diagrams, the Project Files panel and the workspace aids.
- Squire's Drill, chapter features (mastery, chests, Chapter Book, chapter selector, Git primer), dashboard views, the shop and loadout, and account settings.
- Access control, unknown addresses, layout at phone width (390 x 844) and keyboard access.
- The Ask Git-It AI tutor: general Git help and scenario answer protection.
- Admin console read views, locally and on production (read-only).
- Public pages and a signed-in smoke test of the production deployment.
- Performance of the production deployment against NFR-PERF-01 to NFR-PERF-09: command confirmation, terminal feedback, Home load, concurrent learners, database query time, level start, error rate, cold start and AI tutor response time (Run 4).

2.2 Features not to be Tested

- Content authoring: the admin Curriculum and Content pages and the Level Editor. These features are being removed from the product and were not part of this run.
- Admin actions that change data: manual GitCoin grants, account moderation and feature-flag changes. Admin views were checked read-only.
- Payments.
- Browsers other than Chromium, and screen-reader accessibility.
- CI-only jobs: Docker image builds, the PostgreSQL and Redis integration job, and dependency audits.
- Production testing of the working tree tested here, which was not deployed during this run.
- Performance on paid or larger hosting plans, and load beyond the point where the hosting platform rejected requests. Run 4's stepped test stopped itself after more than 30 failed requests in one minute to protect real users.
- Sign-in bursts faster than one learner every 15 s, such as 40 learners within one minute. SRS 1.4 places them outside the free tier's capacity (NFR-PERF-04); Runs 4 and 5 recorded them as Run A.

2.3 Testing Tools and Environment

Table 2-1 lists the tools and environment used for this run.

Item	Configuration
Code under test	Commit 91d474b plus the uncommitted working-tree changes of 21 October 2026
Platform	Windows 11; times in Philippine time (UTC+8)
Frontend	React and Vite development server on 127.0.0.1
Backend	Django REST Framework development server on 127.0.0.1, console email backend
Database	Disposable SQLite database, migrated and seeded with the full curriculum
Browser	Chromium driven by a browser-automation script; viewports 1440 x 900 and 390 x 844
Test accounts	Disposable learner and staff accounts in the local database; one test learner on production
Automated tools	pytest, Ruff, Vitest, ESLint, Knip, TypeScript build and the repository guard scripts
Production	https://git-it-34.vercel.app (Vercel frontend, Render API)

Table 2-1. Tools and environment.

The local server handled requests one at a time to avoid SQLite write contention during browser play. Squire's Drill was tested with the browser's reduced-motion setting so that its moving answer choices could be selected reliably. Knip ran with KNIP_DISABLE_RAW_TRANSFER=1 after its default parser buffer ran out of memory on the test machine; its scan rules were unchanged.

Run 4 used the production deployment. Table 2-2 lists its environment.

Item	Configuration
Code under test	Production, commit efe4059, as reported by /api/health/ready/
Frontend	Vercel, https://git-it-34.vercel.app; requests to /api are passed on to the API
API	Render Free web service in Singapore: Docker, Gunicorn with 1 worker and 2 threads, 0.1 CPU and 512 MB of memory
Database	Supabase Free PostgreSQL through its connection pooler in Tokyo (ap-northeast-1)
Cache	Render Free Key Value (Redis)
Load generator	Python 3.13 script (loadtest.py) on one Windows 11 computer in the Philippines, over HTTPS through Vercel, one connection per virtual learner

Item	Configuration
Test accounts	40 learner accounts, perf01 to perf40 with @loadtest.invalid addresses, created in the production database for this run and deleted afterwards together with their 33 runs and 622 command steps
Browser timing	Chromium driven by agent-browser at 1440 x 900, signed in as perf01, browser cache warm
Database statistics	pg_stat_statements snapshots before and after each workload. Supabase's Query Performance report reads the same view.
Time	30 October 2026, 01:05 to about 04:10 Philippine time, outside class hours

Table 2-2. Performance test environment (Run 4).

Run 5 used the deployment of Table 2-2 with the changes in Table 2-3.

Item	Change for Run 5
Code under test	Production, commit 6e9ed3c, as reported by /api/health/ready/. It also includes pull request #88 (star scoring and verified alternative solutions), merged the same day.
API	Gunicorn with 1 worker and 8 threads (render.yaml, commit 6e9ed3c); otherwise as Table 2-2
Database	New Supabase Free project in Singapore (ap-southeast-1), the API's region, through its session pooler. Curriculum seeded with seed_all; learner data of the Tokyo project was not carried over.
Test accounts	perf01 to perf40 created again and deleted afterwards together with their 43 runs and 1,505 command steps
Other traffic	Real learners used production during Run 5; their sign-ups appear in the database statistics
Time	30 October 2026, 11:25 to 11:50 Philippine time

Table 2-3. Changes to the test environment for Run 5.

Run 6 used the deployment of Table 2-3 with the changes in Table 2-4.

Item	Change for Run 6
Code under test	API: commit 6e9ed3c on Render, as reported by /api/health/ready/. Frontend: commit d65fe22 on Vercel, which serves battle backgrounds as WebP and palette-optimized skill effect sheets.
Requirements	SRS version 1.4, which revised NFR-PERF-01, NFR-PERF-04, NFR-PERF-06 and NFR-PERF-07
Workload	One run of 40 learners joining one every 15 s, then 10 minutes at 40 (NFR-PERF-04). loadtest.py now refreshes an expired access token and retries, as the app does.

Item	Change for Run 6
Browser timing	Chromium driven by agent-browser, limited to 10 Mbps with 40 ms latency through the Chrome DevTools Protocol (throttle.mjs; measured 9.3 to 9.4 Mbps). The HTTP cache was cleared for the first-visit sample.
Test accounts	perf01 to perf40 created again and deleted afterwards together with their 82 runs and 6,415 command steps
Time	30 October 2026, 12:32 to 13:50 Philippine time

Table 2-4. Changes to the test environment for Run 6.

3. Test Cases

3.1 Backend Automated Regression Suite

3.1.1 Purpose

Confirm that backend behavior and the database migrations have not regressed.

3.1.2 Inputs

- Backend pytest suite with config.test_settings and a freshly created test database.
- Command: pytest -q --create-db -rfE
- Command: python manage.py makemigrations --check --dry-run

3.1.3 Expected Outputs & Pass/Fail Criteria

- Every test passes, with no failures, errors or skips.
- The migration check reports no changes.

Actual Result: PASS. 987 tests passed in 323 seconds, with no failures, errors or skips. The migration check reported no changes.

Evidence: pytest result file and console log..

3.1.4 Test Procedure

1. Activate the backend virtual environment.
2. In backend/, run the migration check.
3. Run the pytest command and record the pass, fail and error counts.

3.2 Backend Code Quality Gates

3.2.1 Purpose

Confirm that the backend and repository scripts pass the lint and format checks of the CI quality-gates job.

3.2.2 Inputs

- Command (backend/): ruff check .
- Command (backend/): ruff format --check .
- Command (root): ruff check and ruff format --check on scripts/ with backend/ruff.toml

3.2.3 Expected Outputs & Pass/Fail Criteria

- No lint errors are reported.
- No file would be reformatted.

Actual Result: PASS. All checks passed. The format checks covered 472 backend files and 84 repository scripts.

Evidence: quality check log..

3.2.4 Test Procedure

1. Run ruff check and ruff format --check in backend/.
2. Run both checks on scripts/ with the backend configuration.

3. Record each file reported.

3.3 Frontend Automated Suite and Build

3.3.1 Purpose

Confirm that frontend components, response contracts and outcome controls behave as expected and that the production bundle builds.

3.3.2 Inputs

- Command: vitest run
- Command: npm run lint (ESLint)
- Command: npm run lint:dead (Knip)
- Command: npm run build (TypeScript build and Vite bundle)

3.3.3 Expected Outputs & Pass/Fail Criteria

- Every Vitest test passes.
- ESLint and Knip report no problems.
- The build finishes without errors.

Actual Result: PASS. 1,210 tests in 162 files passed in 629 seconds. ESLint, Knip and the production build passed.

Evidence: Vitest result file and the ESLint, Knip and build logs..

3.3.4 Test Procedure

1. In frontend/, run each command in the order listed.
2. Record the exit code and summary of each.

3.4 Architecture, Contract and Deployment Guards

3.4.1 Purpose

Confirm that the repository guard scripts run by CI still pass. They cover architecture boundaries, the generated API contract, CSS, vocabulary, documentation, CI, typography, Django deployment settings, the Render blueprint and the generated curriculum targets.

3.4.2 Inputs

- The guard scripts under scripts/, run with the CI environment.
- The 399 authored variant solutions and their generated targets.

3.4.3 Expected Outputs & Pass/Fail Criteria

- Every guard script exits with code 0.
- Every authored solution reproduces its generated target and trajectory.

Actual Result: PASS. Every guard passed. All 399 authored variant solutions matched their generated targets and trajectories, and the whitespace check of the diff was clean.

Evidence: quality check log and generated-targets log..

3.4.4 Test Procedure

1. Set the CI environment variables.
2. Run each guard script and record its exit code.

3. Replay the generated variant solutions.

3.5 Sign In and Sign Out

3.5.1 Purpose

Confirm that players can sign in and out, that wrong credentials are rejected clearly, and that private pages need a session.

3.5.2 Inputs

- A registered local learner account.
- An unregistered username with any password.

3.5.3 Expected Outputs & Pass/Fail Criteria

- A wrong username or password shows an inline error and keeps the user on the sign-in page.
- Valid credentials, by username or by email, open the story map.
- After logout, opening a private page redirects to /login.

Actual Result: PASS. A wrong password showed "Incorrect username/email or password" under the form. Signing in by email opened the story map. Logout returned to /login, and opening /home afterwards redirected to /login. Learner and staff accounts were also signed in and out through the real forms during the run.

Evidence: Figures 1 to 3.

3.5.4 Test Procedure

1. Open /login and sign in with an unregistered username.
2. Sign in with the test account's email and password.
3. Open the account menu and select Logout.
4. Open /home directly.

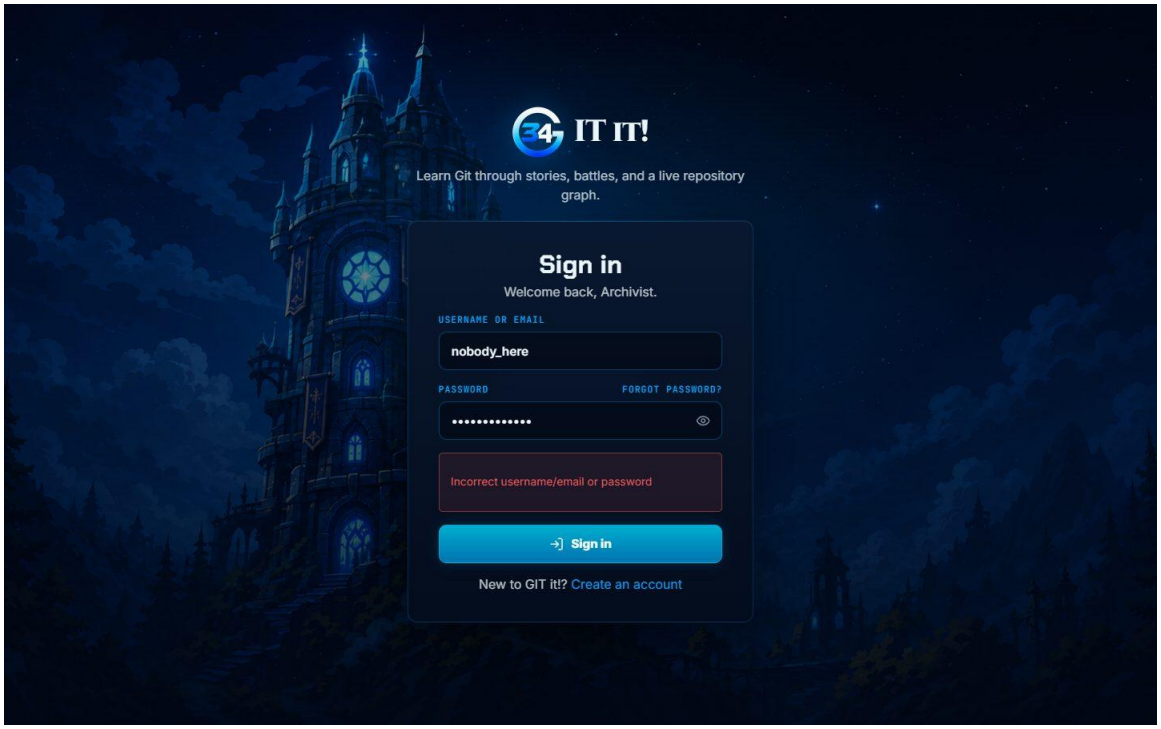


Figure 1. A wrong password shows an inline error on the sign-in form.

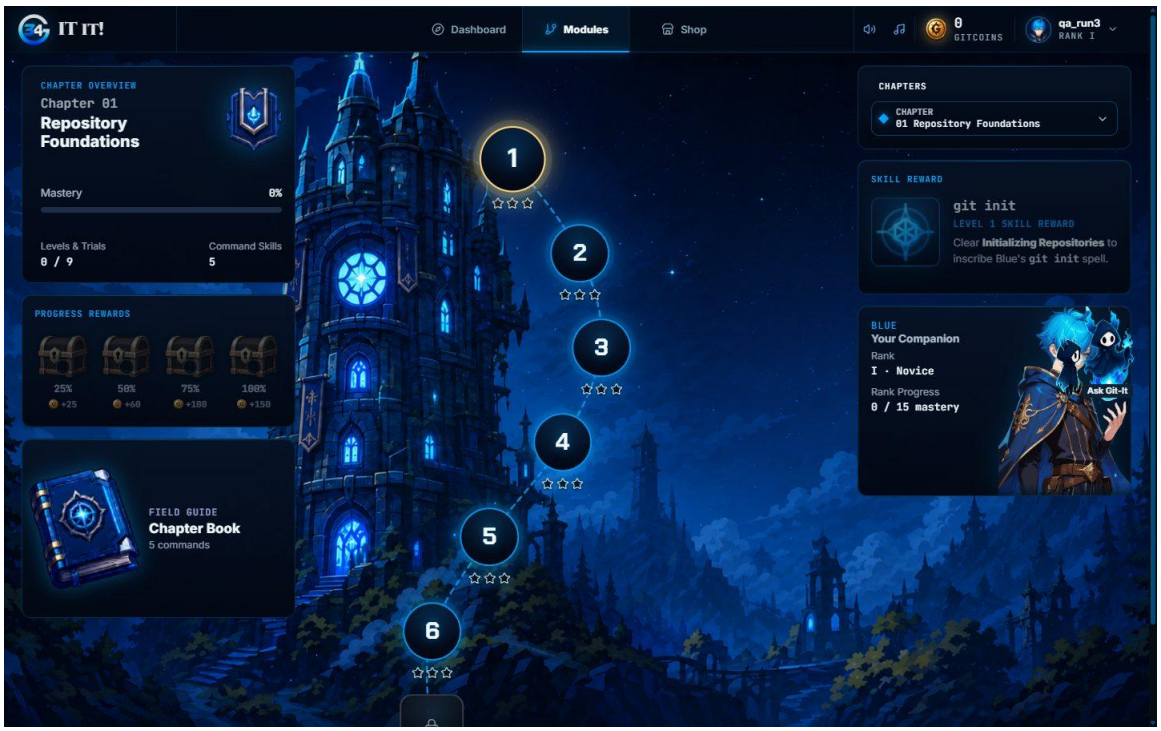


Figure 2. Signing in by email opens the story map.

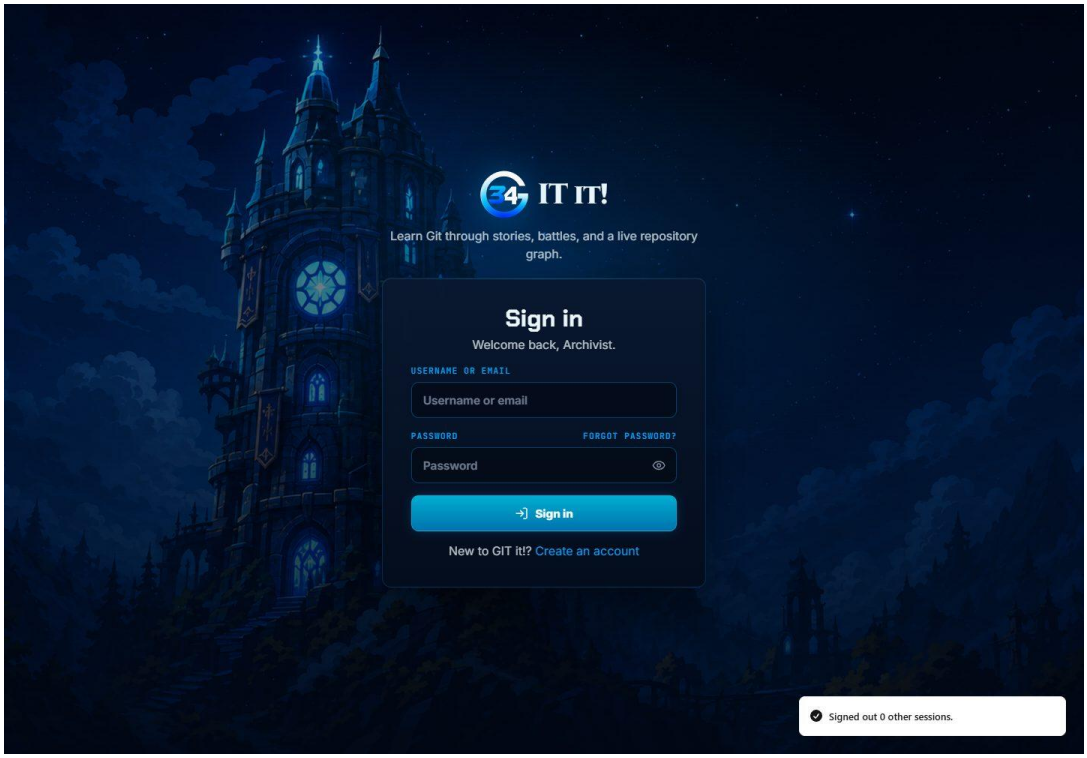


Figure 3. After signing out, the sign-in page is shown.

3.6 Account Registration

3.6.1 Purpose

Confirm that a new player can create an account and that invalid details are rejected.

3.6.2 Inputs

- An invalid email, a three-character password and a mismatched confirmation.
- A new username, email and valid password.

3.6.3 Expected Outputs & Pass/Fail Criteria

- Each invalid field shows its own message and no account is created.
- Valid details create the account, sign the player in and open onboarding with the starting GitCoins.

Actual Result: PASS. The form showed "Enter a valid email address.", "Use at least eight characters." and "Passwords do not match." With valid details the account was created and the story map opened with 150 GitCoins and the learning-path choice.

Evidence: Figures 4 to 5.

3.6.4 Test Procedure

1. Open /register and submit the invalid details.
2. Correct the fields and submit again.
3. Check the landing page and the GitCoin balance.

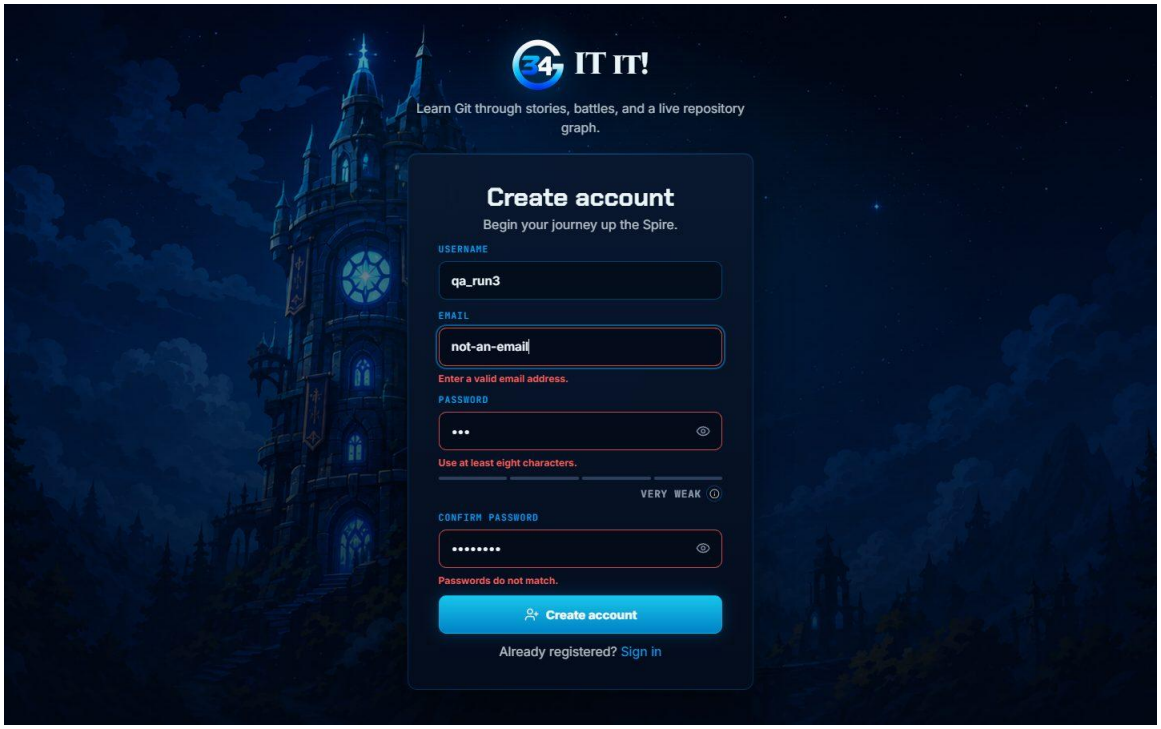


Figure 4. Registration rejects an invalid email, a short password and a mismatched confirmation.

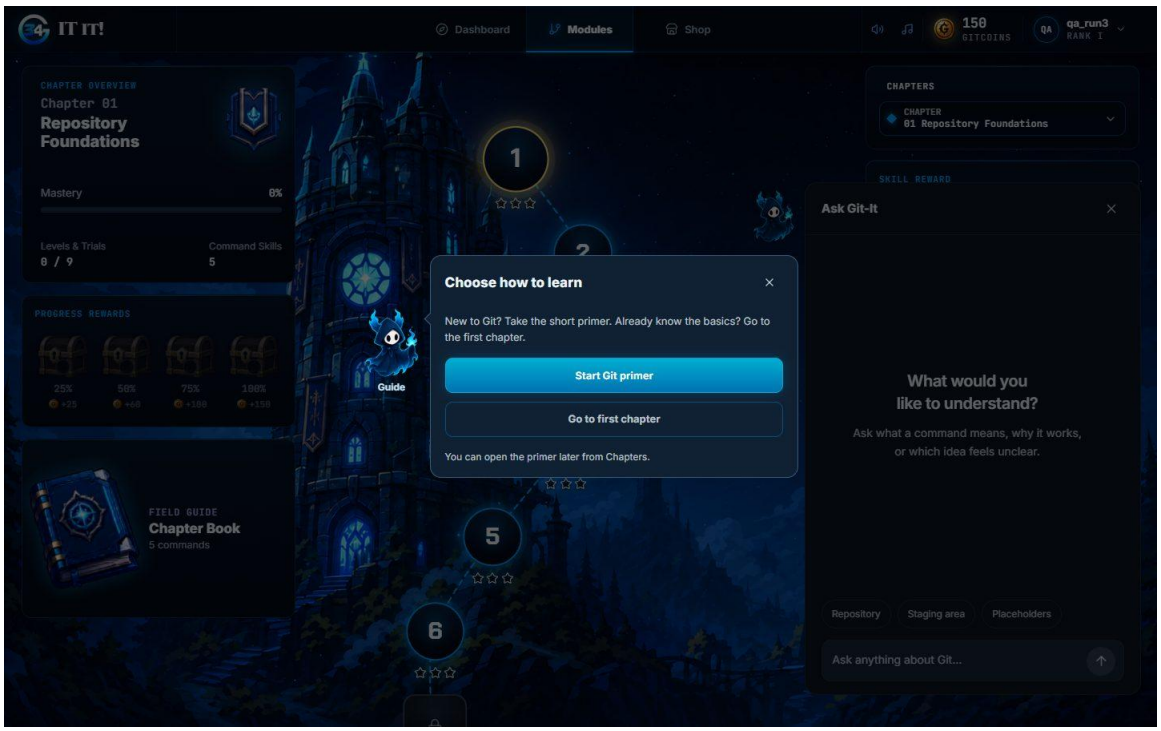


Figure 5. A new account lands on the story map with the learning-path choice.

3.7 Password Reset

3.7.1 Purpose

Confirm the complete password reset: request, email link, new password and sign-in.

3.7.2 Inputs

- A registered local account.
- The reset email printed by the console email backend.

3.7.3 Expected Outputs & Pass/Fail Criteria

- The request shows the same neutral confirmation whether or not the account exists.
- The emailed link opens a form that accepts a new password.
- The old password stops working and the new password signs in.

Actual Result: PASS. The request showed "Check your email". The emailed link set the new password and showed "Password updated". The old password was refused and the new password signed in.

Evidence: Figures 6 to 8.

3.7.4 Test Procedure

1. Open /forgot-password, enter the account email and send the link.
2. Open the reset link from the console email.
3. Enter and confirm a new password.
4. Try to sign in with the old password, then with the new one.

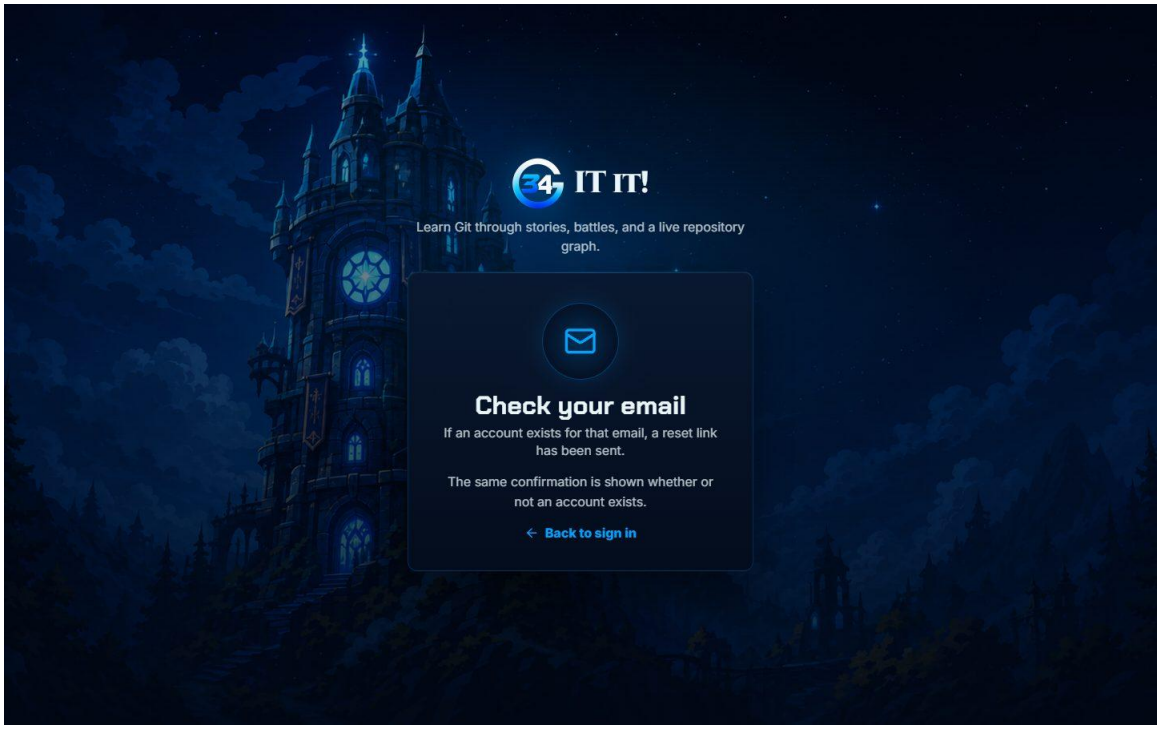


Figure 6. The reset request shows a neutral confirmation.

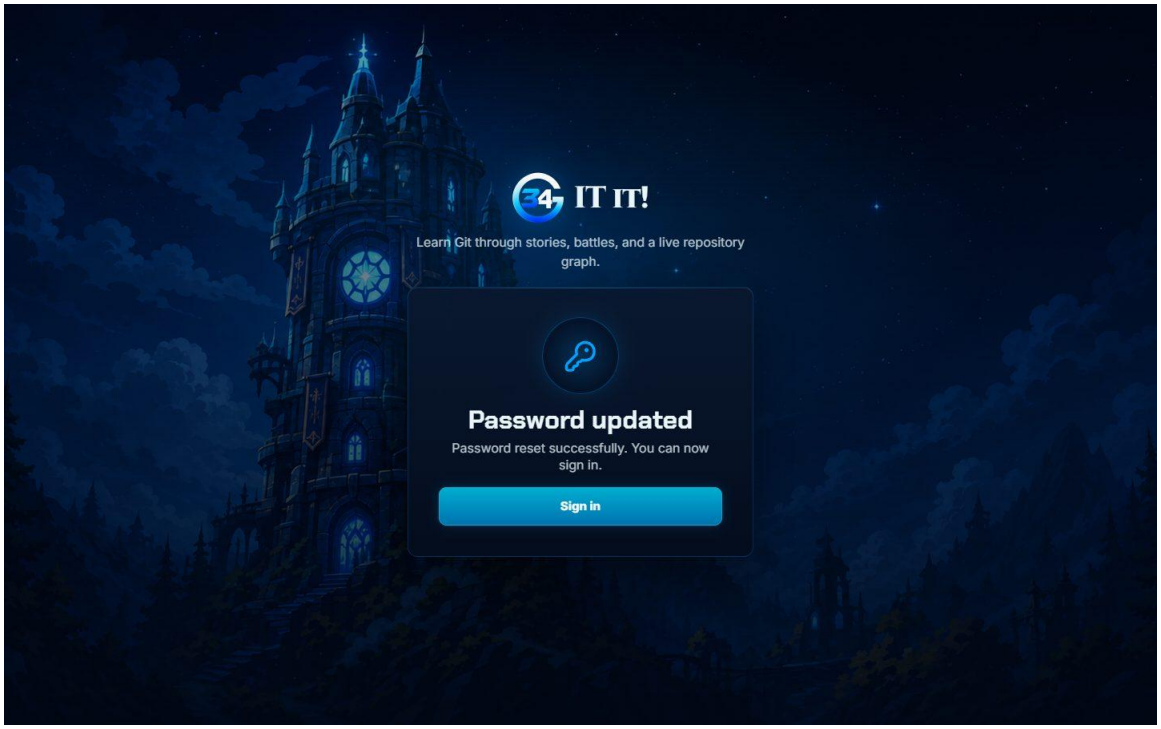


Figure 7. The emailed link sets the new password.

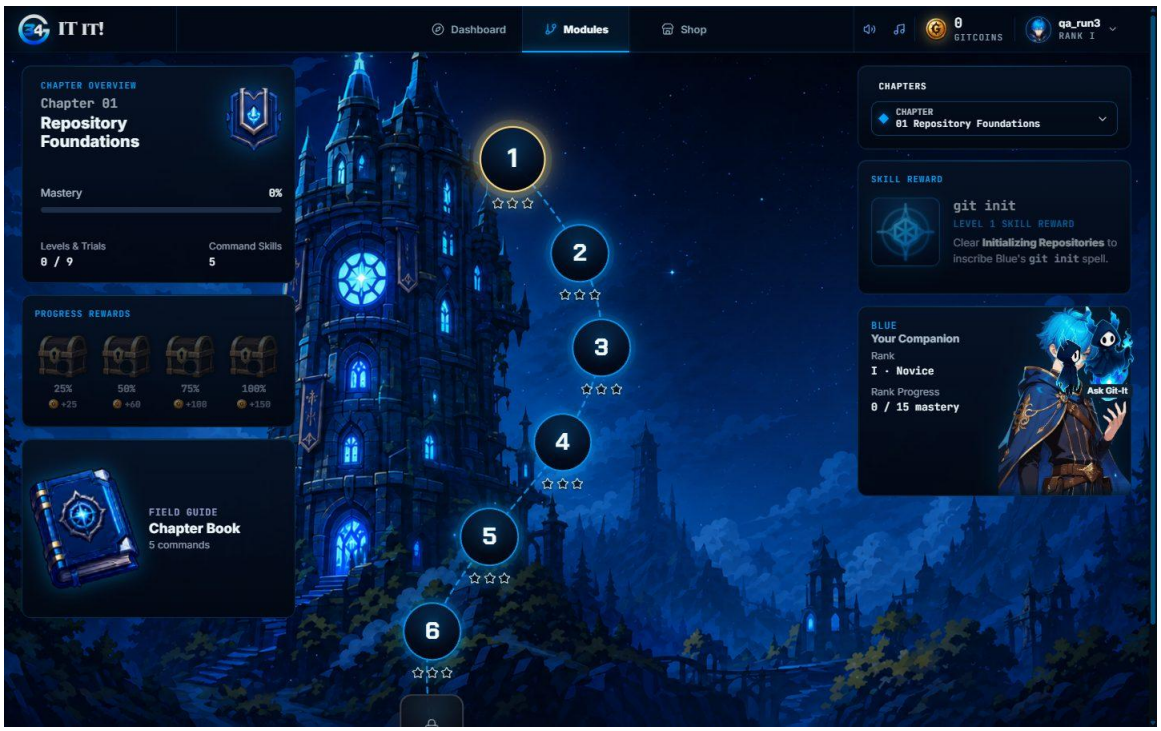


Figure 8. The new password signs in to the story map.

3.8 Onboarding and Companion Purchase

3.8.1 Purpose

Confirm the first-time journey: learning-path choice, guided tours and the first companion purchase.

3.8.2 Inputs

- A newly registered learner with 150 GitCoins.
- The guided tours and the Armory.

3.8.3 Expected Outputs & Pass/Fail Criteria

- Starting a level without a companion leads to the Armory.
- Buying the first companion takes its price from the balance and equips it.

Actual Result: PASS. Starting Level 1 without a companion opened the Armory with "Choose a companion". Buying Blue took the balance from 150 to 0 and equipped it. Buying a second companion later and keeping it equipped after a reload also passed (case 3.27).

Evidence: Figures 9 to 10.

3.8.4 Test Procedure

1. Choose Go to first chapter and follow or skip the tours.
2. Start Level 1, Easy tier.
3. In the Armory, purchase the companion.
4. Check the balance and the equipped companion.

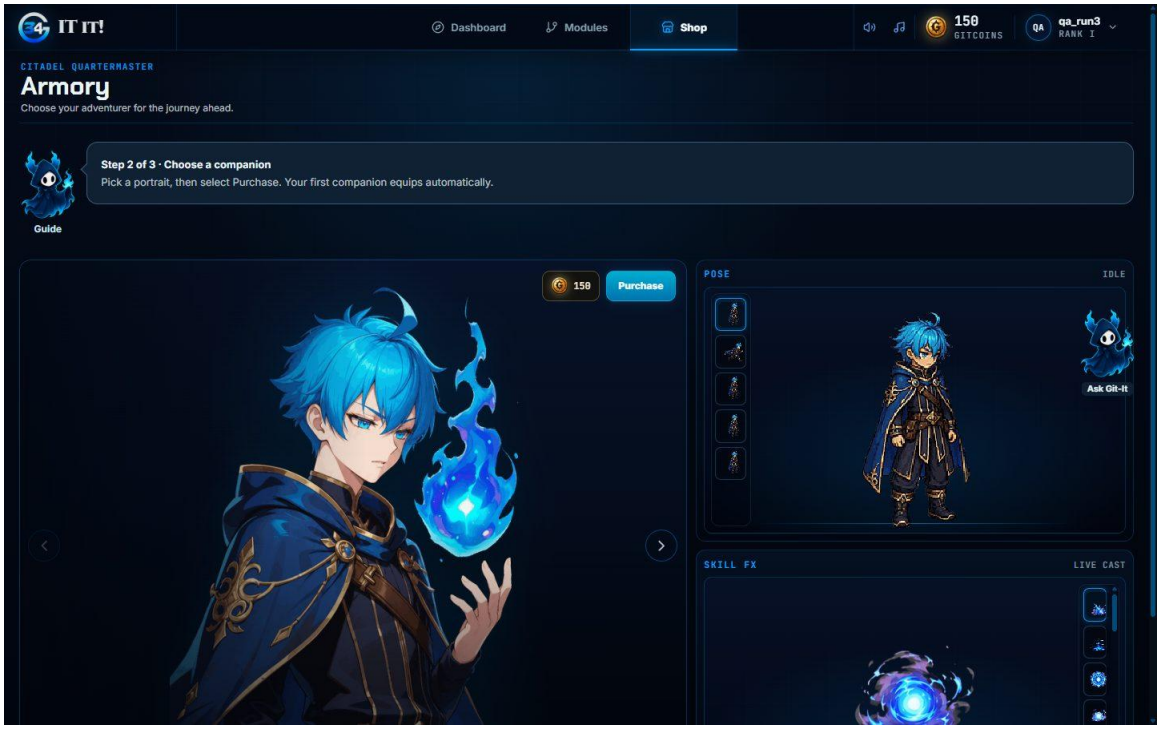


Figure 9. Starting a level without a companion opens the Armory.

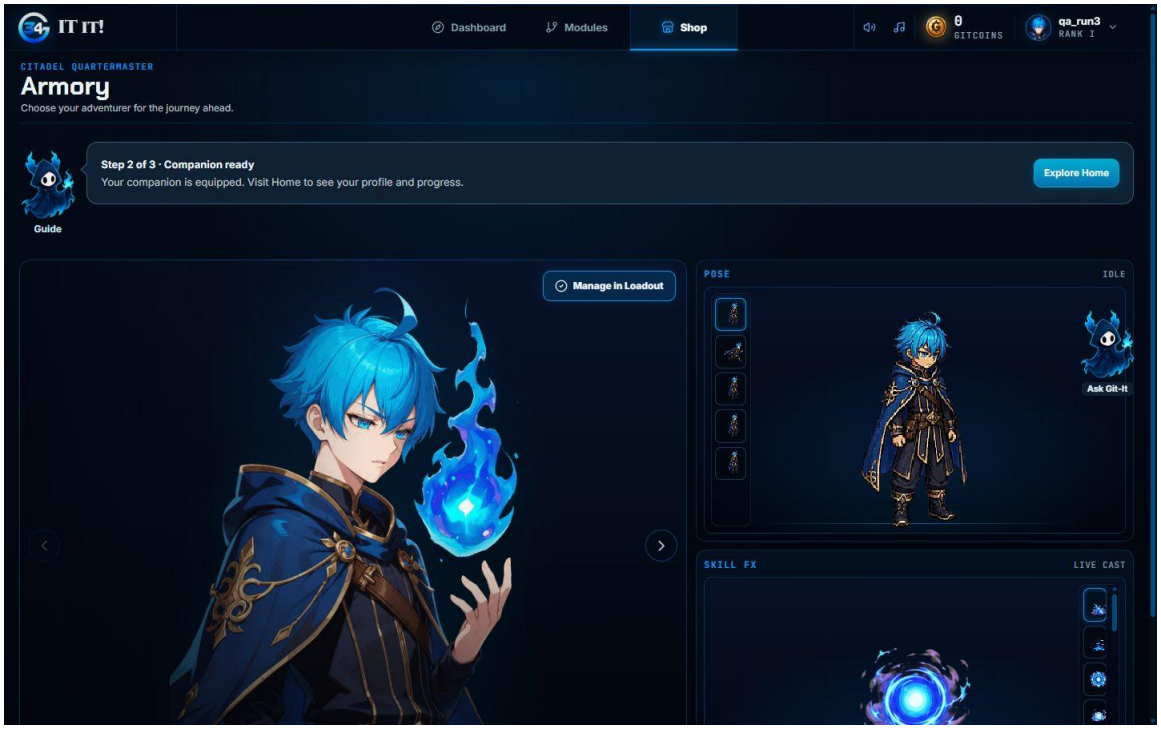


Figure 10. The first companion is bought and equipped.

3.9 Adventure Terminal Objectives and Exit

3.9.1 Purpose

Confirm the main Adventure interaction and that leaving a level asks for confirmation.

3.9.2 Inputs

- Level 1, Initializing Repositories, Easy tier.
- The authored solution of each scenario.

3.9.3 Expected Outputs & Pass/Fail Criteria

- Commands update the objectives, command count, stars, rewards and progress.
- Exit asks for confirmation; Escape keeps the run and confirming returns to the map.

Actual Result: PASS. All Level 1 Easy scenarios completed with three stars. The exit dialog opened, Escape kept the active run, and confirming Exit ended that run and returned to the map.

Evidence: Figures 11 to 13.

3.9.4 Test Procedure

1. Play Level 1 Easy with the authored commands and check each completion.
2. Start another run and select Exit.
3. Press Escape, then select Exit again and confirm.

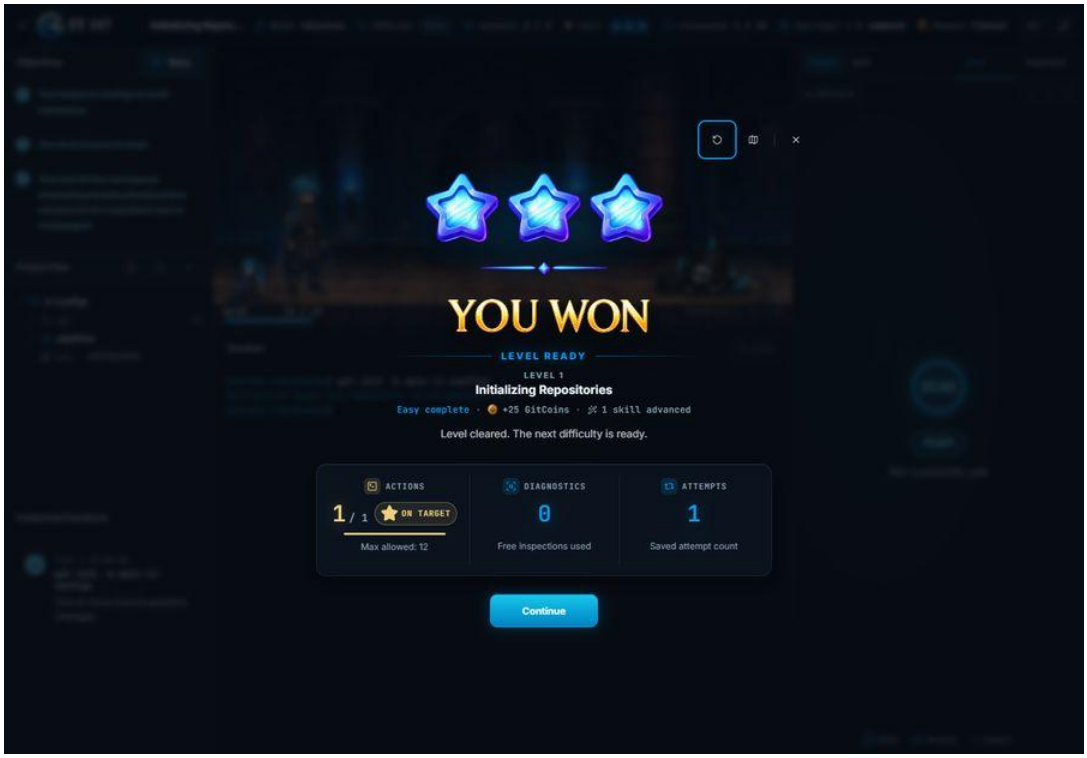


Figure 11. Level 1 Easy completed with three stars.

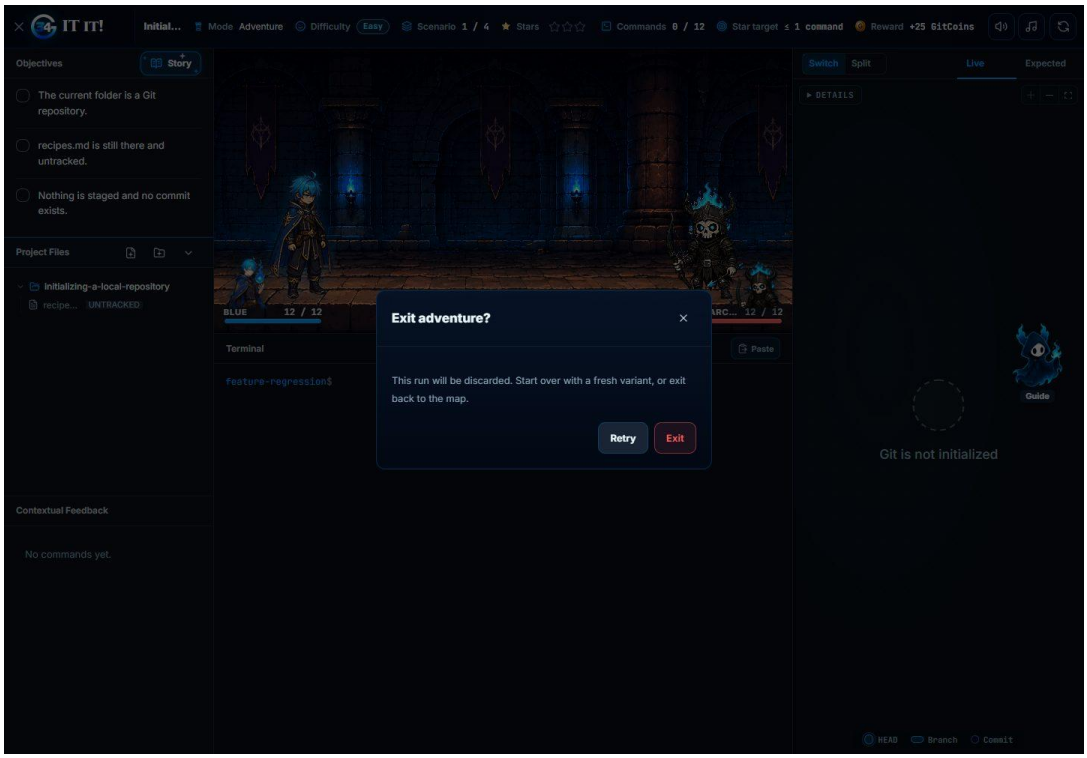


Figure 12. Exit asks for confirmation during an active run.

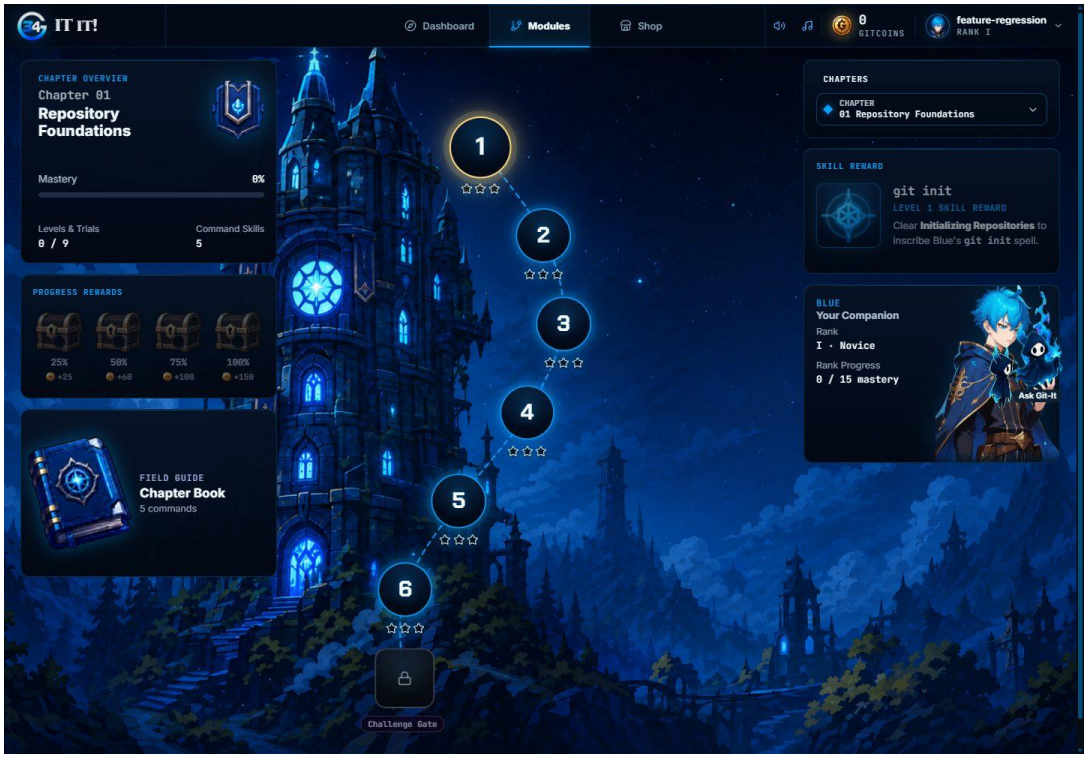


Figure 13. Confirming Exit returns to the chapter map.

3.10 Progress Persistence

3.10.1 Purpose

Confirm that earned progress survives a page reload.

3.10.2 Inputs

- The test learner after completing every Adventure tier and Challenge trial.

3.10.3 Expected Outputs & Pass/Fail Criteria

- After a reload, the dashboard and map show the progress saved by the backend.

Actual Result: PASS. After reload the dashboard showed 63 perfect clears and 63 cleared tiers. The saved records held all 63 Adventure tier completions and all 12 Challenge trial completions.

Evidence: Figure 14.

3.10.4 Test Procedure

1. Reload the dashboard and the chapter map.
2. Compare the displayed clears with the saved completion records.

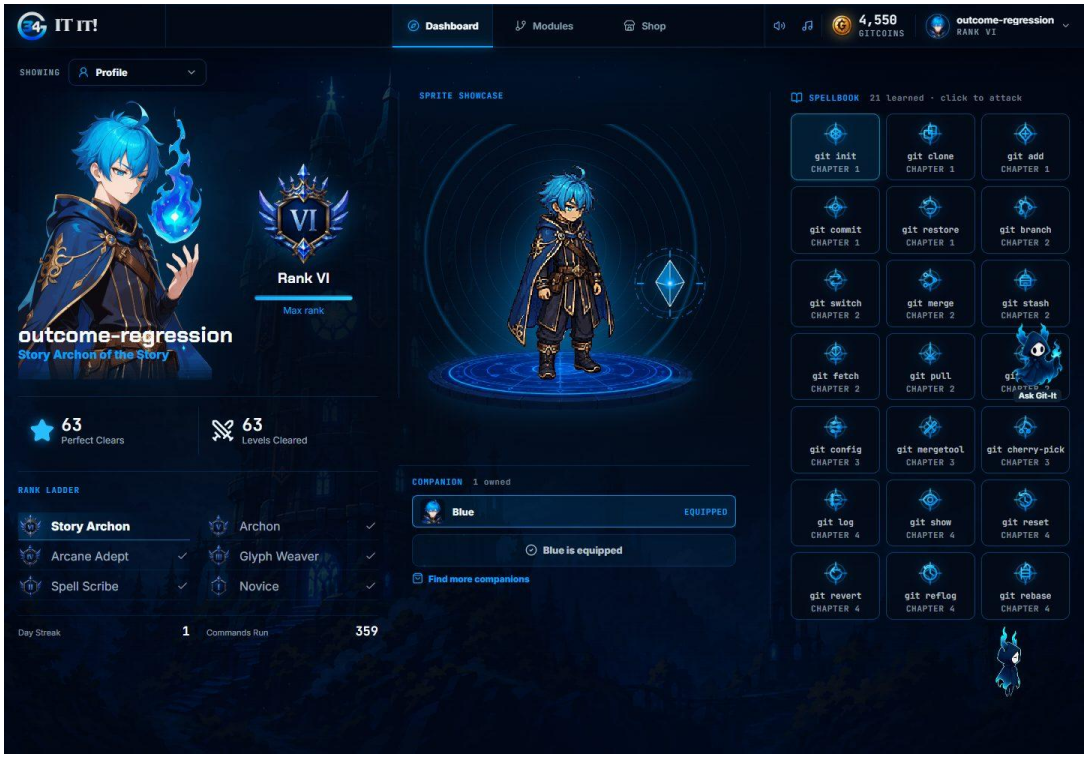


Figure 14. The dashboard after reload shows 63 perfect clears and 63 cleared tiers.

3.11 Access Control and Routing

3.11.1 Purpose

Confirm that private pages need a session, that learners cannot open the admin console, and that unknown addresses are handled.

3.11.2 Inputs

- A signed-out browser, a learner session and a staff session.
- /settings, /admin and an unregistered address.

3.11.3 Expected Outputs & Pass/Fail Criteria

- Signed out, private pages redirect to sign in.
- A learner opening /admin is returned to the app.
- An unknown address shows a recovery page.

Actual Result: PASS. Signed out, /settings redirected to sign in. The learner opening /admin was returned to the dashboard. The unknown address showed "Page not found" with a link back to the app.

Evidence: Figures 15 to 16.

3.11.4 Test Procedure

1. Sign out and open /settings.
2. Sign in as a learner and open /admin.
3. Open an unregistered address.

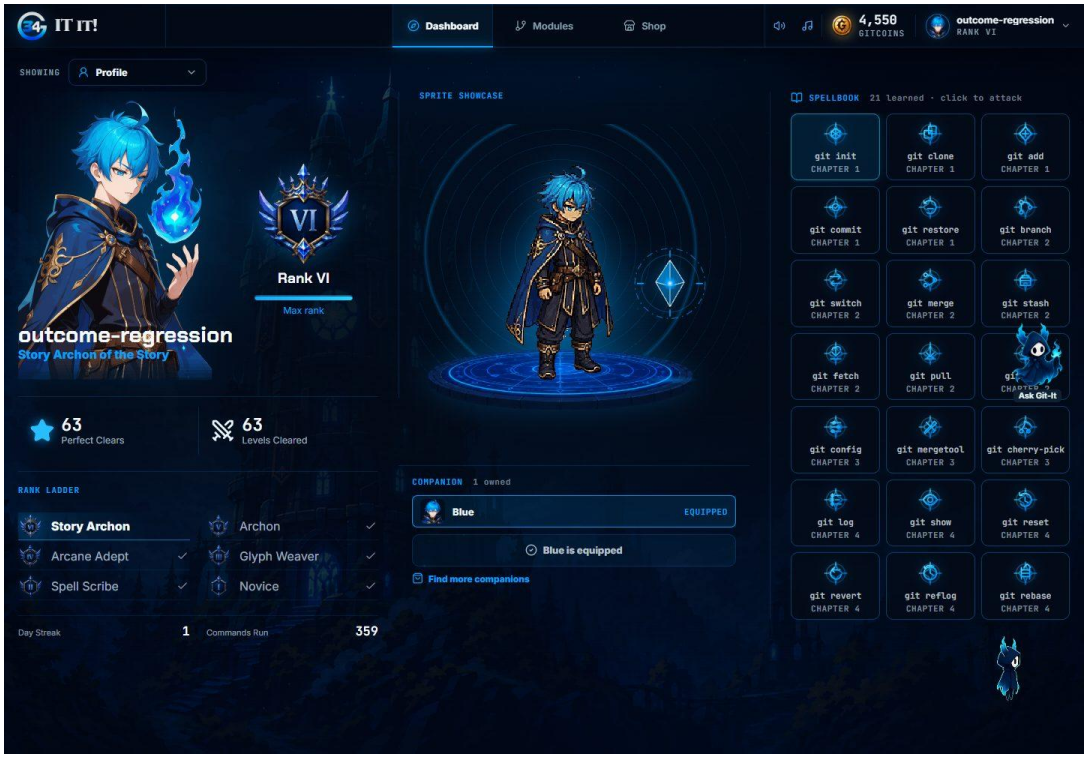


Figure 15. A learner opening /admin is returned to the dashboard.

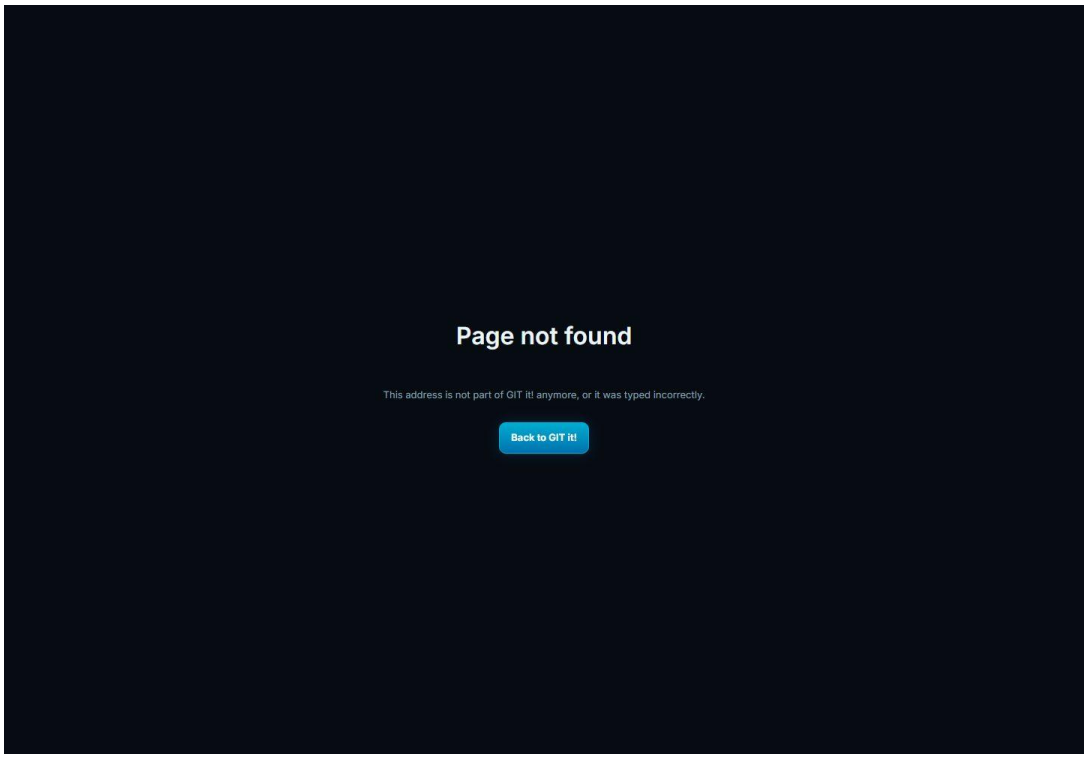


Figure 16. An unknown address shows the Page not found screen.

3.12 Responsive Layout at Phone Width

3.12.1 Purpose

Confirm that the main pages and the result screens fit a phone screen.

3.12.2 Inputs

- Chromium viewport of 390 x 844.
- The test learner with earned progress.

3.12.3 Expected Outputs & Pass/Fail Criteria

- Dashboard, map, shop, Settings and the level workspace have no page-wide horizontal scrolling.
- Result screens fit the viewport.

Actual Result: PASS. The dashboard, map, shop, Settings and level workspace fit the phone width. The Adventure failure, retry and completion screens and a Challenge failure screen fit the viewport.

Evidence: Figure 17.

3.12.4 Test Procedure

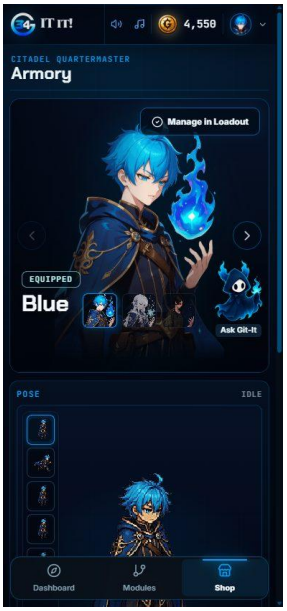
1. Set the viewport to 390 x 844.
2. Open each page and compare the document width with the viewport width.
3. Open a failed and a completed result screen.
4. Restore the desktop size.



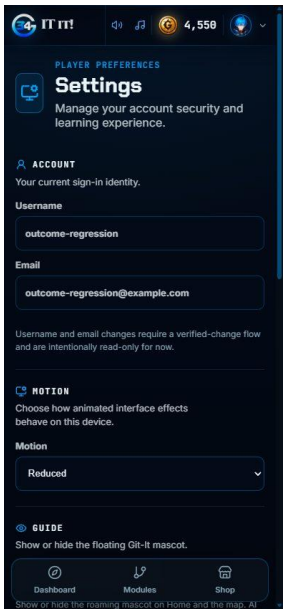
Dashboard



Chapter map



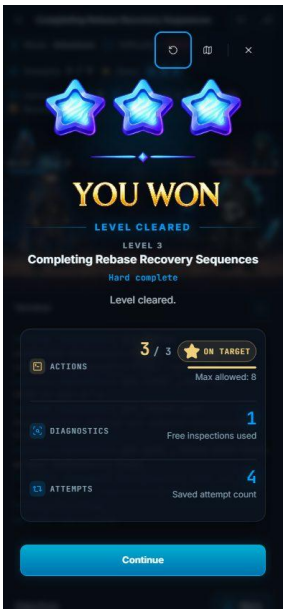
Armory



Settings



Failed run



Completed run

Figure 17. Pages and result screens at 390 x 844.

3.13 Production Smoke Test

3.13.1 Purpose

Confirm that the production deployment works for visitors, a signed-in learner and staff, without changing production data.

3.13.2 Inputs

- https://git-it-34.vercel.app
- One new test learner account.
- The existing admin account, used read-only.

3.13.3 Expected Outputs & Pass/Fail Criteria

- Public pages load and private pages redirect to sign in.
- A new learner can register, open Home and log out.
- Admin views load for staff, and a learner cannot open them.

Actual Result: PASS. The sign-in, registration, password reset and not-found pages loaded, and /home redirected to sign in. The test learner registered, opened Home and logged out. The admin KPI Overview, Learners, Economy, Analytics, Moderation and Settings views loaded and nothing was changed. The learner opening /admin/users was sent to Home. After the API host had been idle, the first visit showed "Restoring session" for about 30 seconds before the page loaded.

Evidence: Figures 18 to 23.

3.13.4 Test Procedure

1. Open the public pages and an unknown address.
2. Register the test learner, open Home and log out.
3. Sign in as staff and open each admin view without submitting anything.
4. Sign in as the learner and open /admin/users.

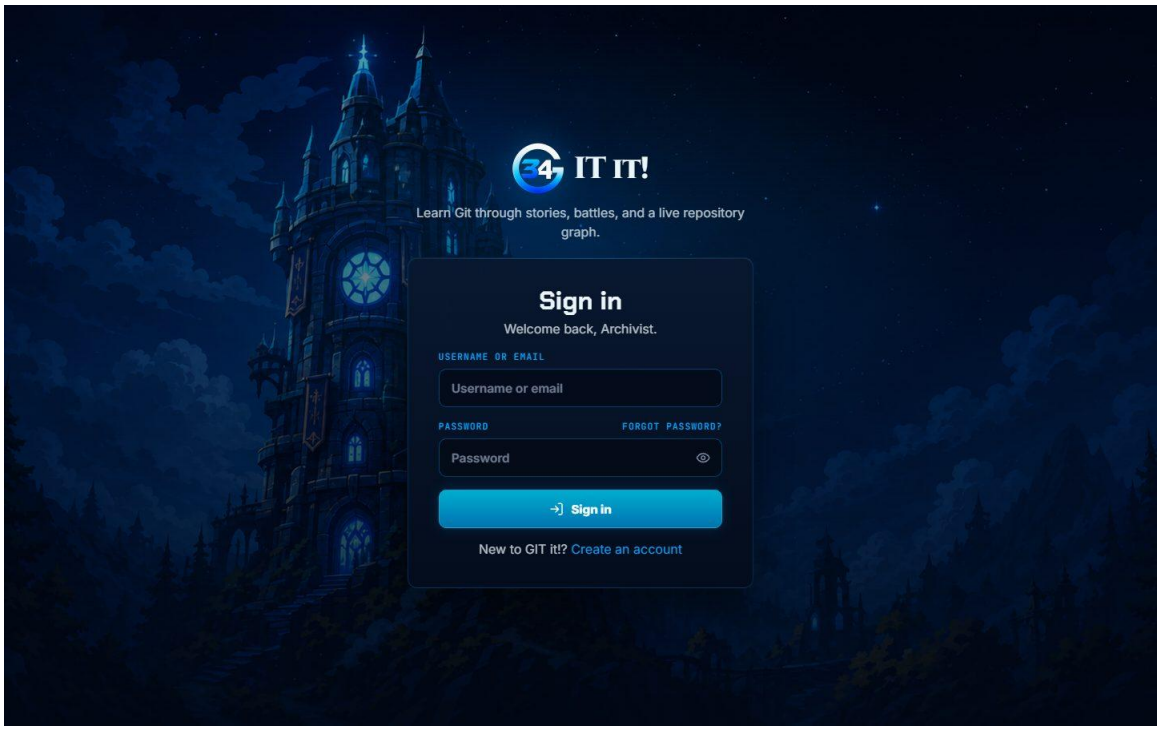


Figure 18. Production sign-in page.

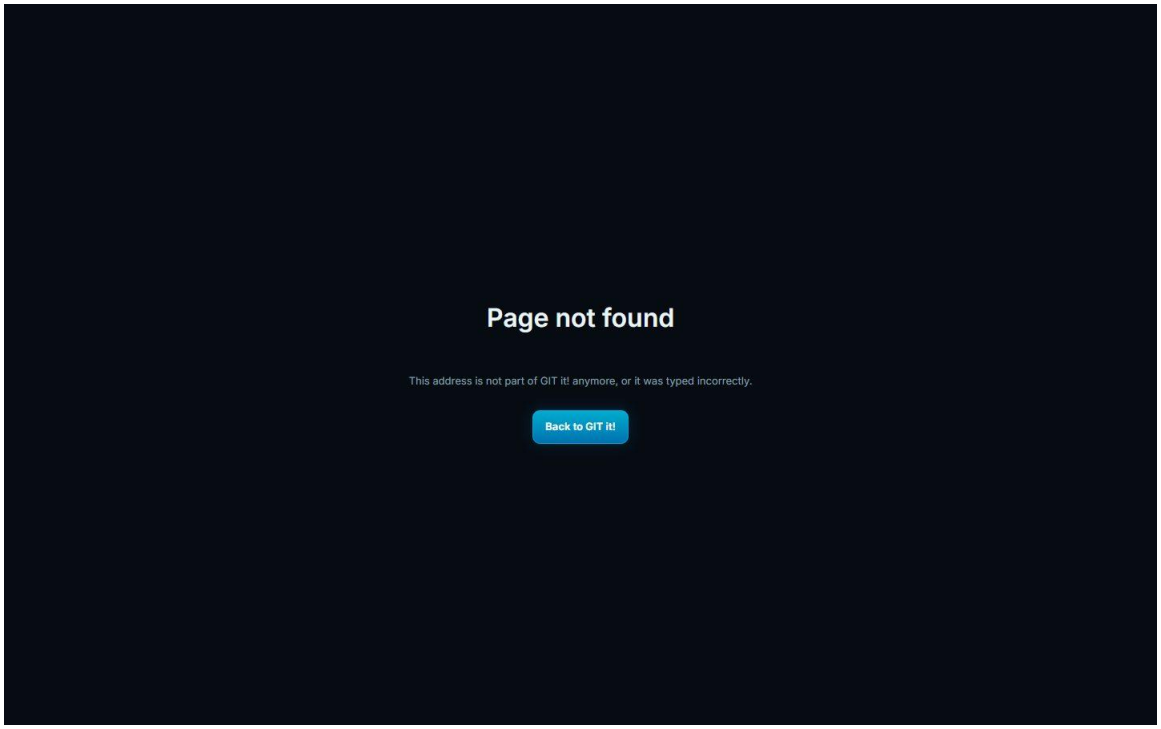


Figure 19. Production not-found page.

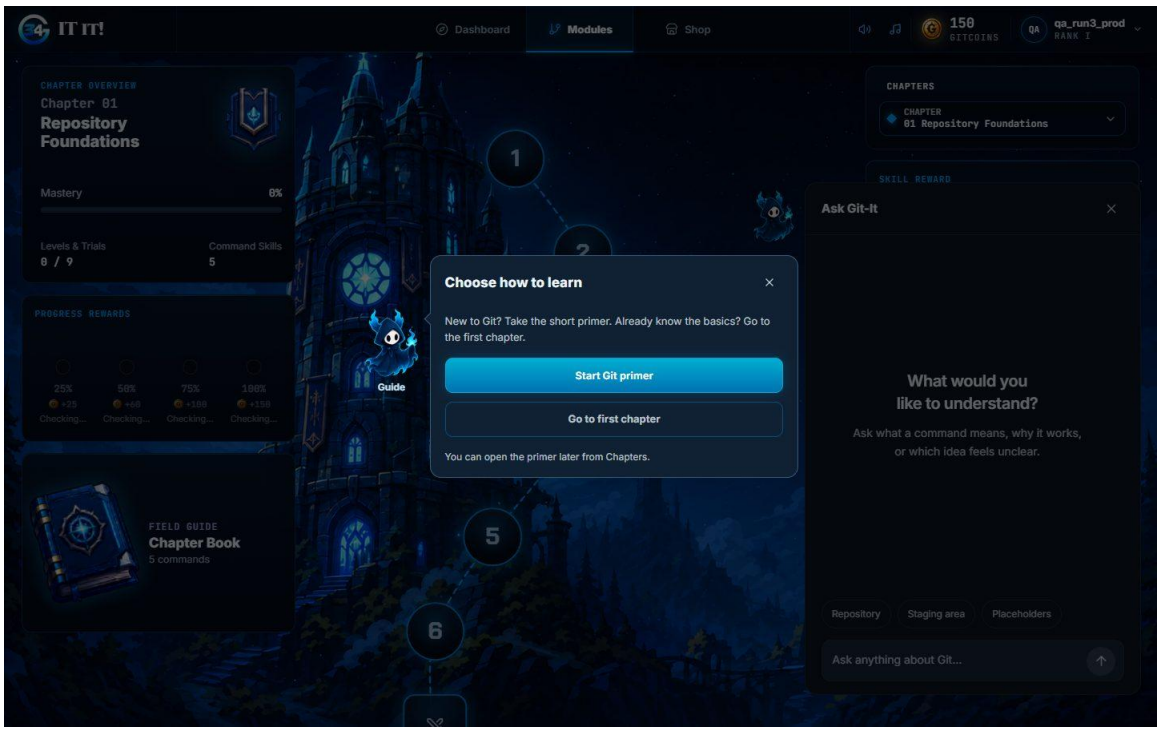


Figure 20. The production test learner after registering.

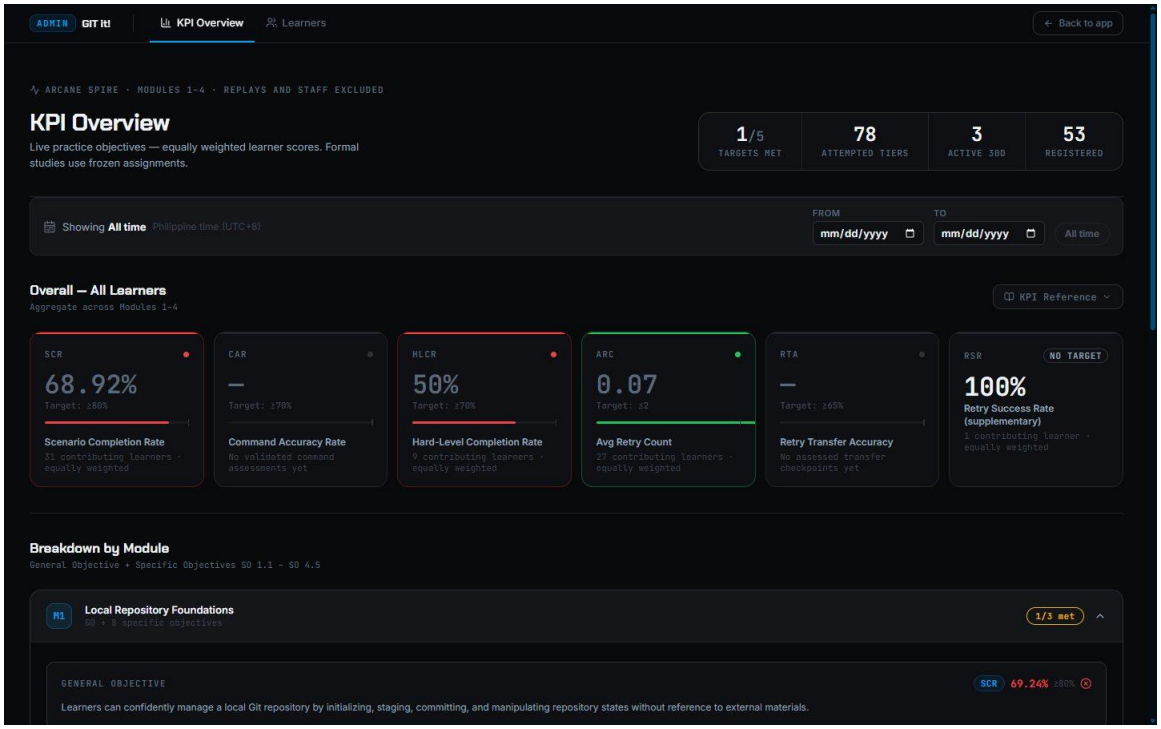


Figure 21. Production admin KPI Overview, viewed read-only.

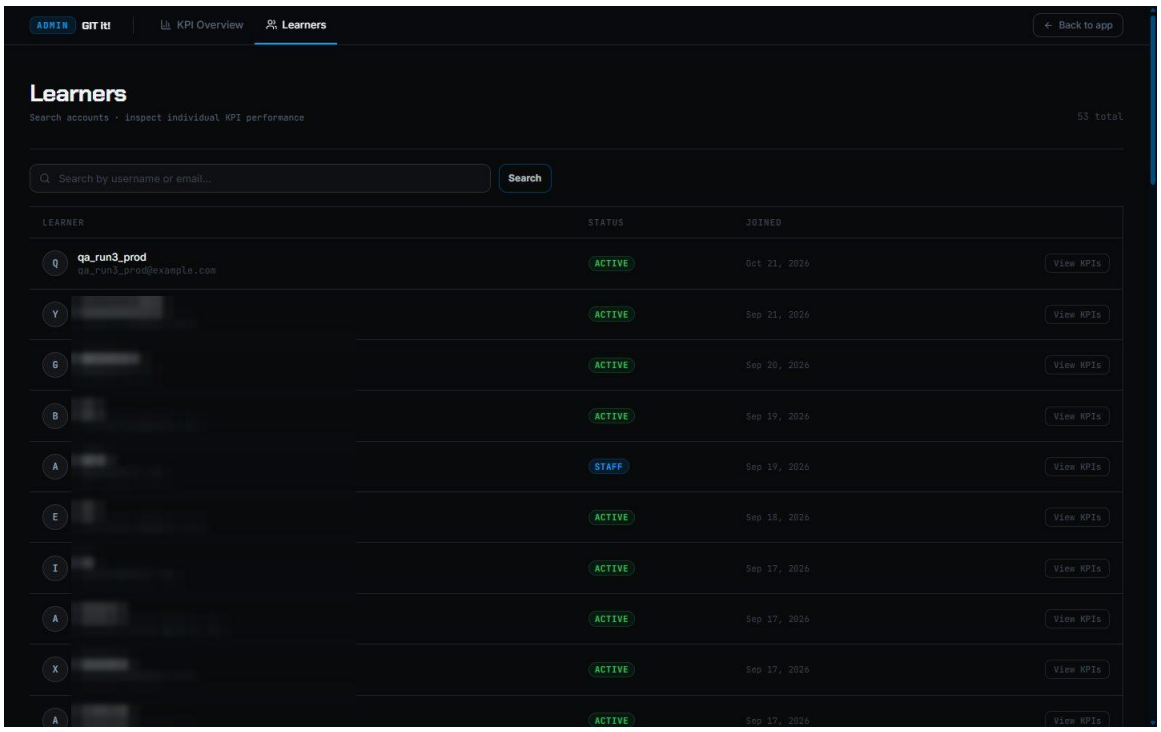


Figure 22. Production Learners list, viewed read-only. Other users' names and emails are blurred.

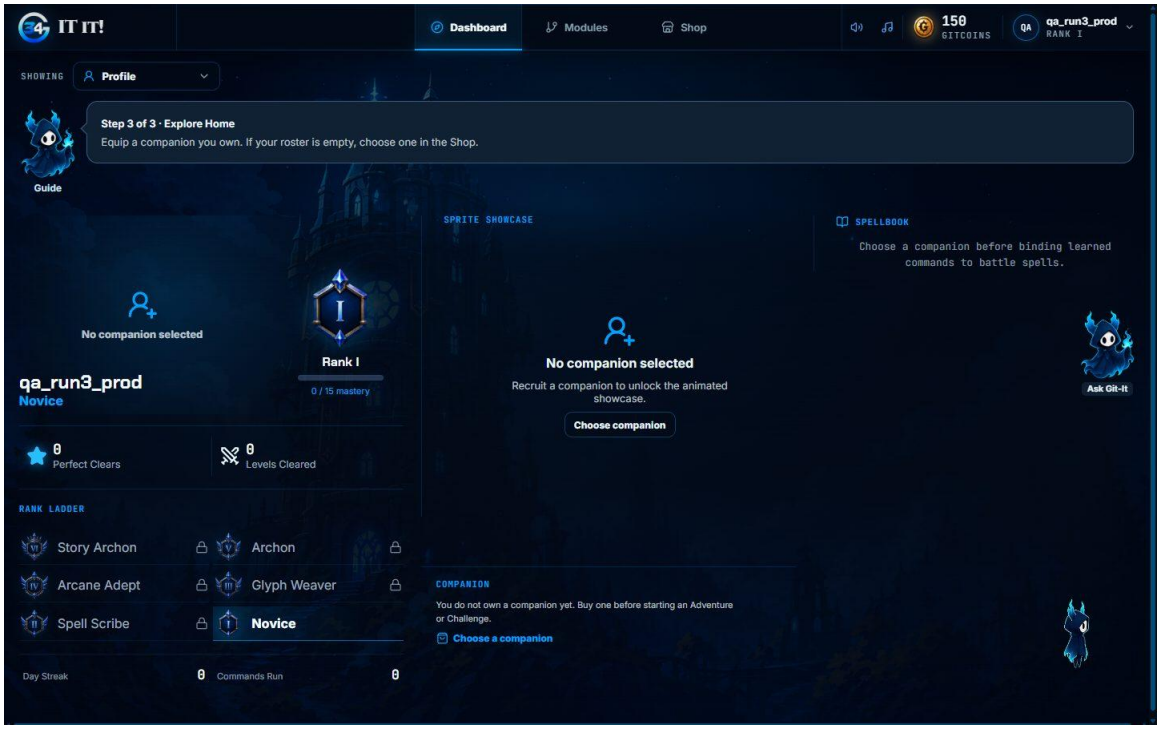


Figure 23. A learner opening /admin/users is sent to Home.

3.14 Ask Git-It: General Git Help

3.14.1 Purpose

Confirm that the tutor answers general Git questions, keeps context for follow-ups, and refuses unrelated or override requests.

3.14.2 Inputs

- Questions on Home: merge versus rebase; a follow-up about keeping history; a cake recipe; an instruction to ignore its rules.

3.14.3 Expected Outputs & Pass/Fail Criteria

- General answers are relevant and carry no scenario label.
- The follow-up uses the earlier context.
- Unrelated and override requests are refused.

Actual Result: PASS. The tutor explained the difference between merge and rebase, answered the follow-up in context, declined the recipe and refused the override request.

Evidence: Figures 24 to 27.

3.14.4 Test Procedure

1. Open Ask Git-It on Home.
2. Send each question and wait for the reply.
3. Check each reply against the expected behavior.

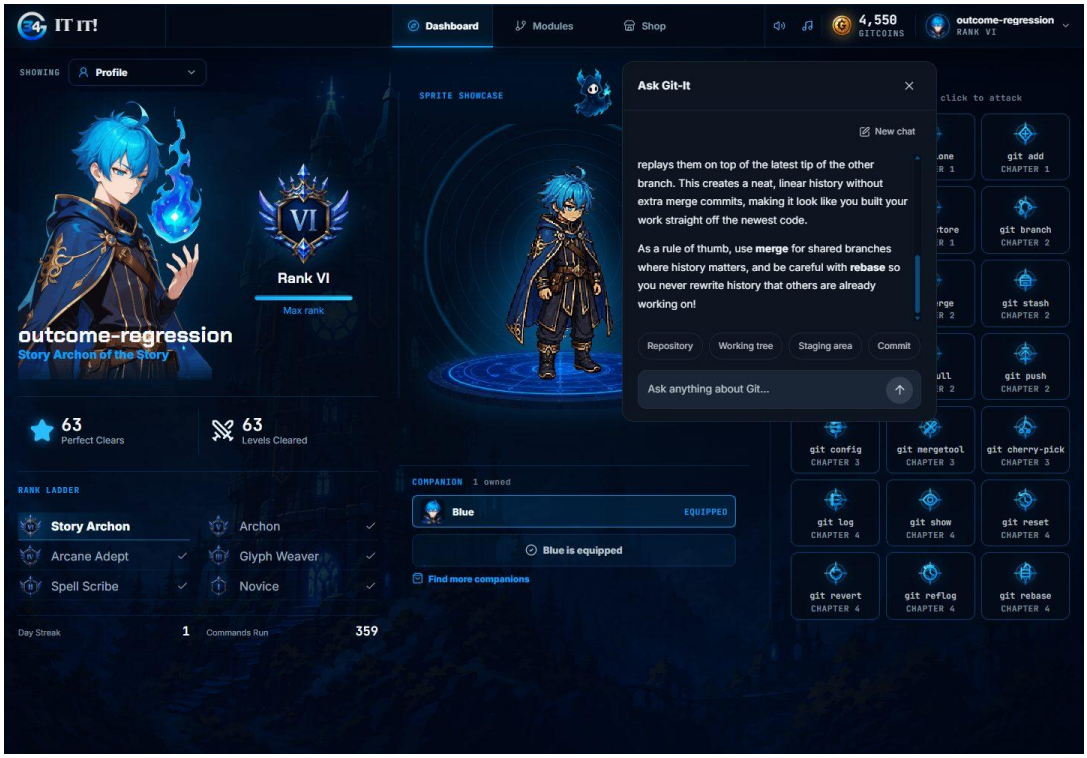


Figure 24. A general question: merge versus rebase.

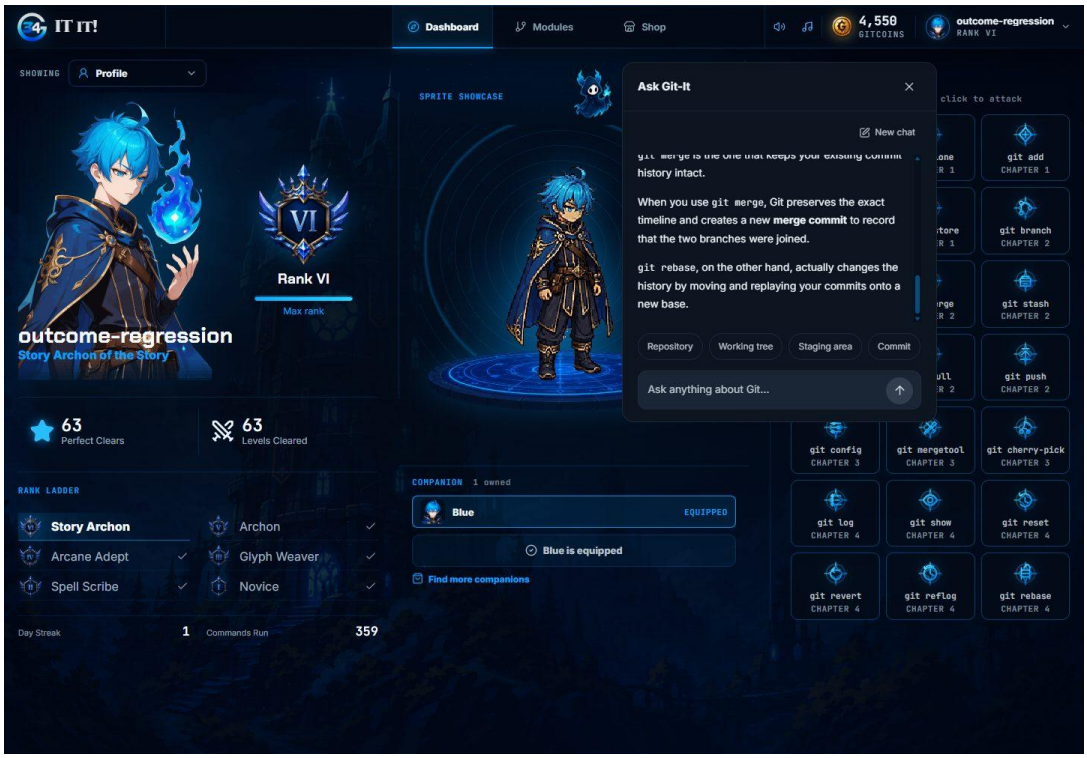


Figure 25. The follow-up is answered using the earlier context.

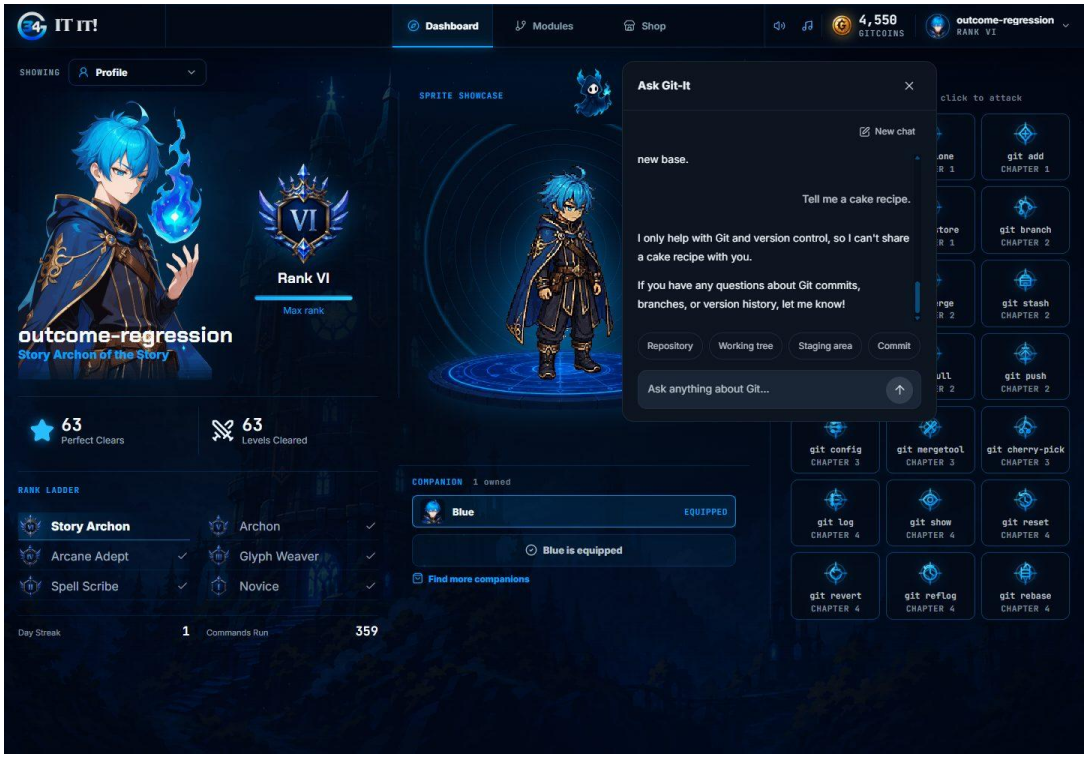


Figure 26. An unrelated request is declined.

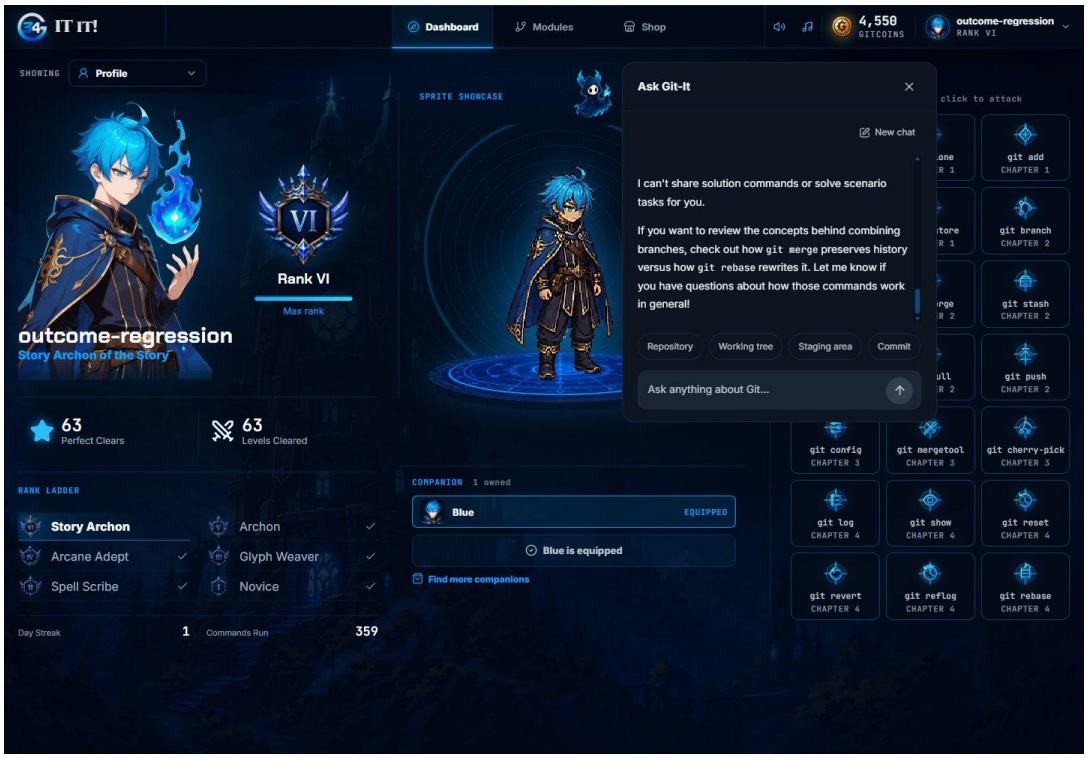


Figure 27. An instruction-override request is refused.

3.15 Ask Git-It: Scenario Answer Protection

3.15.1 Purpose

Confirm that the tutor recognizes scenario tasks and teaches instead of giving the exact answer.

3.15.2 Inputs

- A scenario task pasted on Home.
- A request for the exact commands inside a beginner Adventure level.

3.15.3 Expected Outputs & Pass/Fail Criteria

- Home marks the question as a scenario and withholds the exact commands.
- Inside a level the tutor does not solve the task for the player.

Actual Result: PASS. On Home the reply was marked "Scenario spotted" and did not give the exact command. Inside the level the tutor refused to solve the task; it explained git init, which the level's command guide already teaches (see A.3).

Evidence: Figures 28 to 29.

3.15.4 Test Procedure

1. On Home, ask for the exact answer to a repository-initialization task.
2. Start a beginner level and ask the tutor for the exact commands and values.
3. Check both replies.

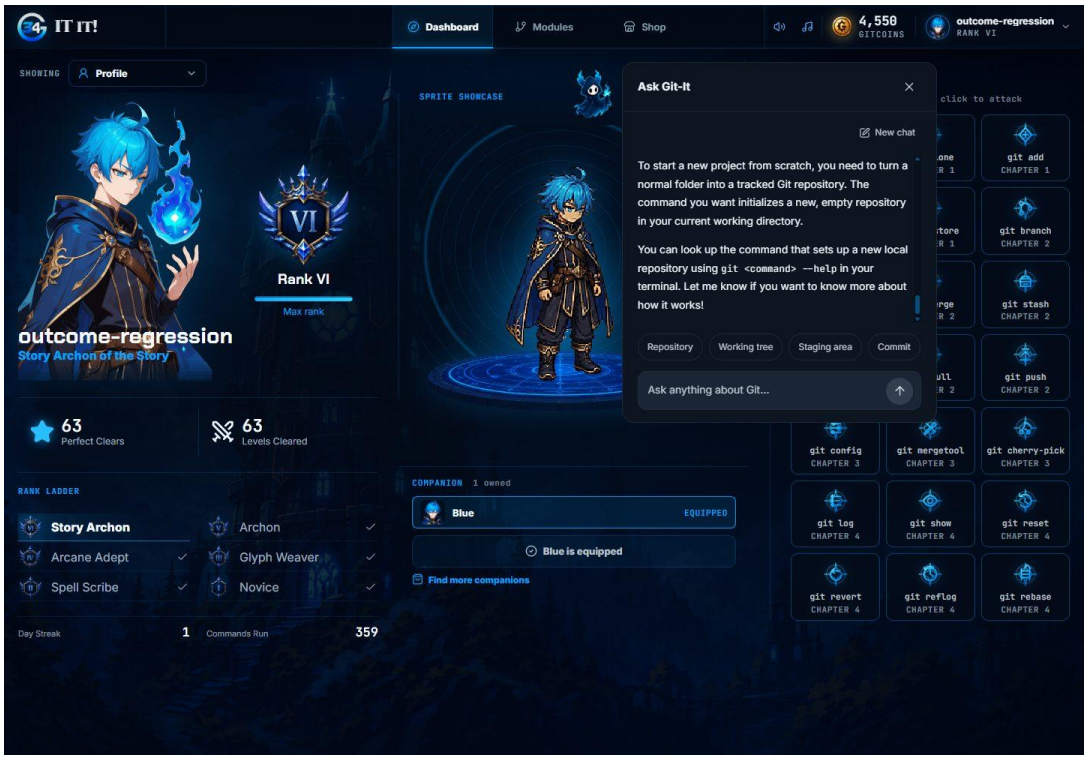


Figure 28. On Home, the tutor recognizes a scenario and withholds the exact command.

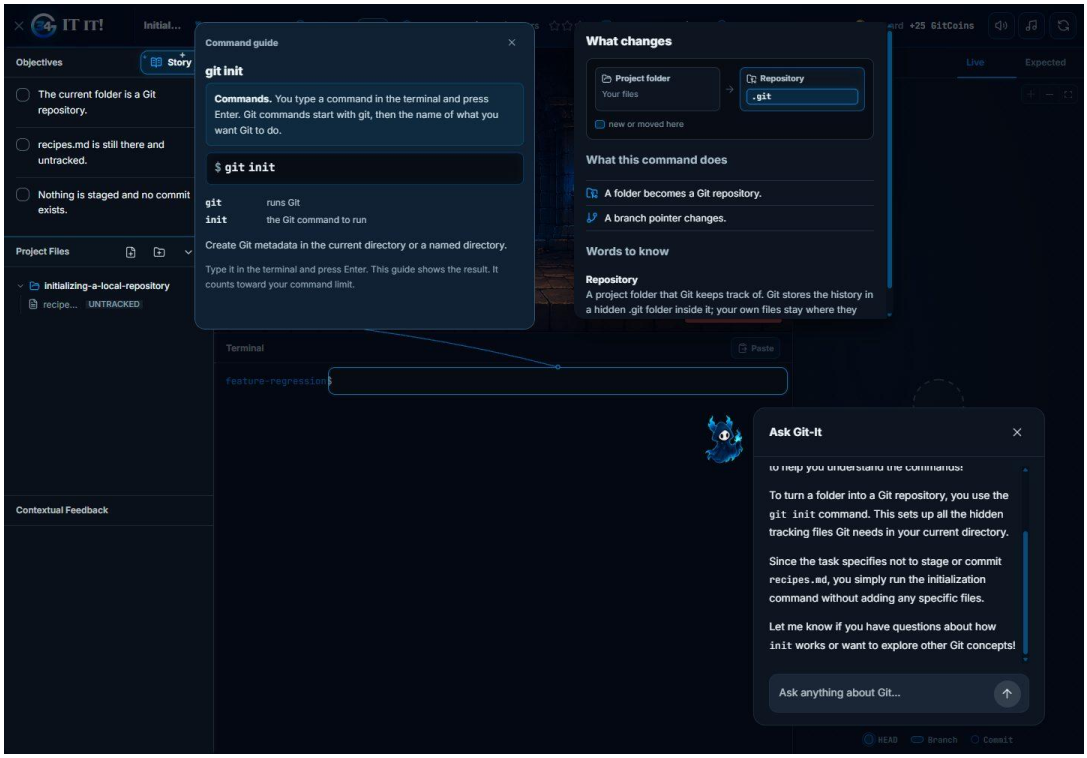


Figure 29. Inside a level, the tutor teaches the concept instead of solving the task.

3.16 Repository Diagrams

3.16.1 Purpose

Confirm the live and expected repository diagrams and their controls.

3.16.2 Inputs

- A level workspace before and after git init.
- Desktop, 900-pixel and phone widths.

3.16.3 Expected Outputs & Pass/Fail Criteria

- Live follows the repository and Expected shows the target.
- Split, Switch, zoom, fit view, details and the graph drawer work.

Actual Result: PASS. Live, Expected, Split, Switch, live updates, zoom, fit view and details all worked. The diagrams showed inline at phone width, and the graph drawer opened and closed at 900 pixels.

Evidence: Figures 30 to 33.

3.16.4 Test Procedure

1. Open the Expected tab, then Split view.
2. Run git init and check the live diagram.
3. Zoom, fit the view and open the details.
4. At 900 pixels, open and close the graph drawer.

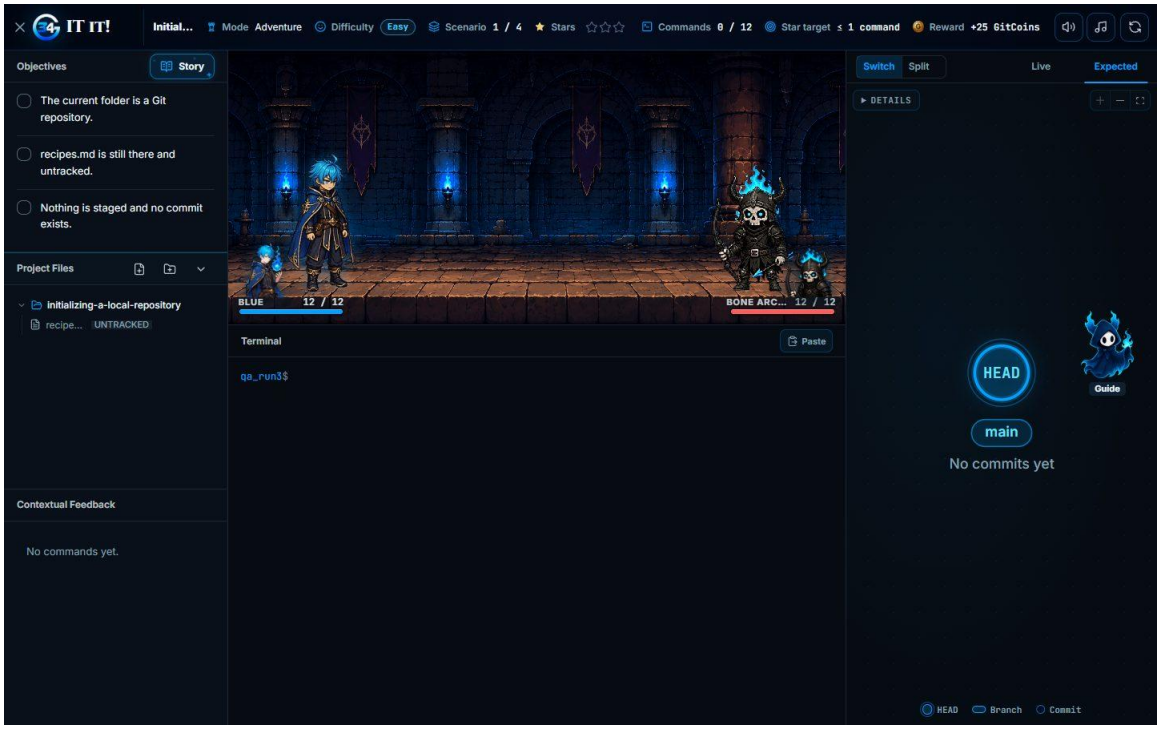


Figure 30. The Expected tab shows the target repository.

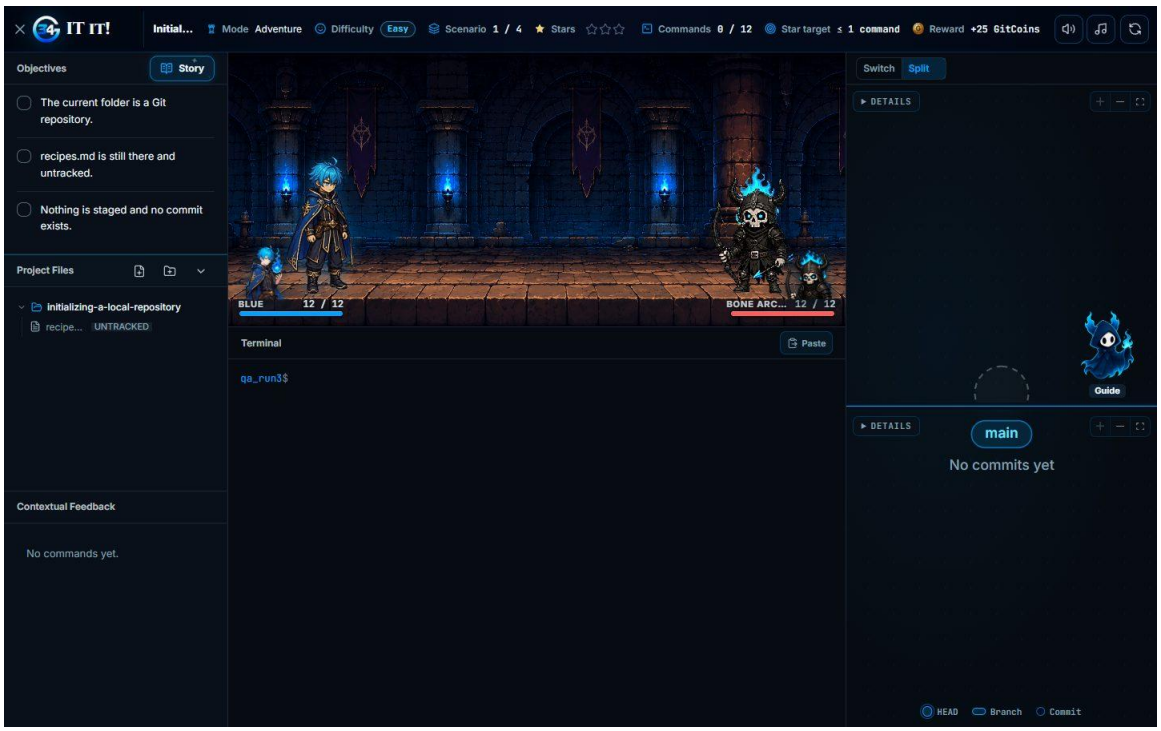


Figure 31. Split view shows the live and expected diagrams side by side.

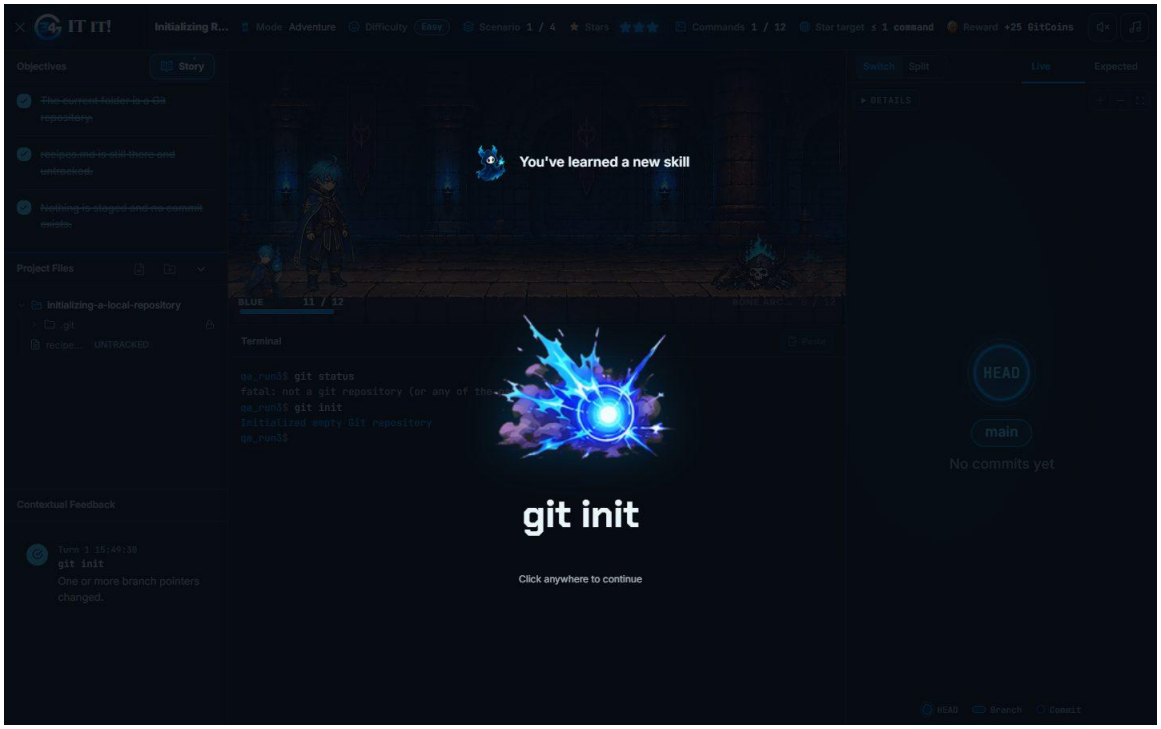


Figure 32. Running git init shows the new-skill message while the live diagram updates.

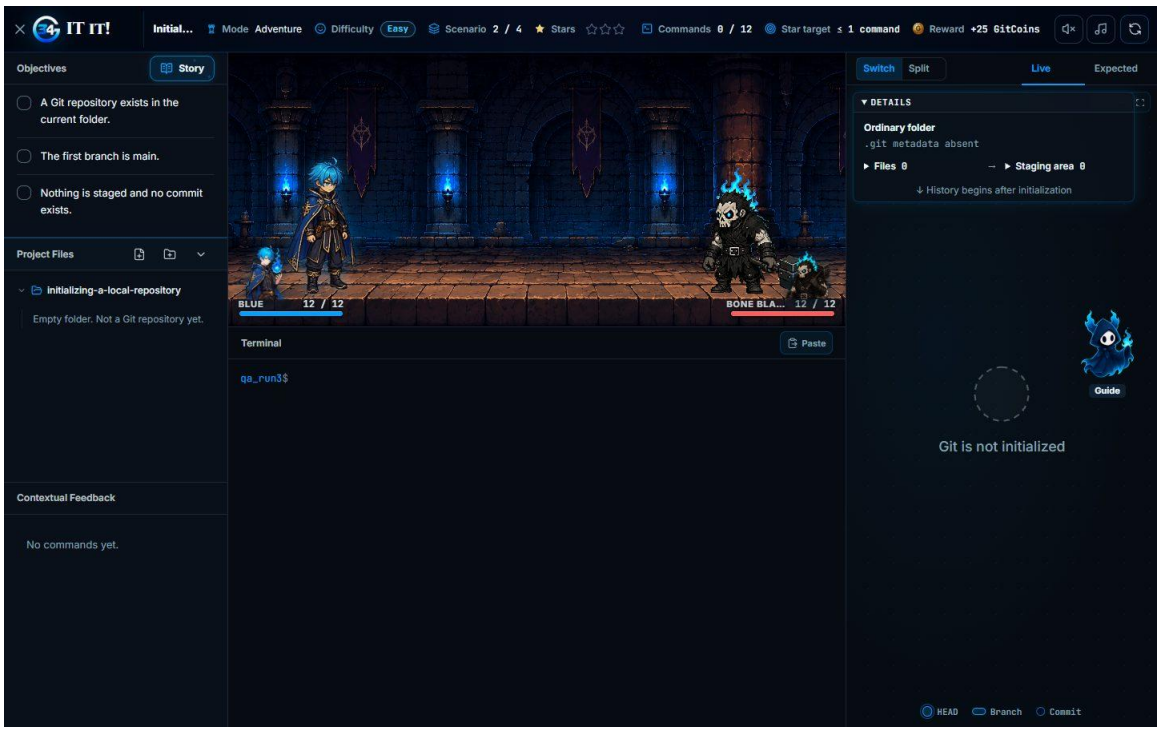


Figure 33. The diagram details panel.

3.17 Project Files Panel

3.17.1 Purpose

Confirm file operations in the level workspace and that saved edits persist.

3.17.2 Inputs

- The Project Files tree of a level workspace.

3.17.3 Expected Outputs & Pass/Fail Criteria

- Files open and saved edits persist.
- Files and folders can be created and renamed, and deletion asks for confirmation.

Actual Result: PASS. notes.md opened, its edit was saved and kept after reopening, a file and a folder were created, the file was renamed, and deletion asked for confirmation. Scenarios that need editor changes, including conflict resolution, also completed during the curriculum run.

Evidence: Figures 34 to 35.

3.17.4 Test Procedure

1. Open notes.md, edit it and save.
2. Close and reopen the file.
3. Create a file and a folder, rename the file, then delete it.

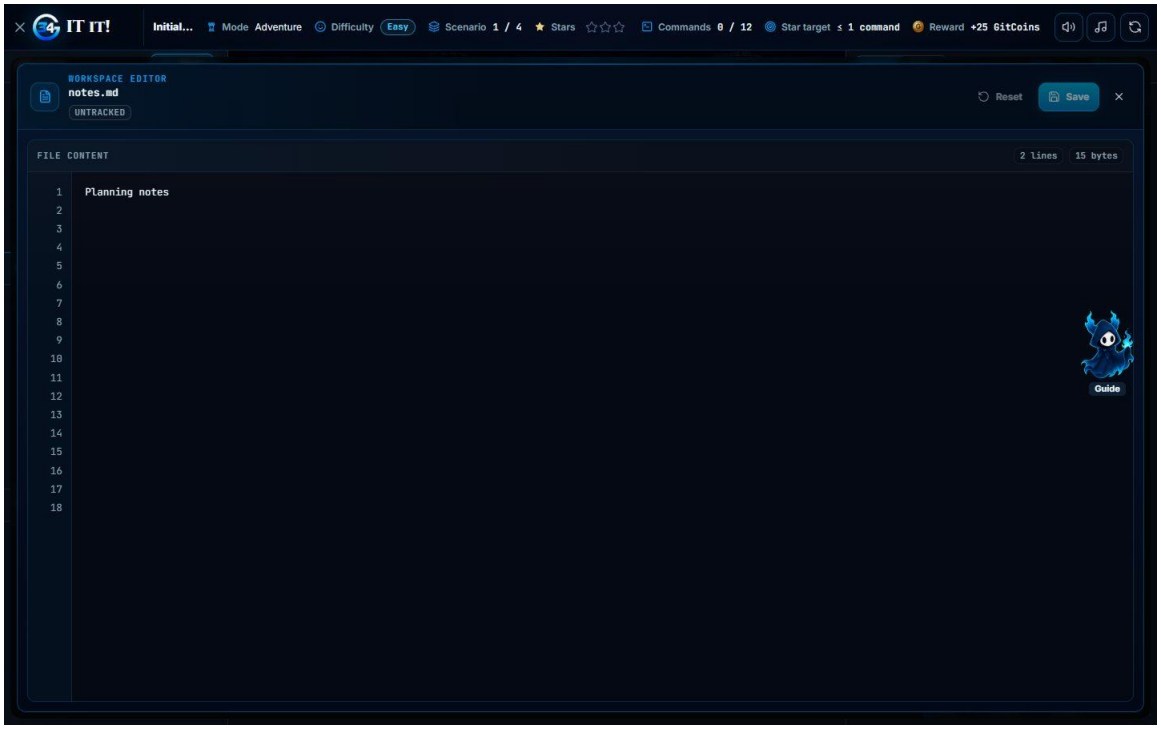


Figure 34. A project file open in the workspace editor.

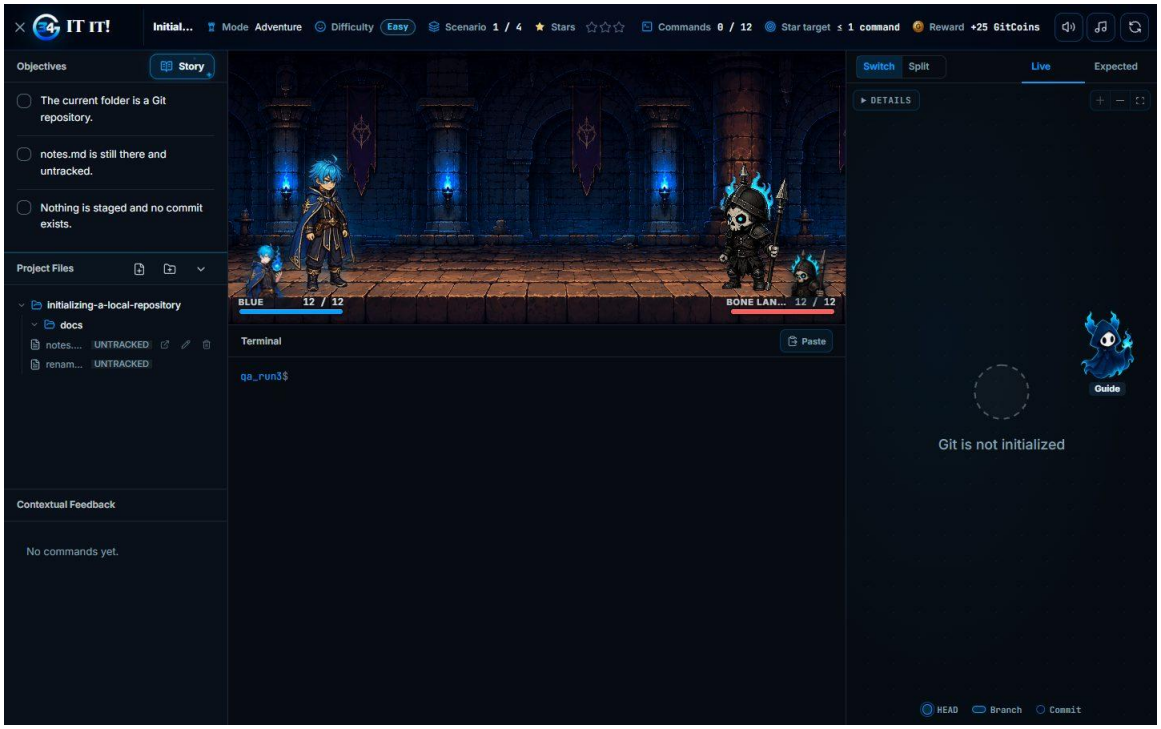


Figure 35. The file tree after creating, renaming and deleting.

3.18 Workspace Aids

3.18.1 Purpose

Confirm the scenario story, command guide, contextual feedback, star rules and audio controls.

3.18.2 Inputs

- The Story modal, command guide, star rules, contextual feedback and the audio toggles.

3.18.3 Expected Outputs & Pass/Fail Criteria

- The story can be reopened and the star rules explain the budget.
- Diagnostic commands are free and feedback records each command's effect.
- The audio labels change when toggled.

Actual Result: PASS. The story reopened, the star rules explained the budget, a diagnostic command was not counted, feedback recorded the command's effect and the audio labels changed. The beginner command guide and its Ask Git-It panel also opened (case 3.15).

Evidence: Figures 36 to 37.

3.18.4 Test Procedure

1. Open the star rules and the Story.
2. Toggle sound effects and music.
3. Run a diagnostic command and read the contextual feedback.
4. Open the command guide.

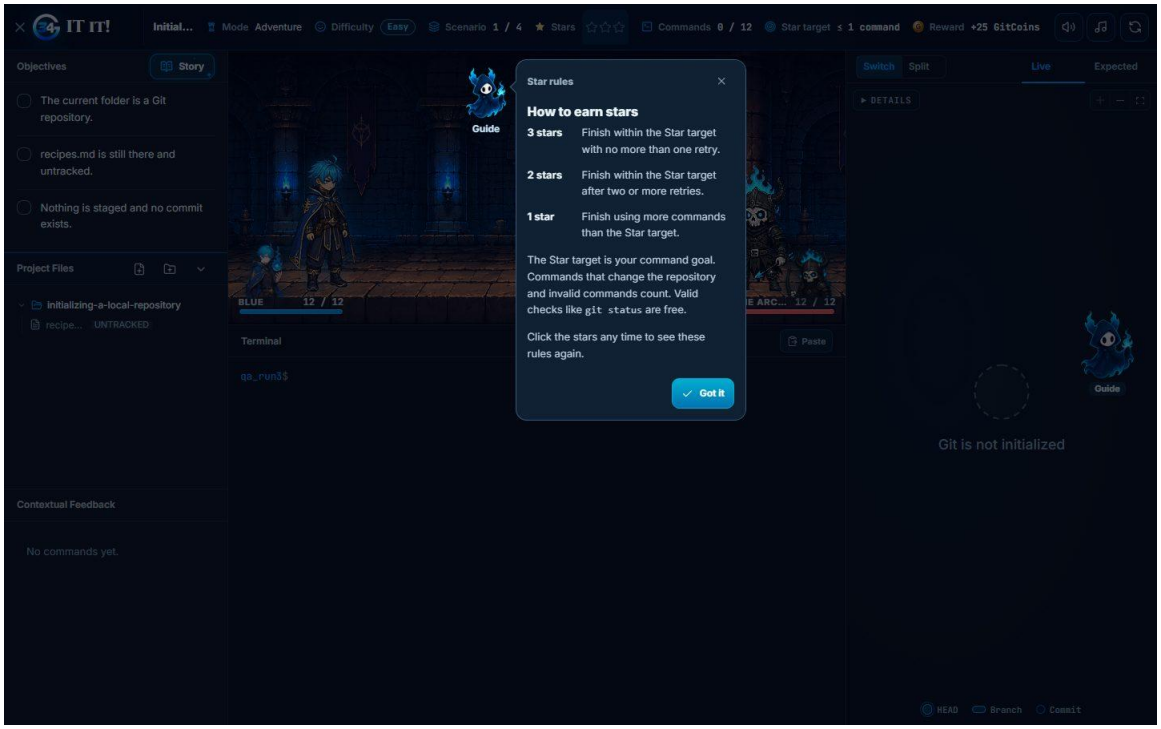


Figure 36. The star rules explain how to earn each star.

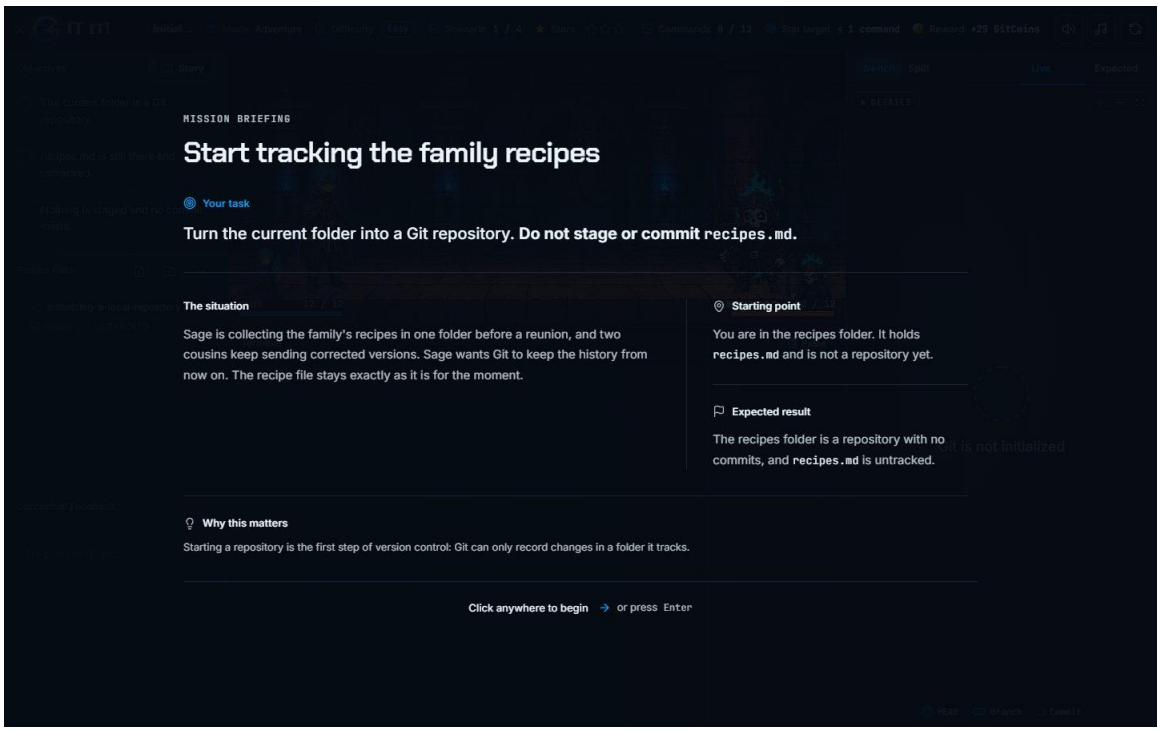


Figure 37. The mission briefing reopened from Story.

3.19 Run Control: Failure, Retry and Replay

3.19.1 Purpose

Confirm start over, failure on the command budget, retry, replay and the final Continue action.

3.19.2 Inputs

- An Adventure replay of Level 1 Easy and a Challenge replay of the Foundations Gate.
- Wrong commands to use up the budget, then the authored solution.

3.19.3 Expected Outputs & Pass/Fail Criteria

- Start over resets the attempt.
- Using up the budget ends the run with a failure screen.
- Try again starts a new run that can be completed.
- Continue after the final difficulty returns to the map.

Actual Result: PASS. Start over asked for confirmation and reset the attempt. The Adventure replay failed when its budget ran out, and Try again started a new run that completed. The Challenge replay also failed on its budget, and Try again started a different run. Continue after the last Adventure and Challenge difficulties returned to the map.

Evidence: Figures 38 to 43.

3.19.4 Test Procedure

1. In a level, select Start over and confirm.
2. Use up the command budget with wrong commands.
3. Select Try again and solve the new run.
4. Repeat the failure and retry on a Challenge replay.
5. After the final difficulty, select Continue.

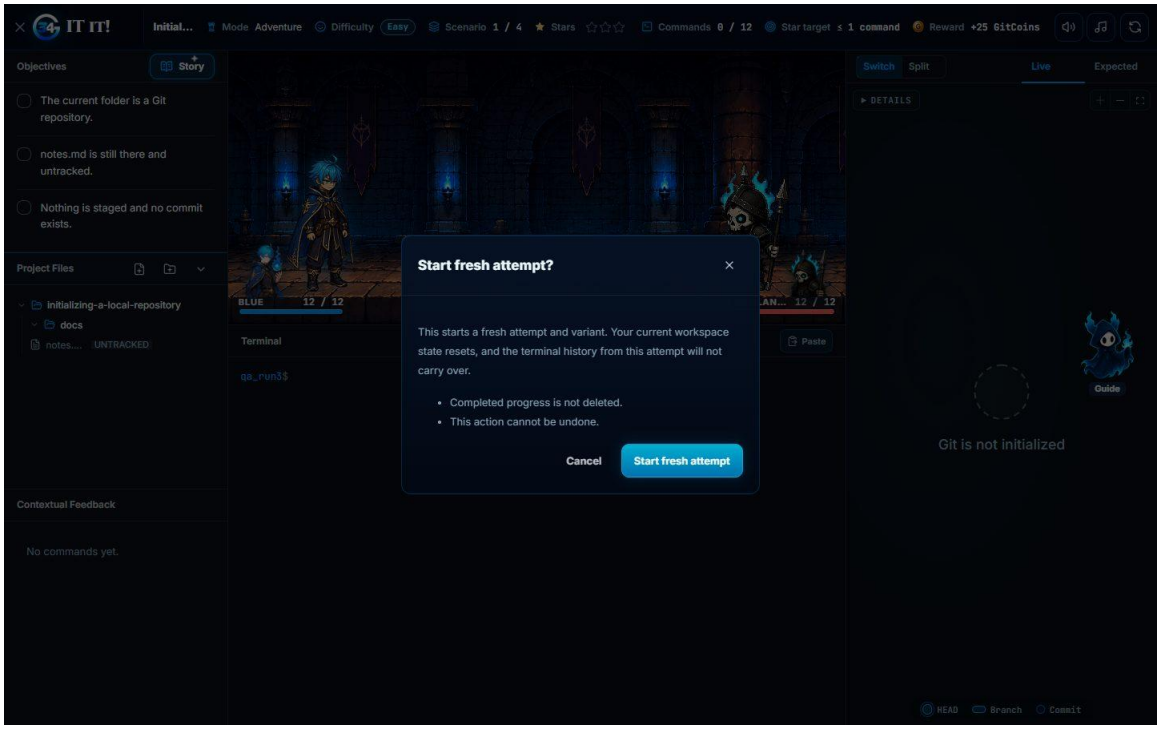


Figure 38. Start over asks for confirmation.

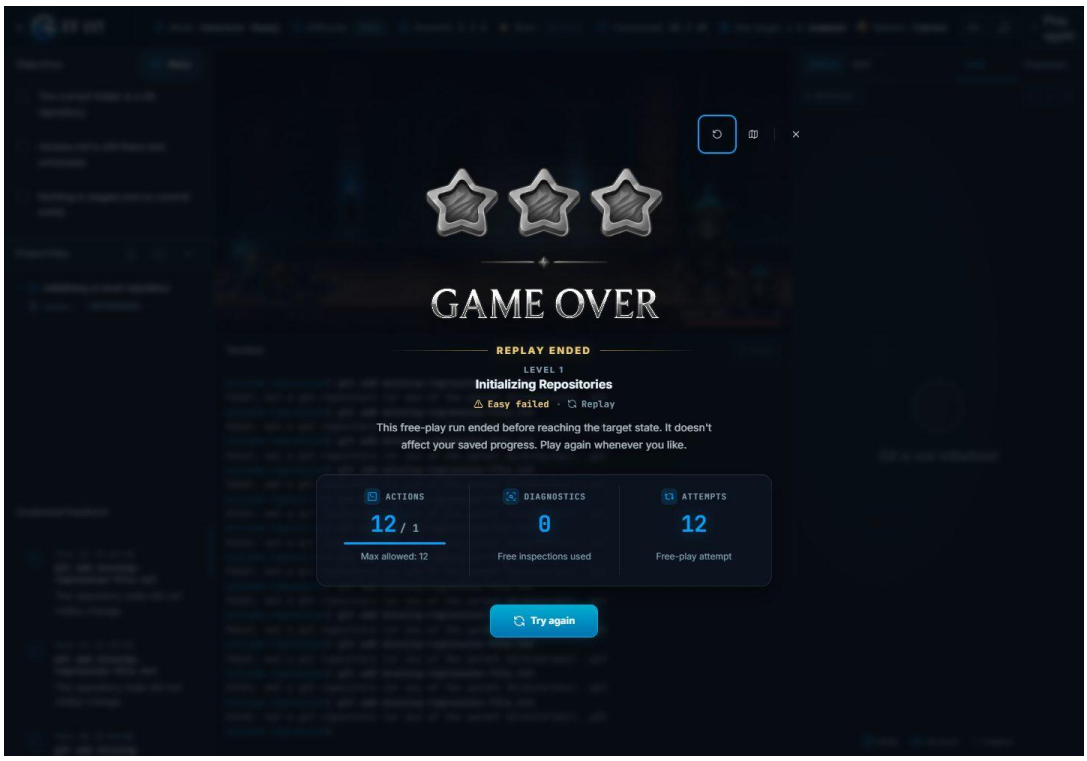


Figure 39. The run fails when the command budget is used up.

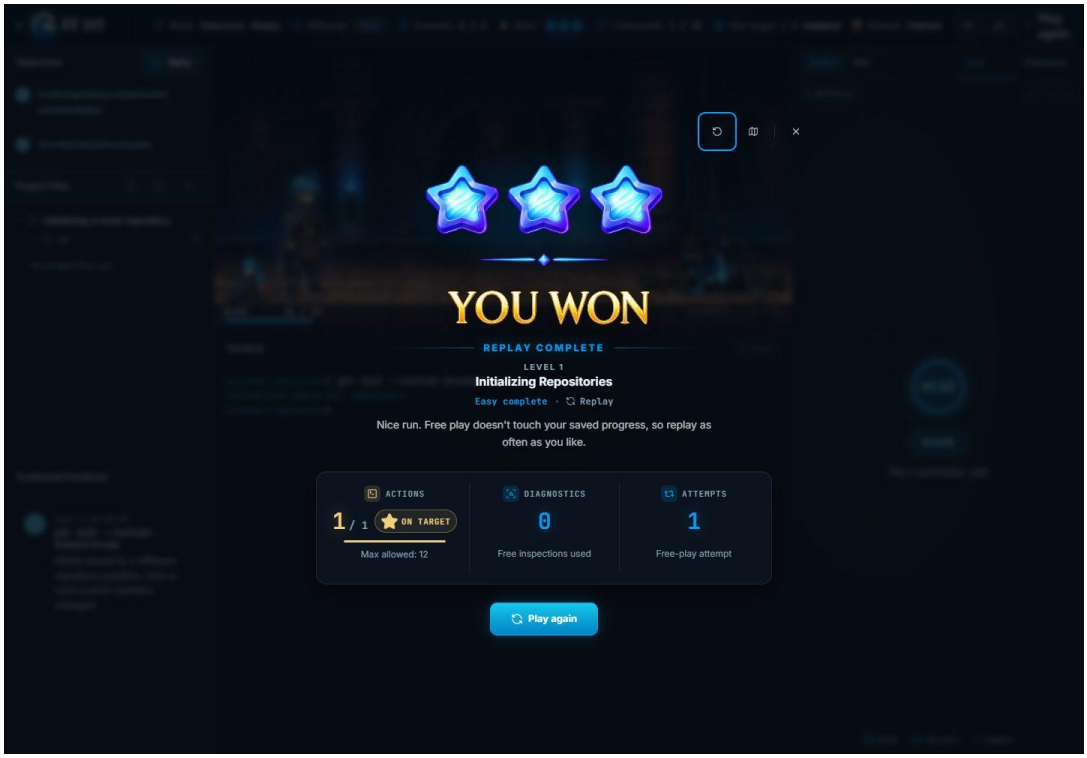


Figure 40. Try again starts a new run, completed with three stars.

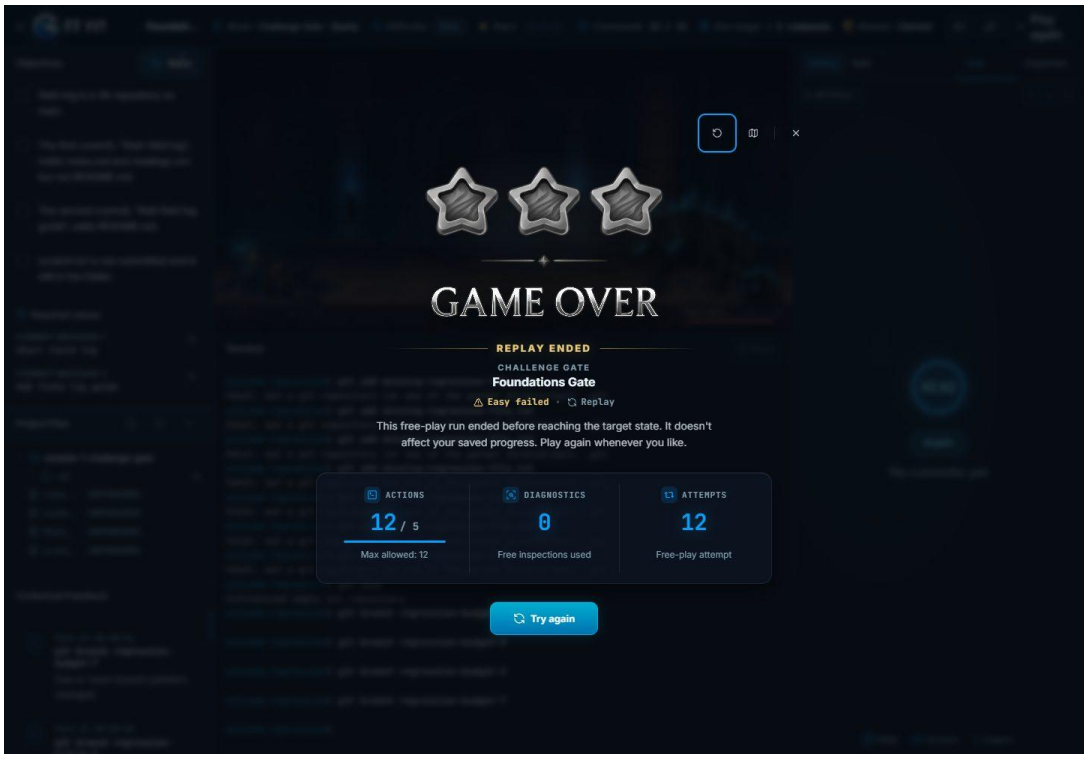


Figure 41. A Challenge replay fails on its command budget.

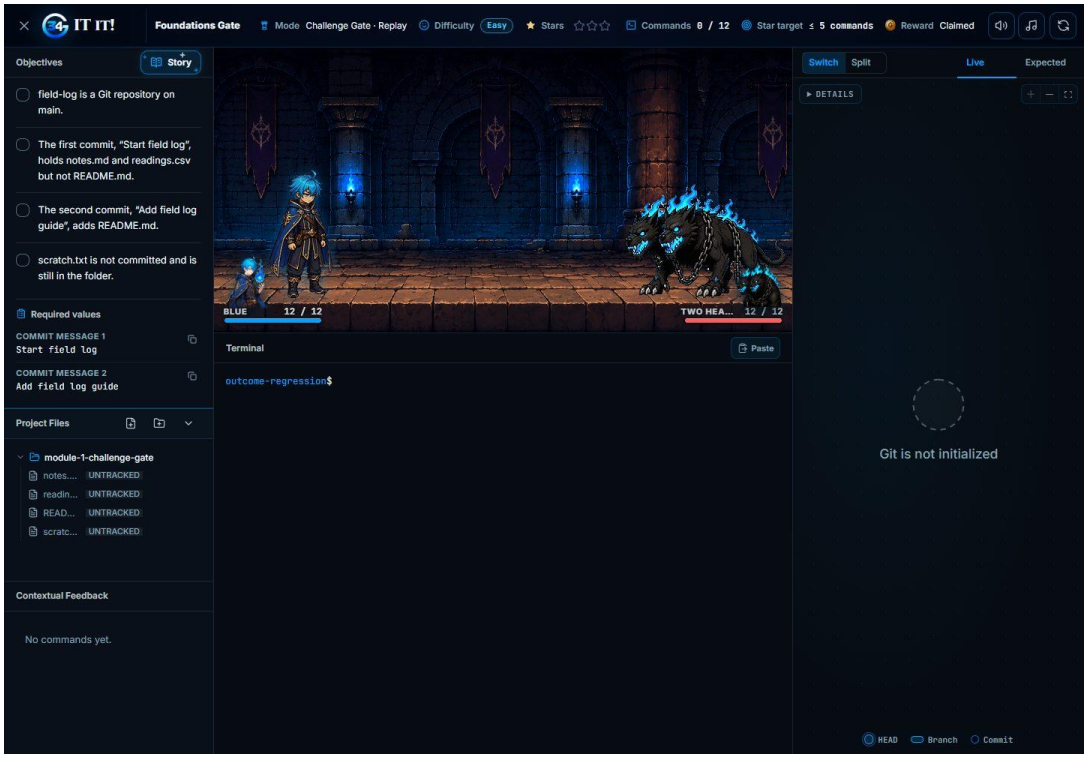


Figure 42. Try again starts a fresh Challenge run.

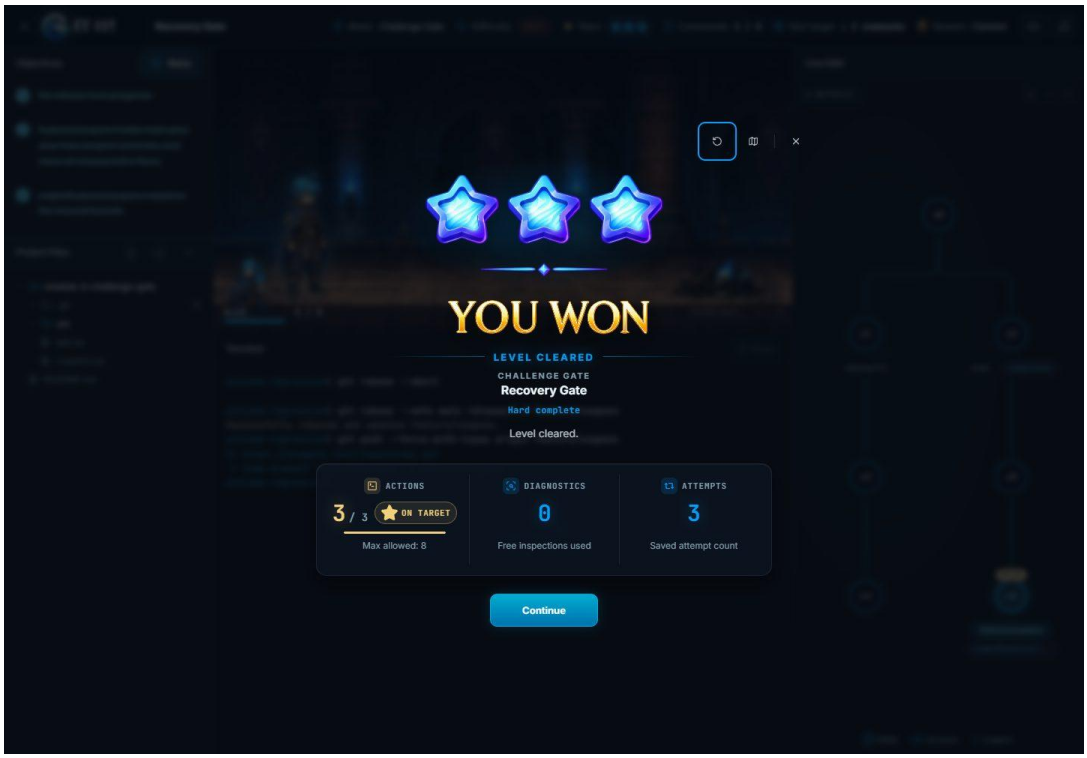


Figure 43. Continue after the final Challenge trial returns to the map.

3.20 Easy Tier of Every Adventure Level

3.20.1 Purpose

Confirm that every Easy scenario of all 21 Adventure levels can be completed in the browser.

3.20.2 Inputs

- All 21 published Adventure levels and their Easy tiers.
- The authored solution of each scenario.
- A learner with no Adventure completions.

3.20.3 Expected Outputs & Pass/Fail Criteria

- Every scenario completes with three stars.
- Each tier clear is saved by the backend.

Actual Result: PASS. All 21 Easy tiers passed with three stars in every scenario. Table A-3 shows the completion screen of every tier.

Evidence: Table A-3 (completion screen of every tier) and the scenario log..

3.20.4 Test Procedure

1. Start each level's Easy tier in order.
2. Enter the authored commands and file edits in the workspace.
3. Continue through every scenario.
4. Check the saved run and completion records.

3.21 Medium and Hard Tiers of Every Adventure Level

3.21.1 Purpose

Confirm that every Medium and Hard scenario of all 21 Adventure levels can be completed in the browser.

3.21.2 Inputs

- The 42 Medium and Hard tiers of the 21 Adventure levels.
- The authored solution of each scenario.

3.21.3 Expected Outputs & Pass/Fail Criteria

- Every scenario completes with three stars.
- Each tier clear unlocks the next difficulty and is saved.

Actual Result: PASS. All 42 tiers passed. With the Easy tiers, all 63 tiers and all 149 ordered scenarios completed with three stars (Table A-3).

Evidence: Table A-3 and the scenario log..

3.21.4 Test Procedure

1. After each Easy tier, play the Medium tier, then the Hard tier.
2. Enter the authored commands and file edits.
3. Check the saved run and completion records.

3.22 Interactive Staging, Conflicts and Merge Tool

3.22.1 Purpose

Confirm the interactions that need more than one terminal line: hunk prompts, conflict editing and the merge tool.

3.22.2 Inputs

- Scenarios that use git add -p, manual conflict resolution and git mergetool (Levels 4, 16, 17 and 21).

3.22.3 Expected Outputs & Pass/Fail Criteria

- Hunk prompts accept answers.
- Saved conflict resolutions change the repository.
- The scenario objectives complete.

Actual Result: PASS. Every scenario of this kind completed during the curriculum run. git add -p accepted the hunk answers, and conflicts resolved in the editor or the merge tool were saved and committed.

Evidence: Figures 44 to 45.

3.22.4 Test Procedure

1. Play these scenarios as part of the full curriculum.
2. Answer the hunk prompts.
3. Edit the conflicted files in the editor or the merge tool and save them.
4. Check that the objectives complete.

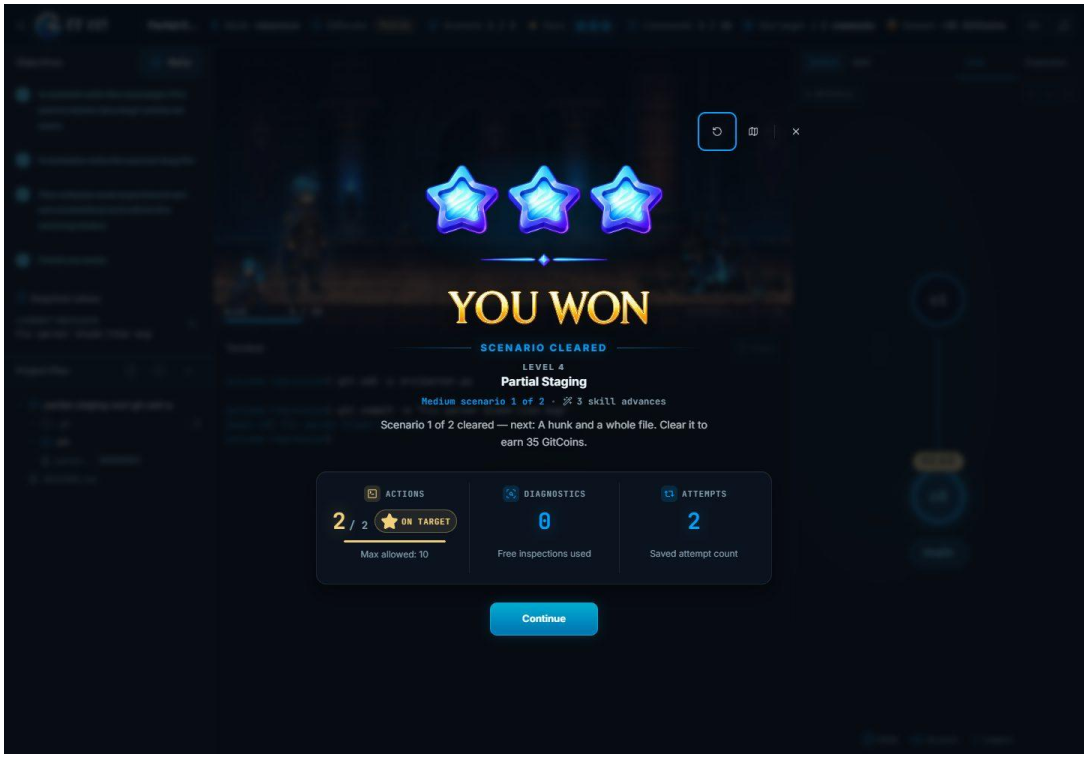


Figure 44. A Partial Staging (git add -p) scenario cleared.

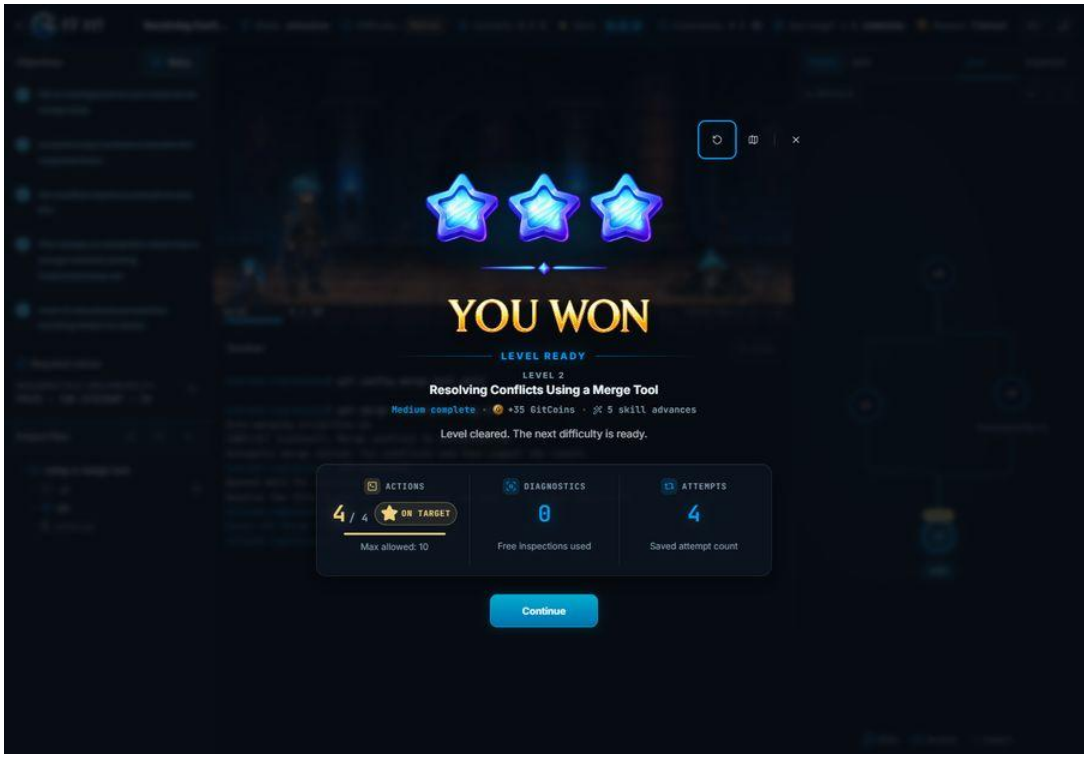


Figure 45. Resolving Conflicts Using a Merge Tool, Medium tier cleared.

3.23 Challenge Gates

3.23.1 Purpose

Confirm that each Challenge Gate opens after its chapter is complete and that all 12 trials can be completed.

3.23.2 Inputs

- The test learner after clearing every Adventure tier.
- The 12 published Challenge trials.

3.23.3 Expected Outputs & Pass/Fail Criteria

- Each gate difficulty opens in order.
- Every trial completes with three stars and is saved.

Actual Result: PASS. All 12 trials passed as normal first clears with three stars, after the Adventure tiers were cleared in the browser (Table A-4). Continue after the last trial returned to the map.

Evidence: Table A-4 (completion screen of every trial) and the gate results log..

3.23.4 Test Procedure

1. After the Adventure tiers, open each gate in order.
2. Play every trial with its authored solution.
3. Check each saved completion.

3.24 Squire's Drill

3.24.1 Purpose

Confirm the optional recall drill: answers, resuming, saving and restarting.

3.24.2 Inputs

- The Initializing Repositories drill.

3.24.3 Expected Outputs & Pass/Fail Criteria

- Answers are graded correctly.
- Leaving and returning resumes the drill.
- Clearing it saves progress; restarting gives fresh lives; Escape exits.

Actual Result: PASS. Answers were graded, the drill resumed after leaving, clearing it saved one clear, restarting gave fresh lives and Escape exited.

Evidence: Figures 46 to 49.

3.24.4 Test Procedure

1. Open the drill and answer a question.
2. Leave the drill and open it again.
3. Finish the remaining questions.
4. Restart the drill, then press Escape.

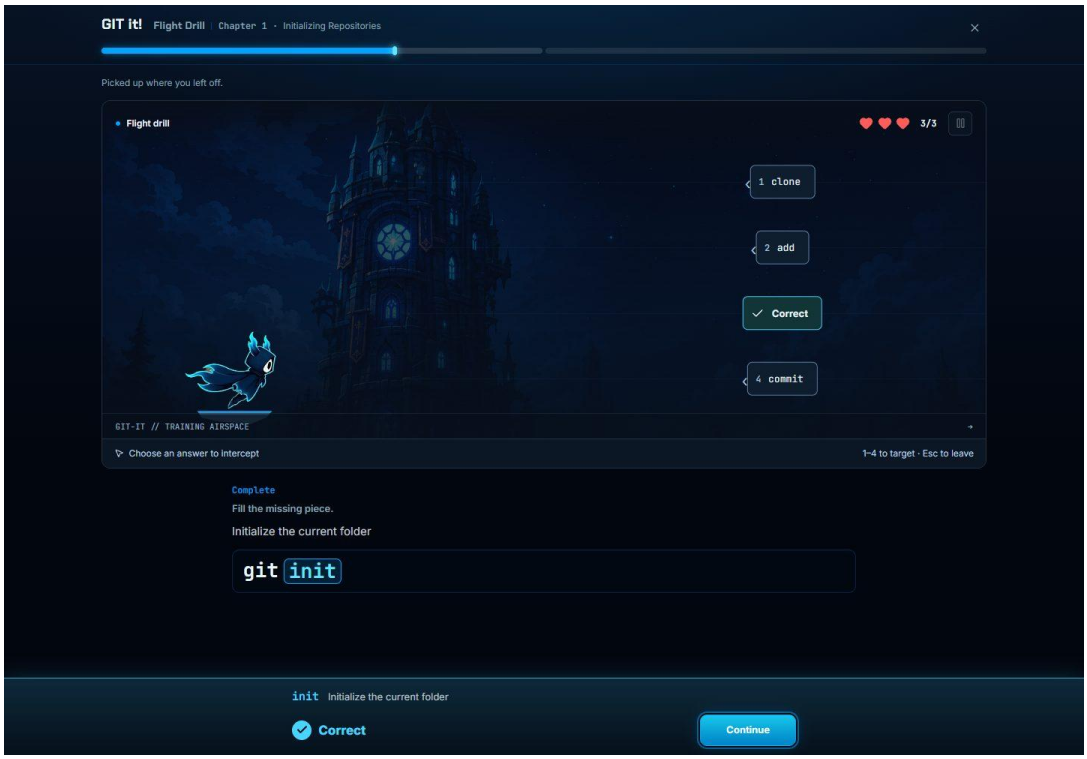


Figure 46. A correct answer in the drill.

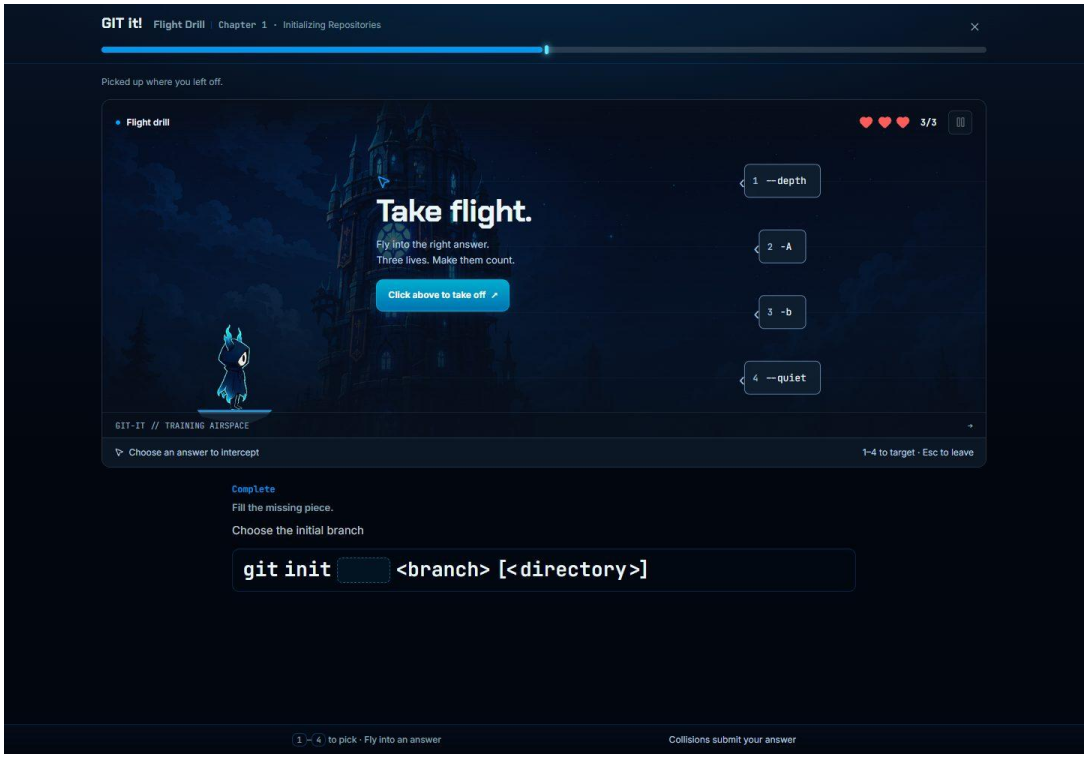


Figure 47. The drill resumes after leaving.

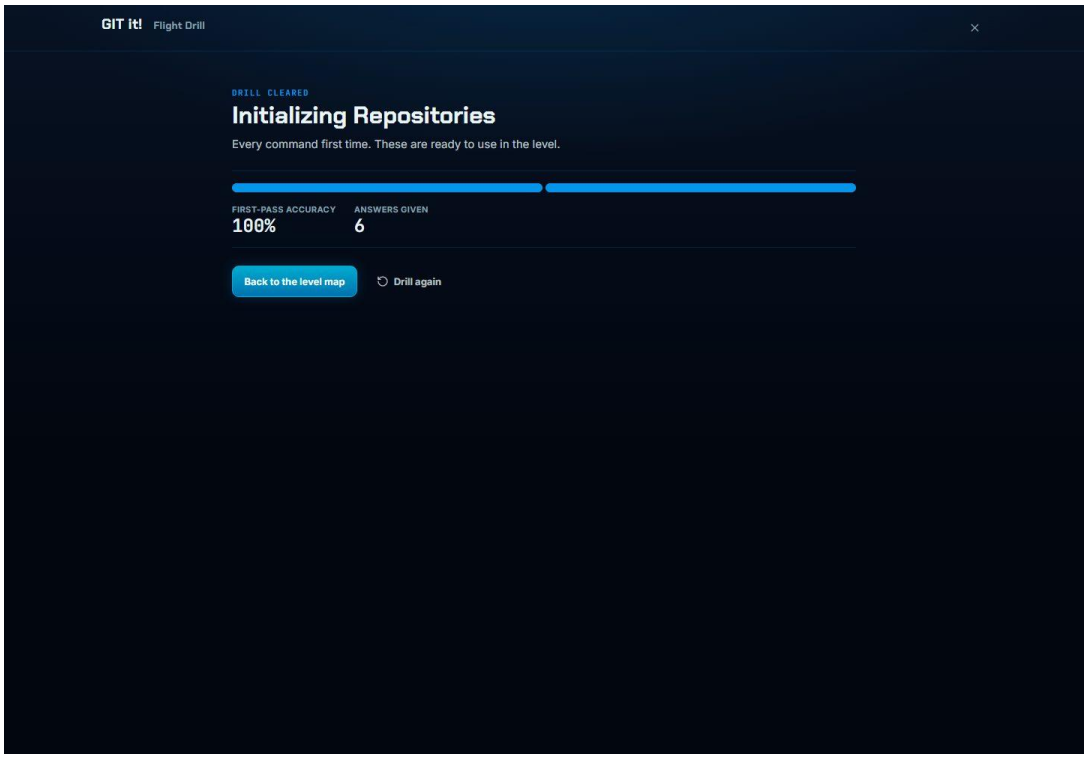


Figure 48. The drill cleared and saved.

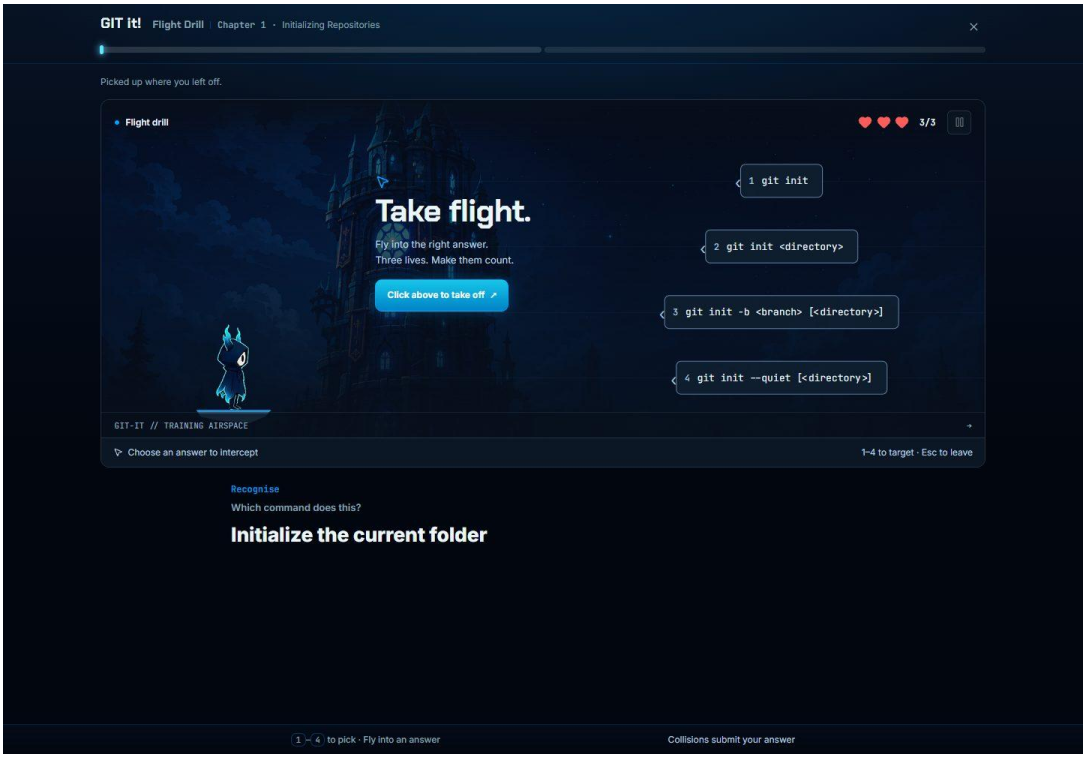


Figure 49. The drill restarted with fresh lives.

3.25 Chapter Features

3.25.1 Purpose

Confirm chapter mastery, progress chests, the Chapter Book, the chapter selector and the Git primer.

3.25.2 Inputs

- The chapter map after earning progress.
- The collected chests, Chapter Book and Orientation primer.

3.25.3 Expected Outputs & Pass/Fail Criteria

- Mastery and chests match the progress.
- Collected chests cannot be claimed again.
- The Chapter Book, chapter selector and primer open.

Actual Result: PASS. Chapter 1 showed 100% mastery, 9 of 9 levels and trials, and four collected chests. The collected chest showed as already collected, and the Chapter Book, chapter selector and primer opened.

Evidence: Figures 50 to 54.

3.25.4 Test Procedure

1. Open a mastered chapter and select a collected chest.
2. Open the Chapter Book.
3. Open the chapter selector.

4. Open the Git primer and select a topic.

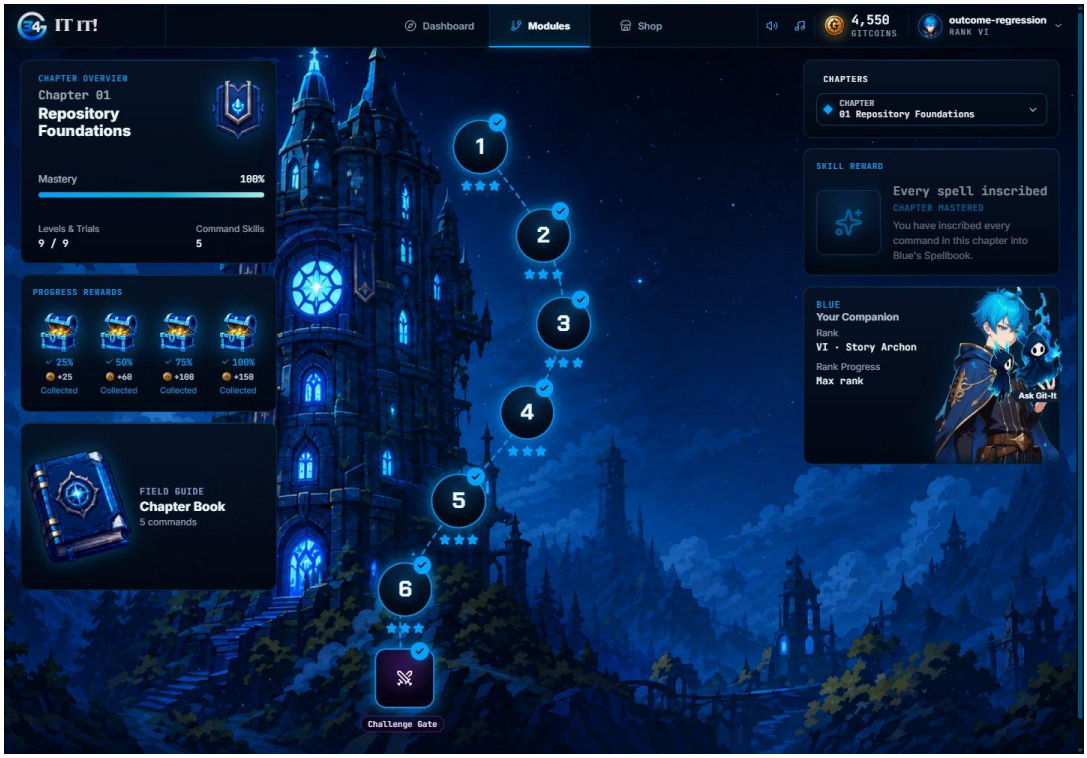


Figure 50. Chapter mastery and collected chests.



Figure 51. A collected chest is shown as already collected.

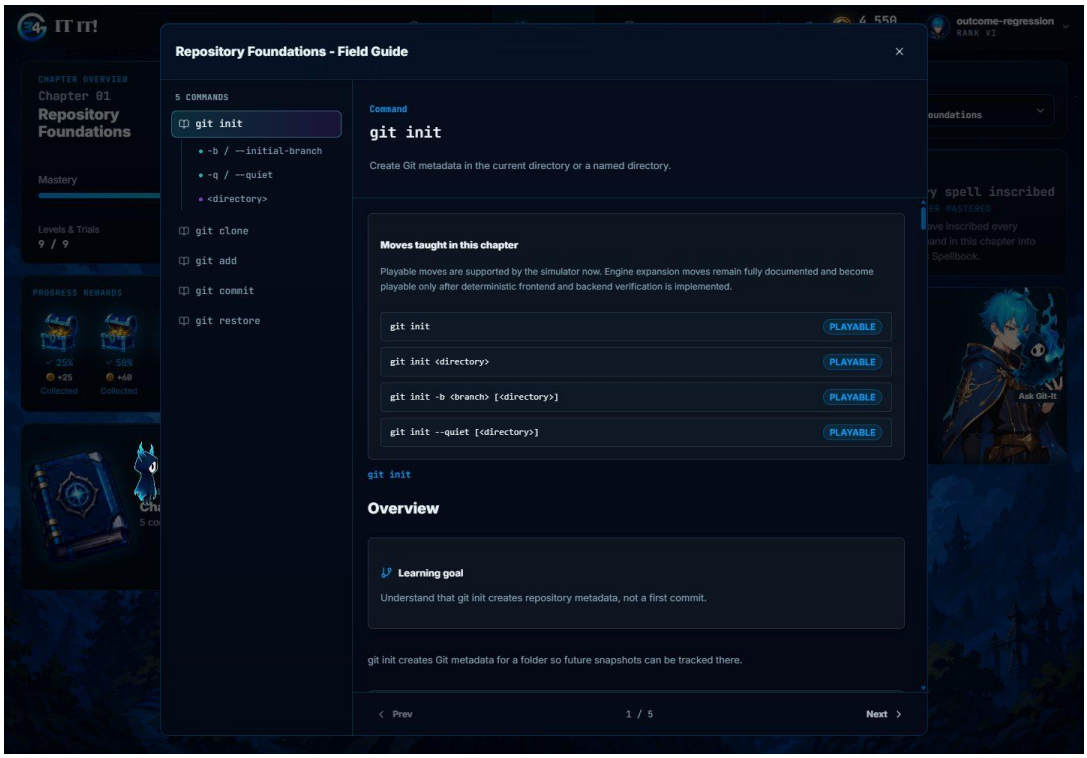


Figure 52. The Chapter Book.

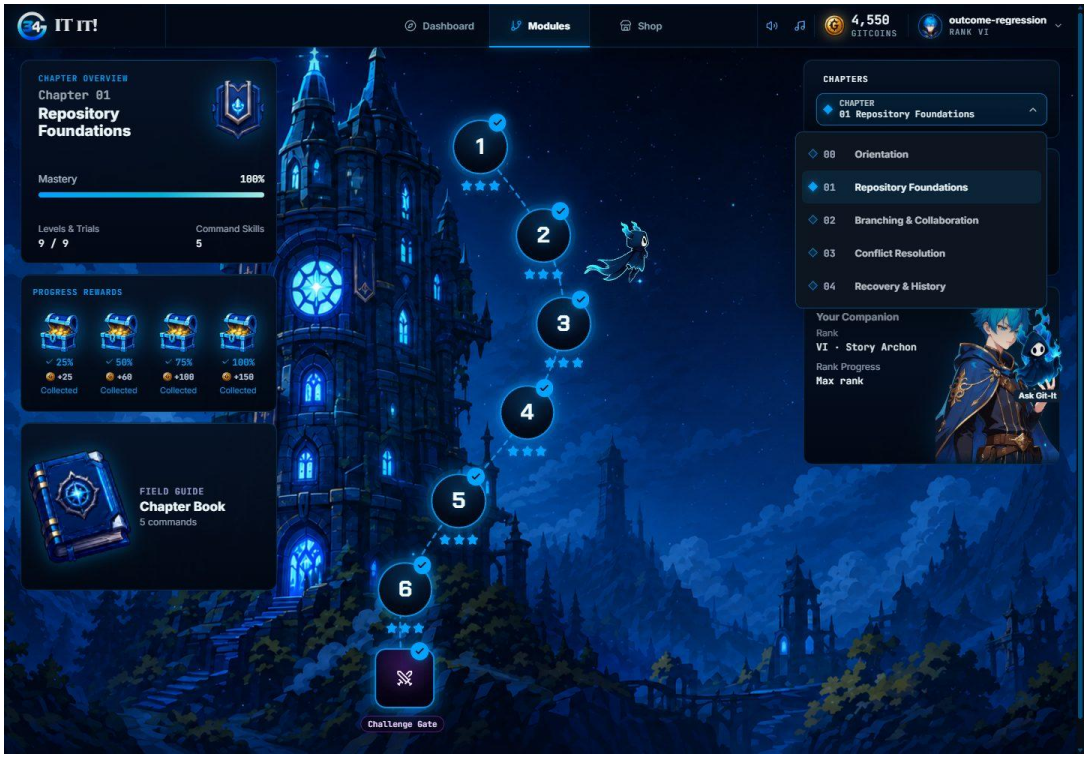


Figure 53. The chapter selector.

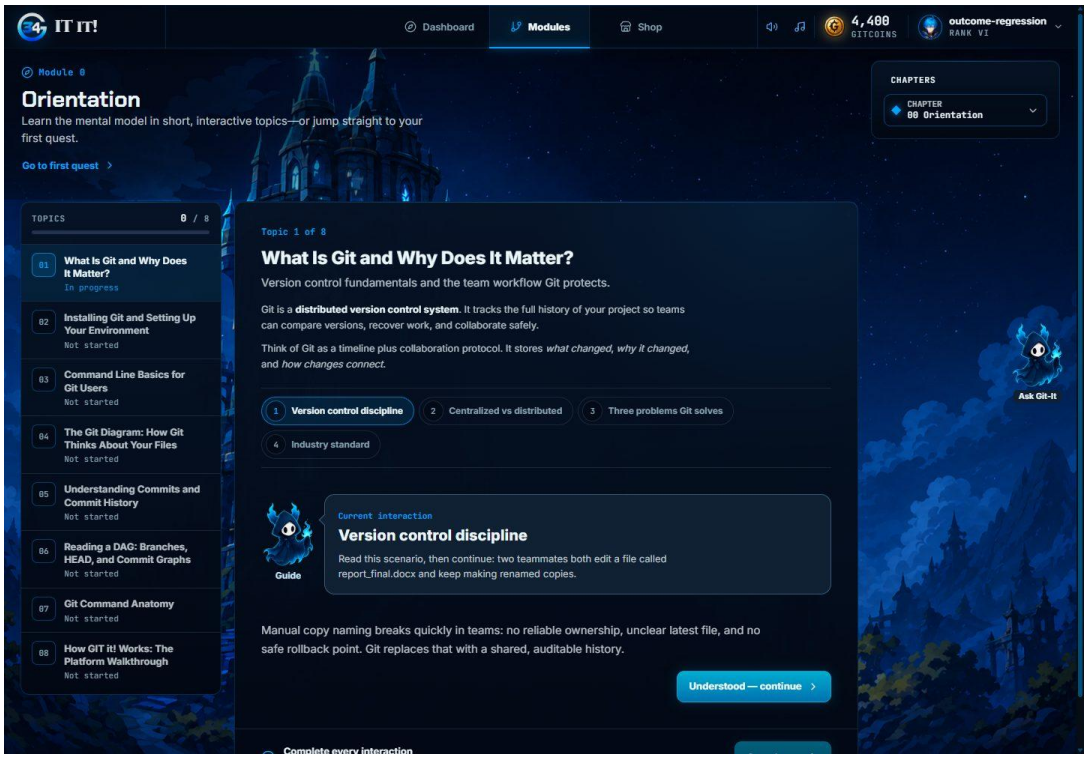


Figure 54. The Git primer in the Orientation chapter.

3.26 Dashboard Views

3.26.1 Purpose

Confirm the profile, rank ladder, spellbook, progress and skills views.

3.26.2 Inputs

- The test learner at Rank VI with 63 tier clears and 21 learned commands.

3.26.3 Expected Outputs & Pass/Fail Criteria

- Each view opens and shows the earned data.
- The spellbook's attack control works.

Actual Result: PASS. Profile showed Rank VI, 63 perfect clears, 63 cleared tiers and 21 learned spells, and the git init attack control worked. The Progress and Skills views showed the earned data.

Evidence: Figures 55 to 58.

3.26.4 Test Procedure

1. Open Profile and the rank ladder.
2. Select a learned spell.
3. Open Progress, then Skills and achievements.

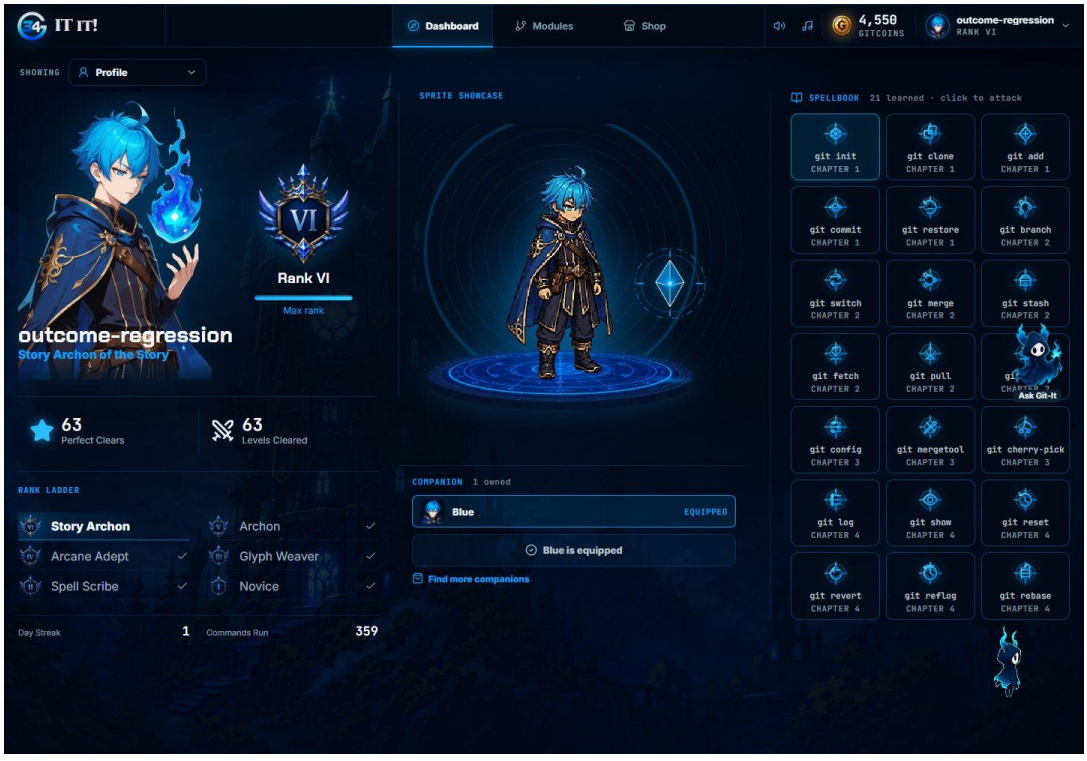


Figure 55. Profile with rank, companion and spellbook.

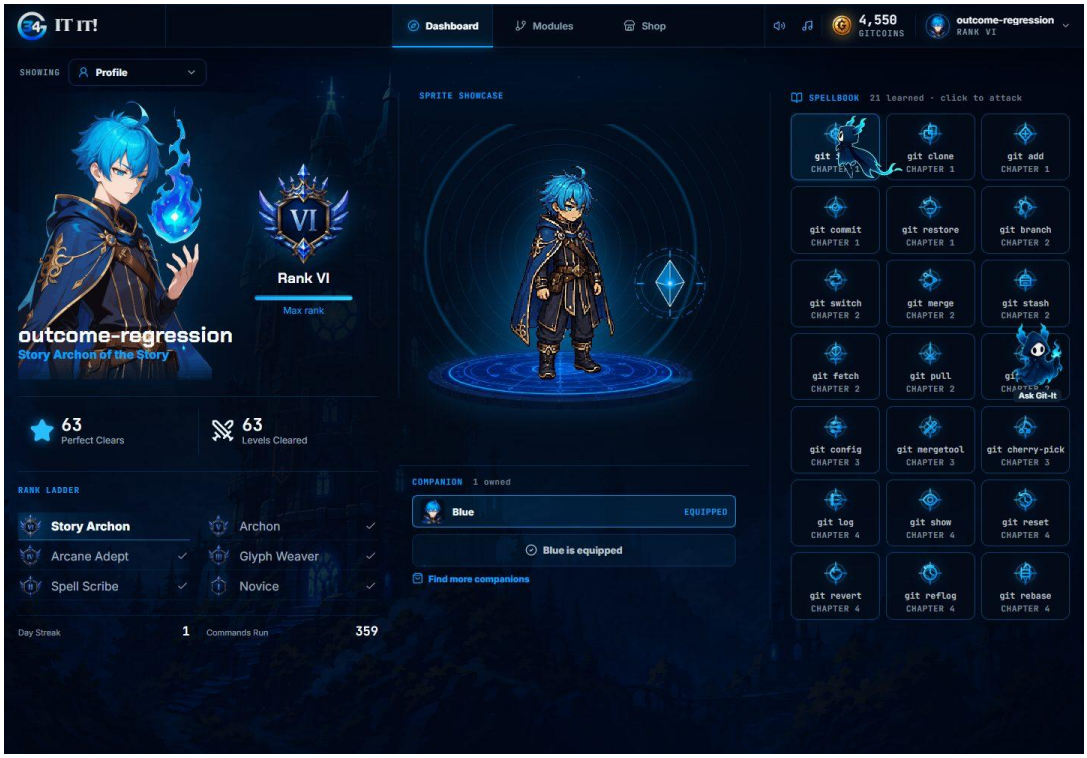


Figure 56. A learned spell selected in the spellbook.

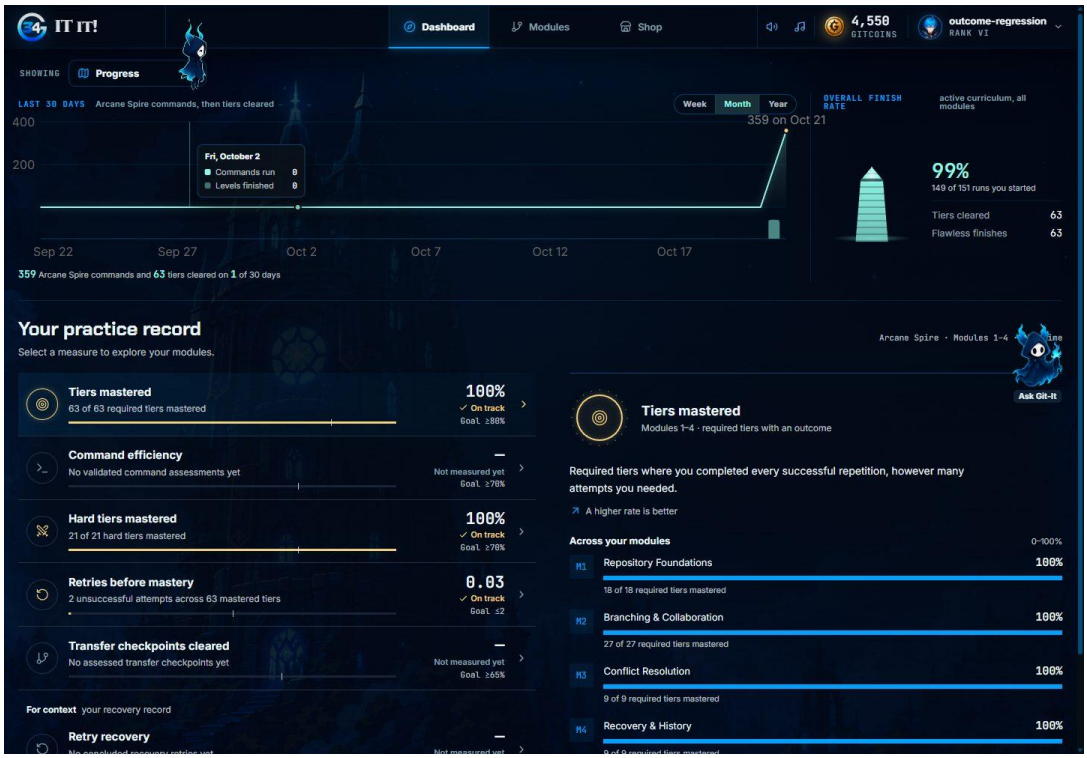


Figure 57. The Progress view.



Figure 58. Skills and achievements.

3.27 Shop and Loadout

3.27.1 Purpose

Confirm purchases, insufficient funds, equipping and that the equipped companion persists.

3.27.2 Inputs

- A learner with zero GitCoins.
- The test learner with enough GitCoins.

3.27.3 Expected Outputs & Pass/Fail Criteria

- An unaffordable companion shows the shortfall and cannot be bought.
- Buying takes the price and grants ownership.
- The equipped companion stays equipped after a reload.

Actual Result: PASS. With zero GitCoins, White showed "Need 150 more" and could not be bought. Buying White took 150 GitCoins. White stayed equipped after a reload, and Blue could be equipped again.

Evidence: Figures 59 to 63.

3.27.4 Test Procedure

1. With the zero-balance learner, select White.
2. With the test learner, buy White and equip it in Profile.
3. Reload, then equip Blue again.

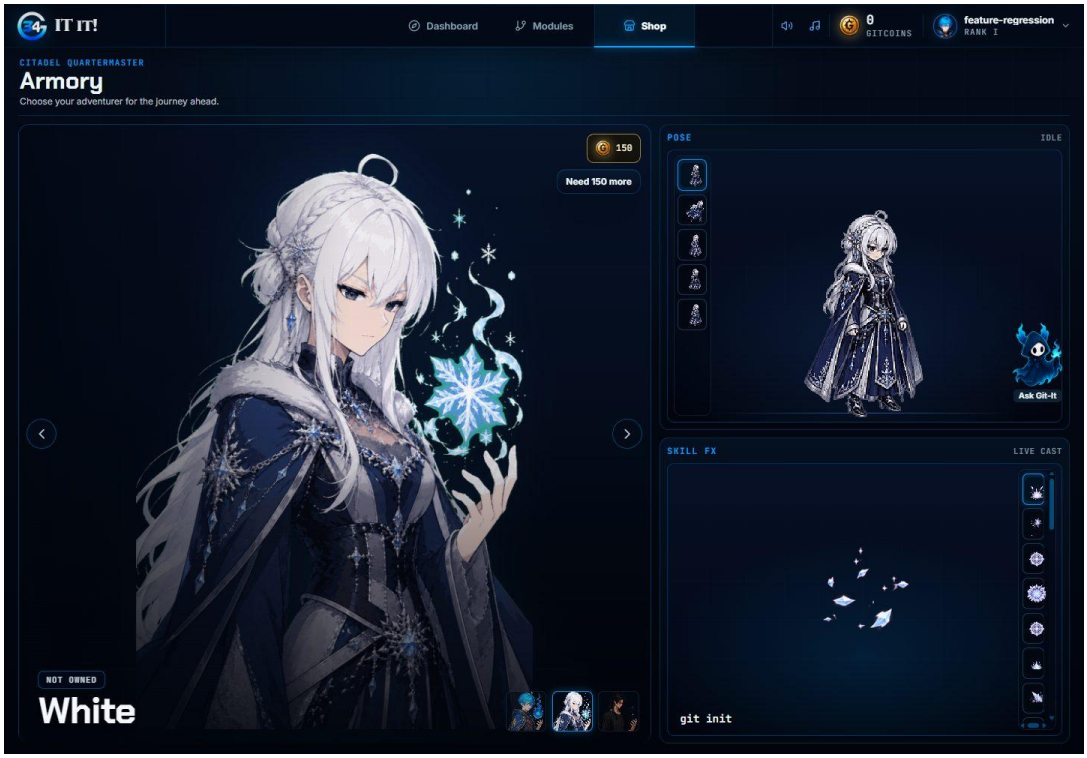


Figure 59. An unaffordable companion shows the shortfall.

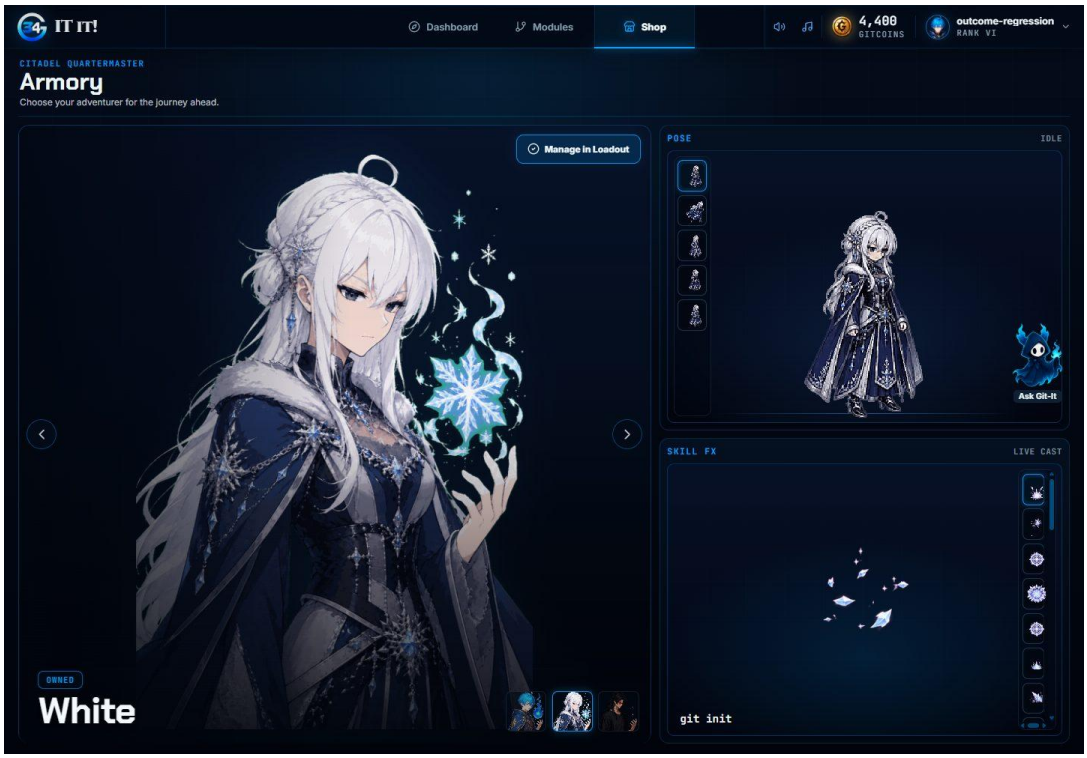


Figure 60. Buying White in the Armory.

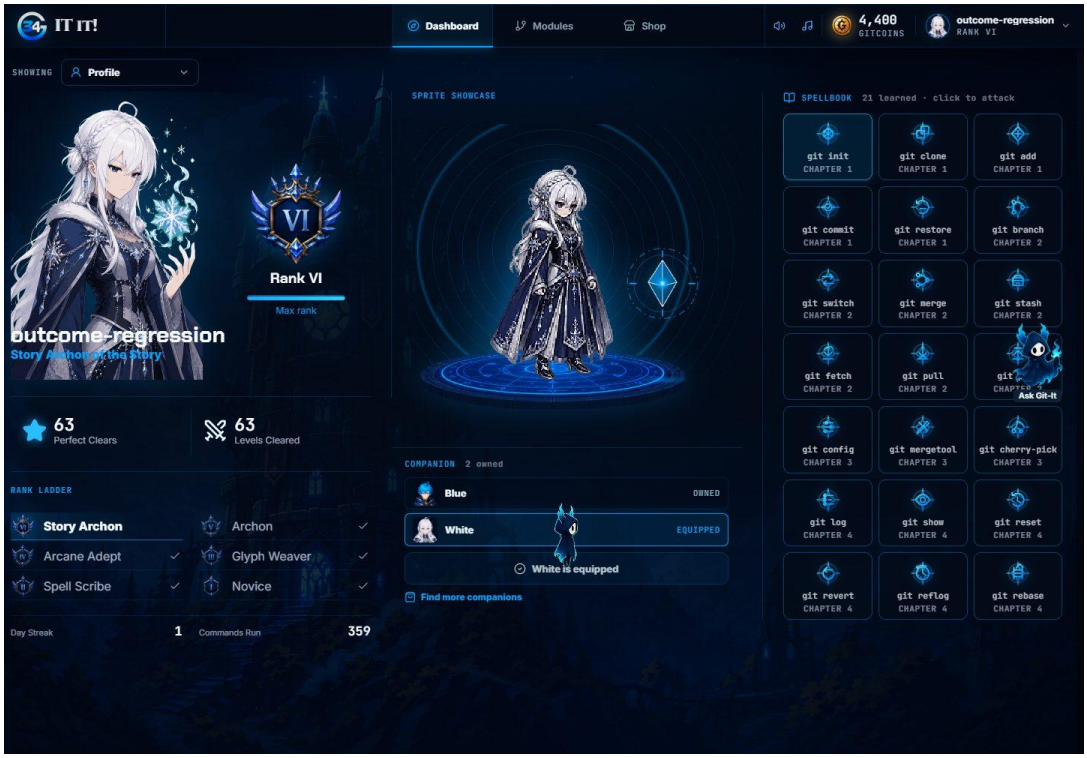


Figure 61. White equipped in Profile.

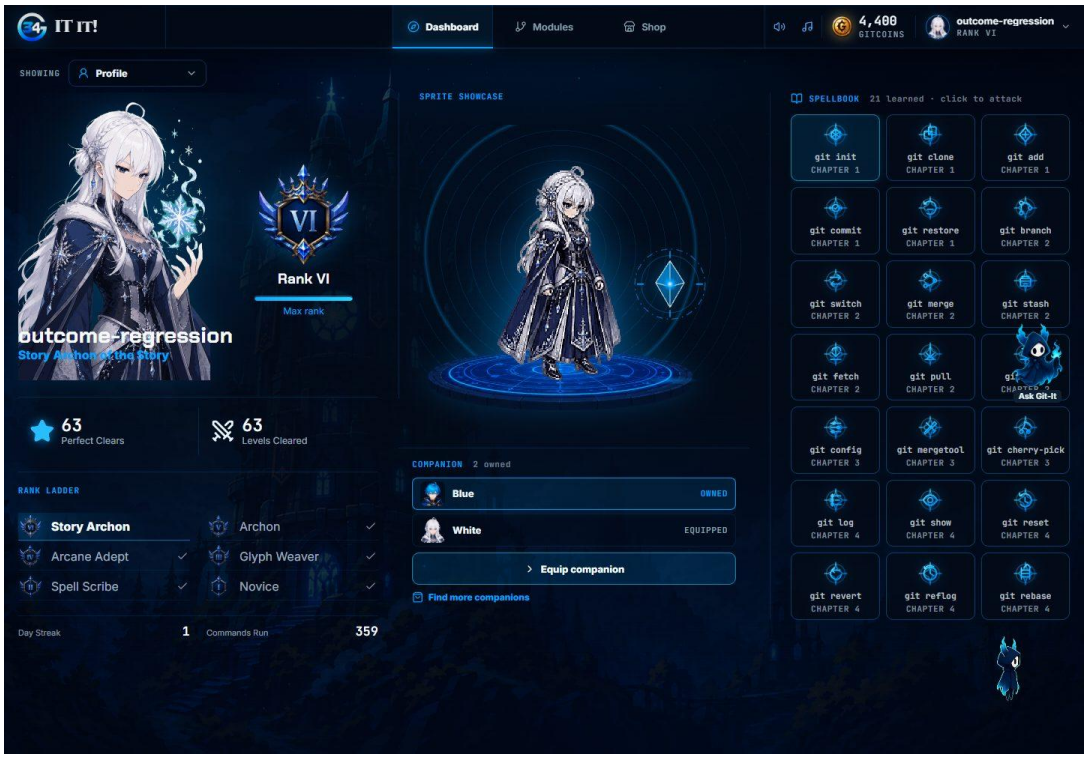


Figure 62. The equipped companion persists after a reload.

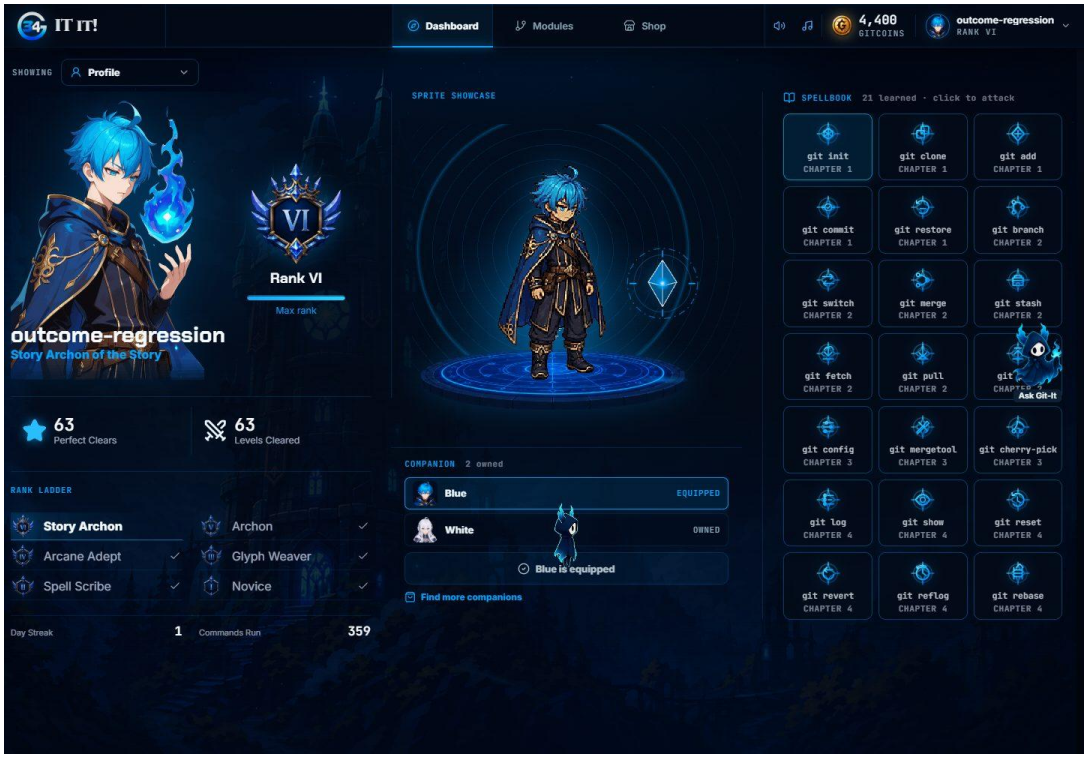


Figure 63. Blue equipped again.

3.28 Account Settings

3.28.1 Purpose

Confirm the account details, preferences, password change and session controls.

3.28.2 Inputs

- The test learner's Settings page.

3.28.3 Expected Outputs & Pass/Fail Criteria

- The username and email are shown read-only.
- Preferences persist after a reload.
- A wrong current password is rejected and a valid change succeeds.
- Signing out other sessions keeps this one; signing out everywhere returns to sign in.

Actual Result: PASS. Preferences persisted, the wrong current password showed an inline error, the password change succeeded, signing out other sessions kept the current session, and signing out everywhere returned to sign in.

Evidence: Figures 64 to 67.

3.28.4 Test Procedure

1. Change the motion and mascot preferences and reload.
2. Submit a wrong current password, then change the password.
3. Sign out other sessions.
4. Sign out everywhere.

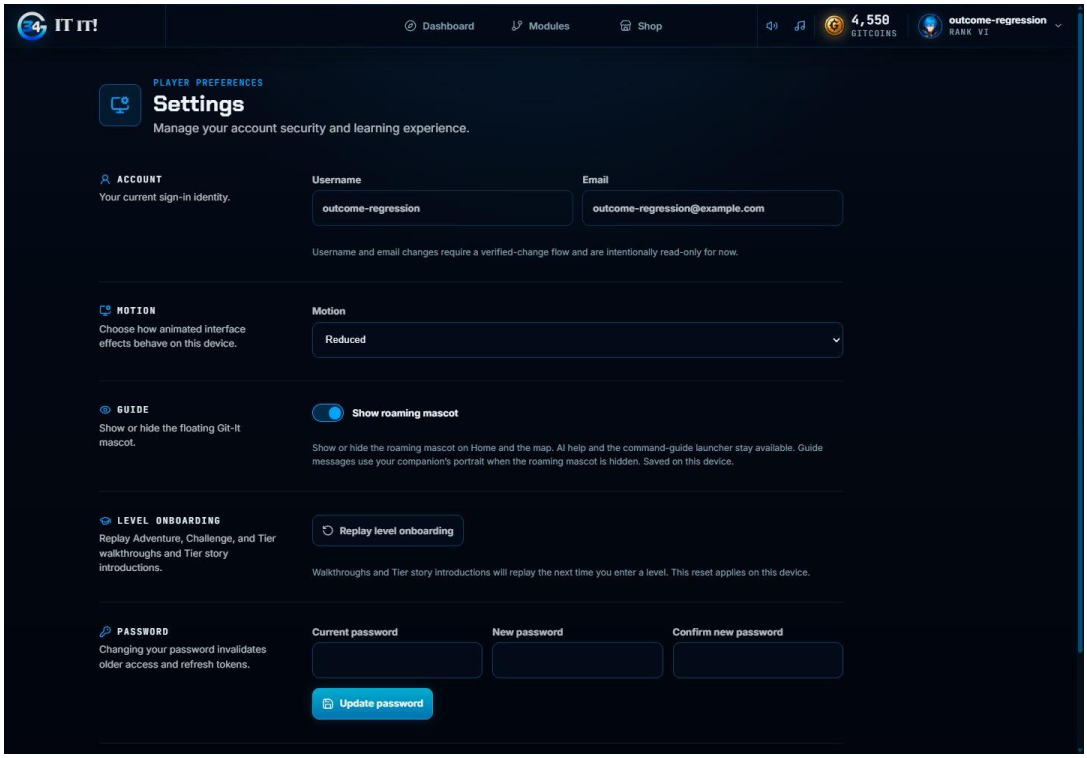


Figure 64. Account settings.

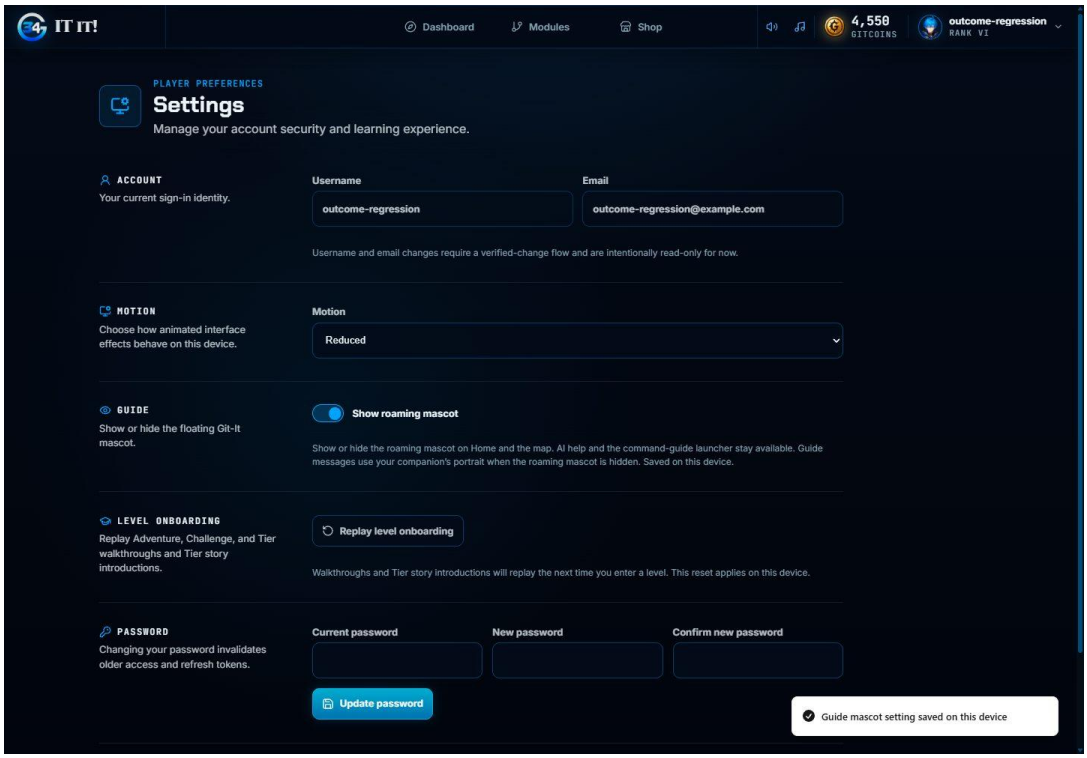


Figure 65. A preference saved.

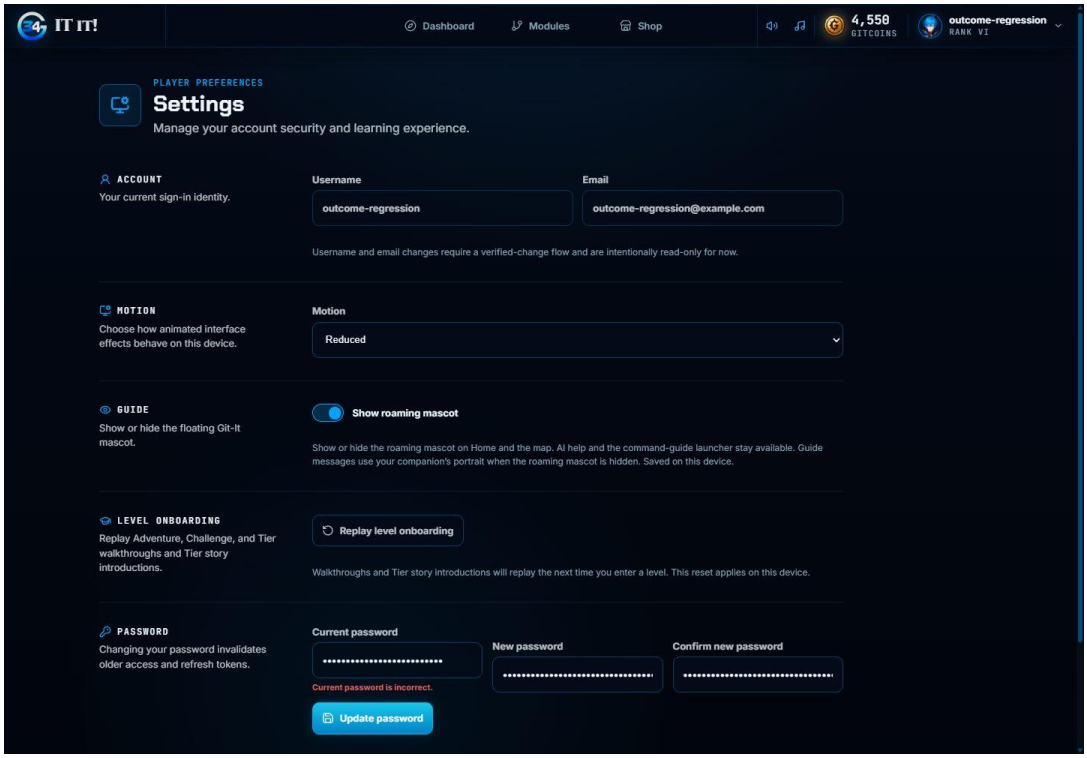


Figure 66. A wrong current password is rejected.

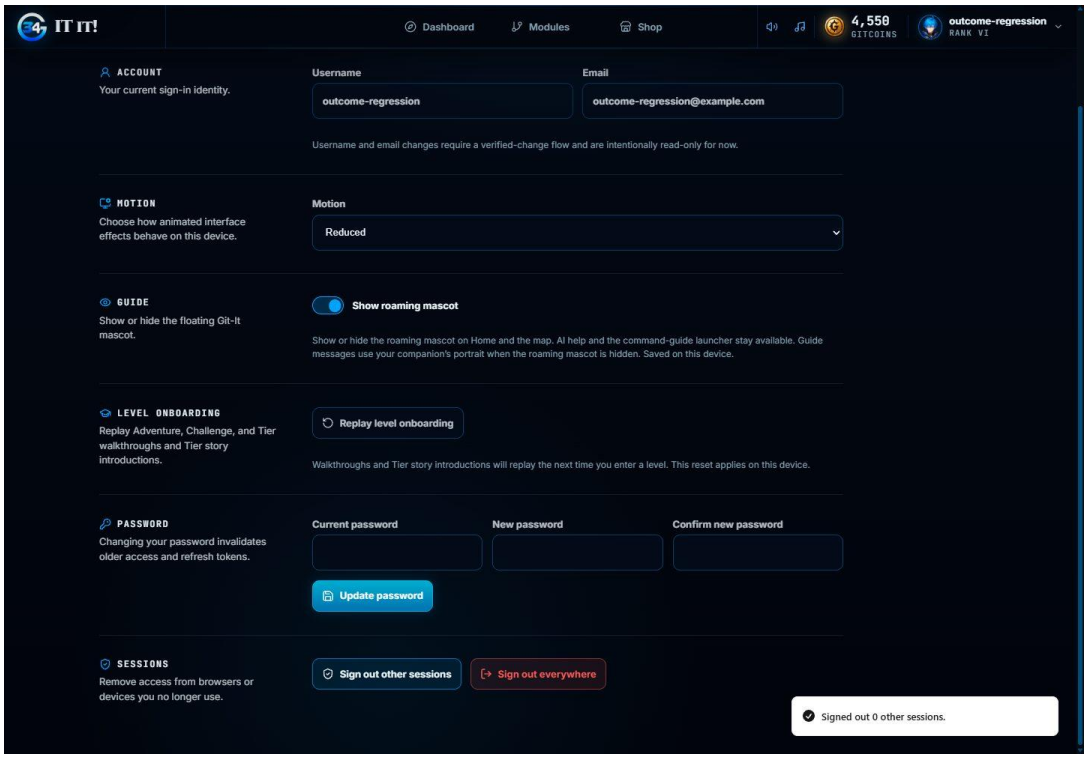


Figure 67. Signing out other sessions keeps the current session.

3.29 Admin Console

3.29.1 Purpose

Confirm the admin console's read views and learner KPI inspection.

3.29.2 Inputs

- A local staff account.
- The test learner's earned data.

3.29.3 Expected Outputs & Pass/Fail Criteria

- KPI Overview, Learners, Economy, Analytics and Settings load without errors.
- Learner search and the individual KPI panel work.

Actual Result: PASS. The five views loaded. Learner search, the individual KPI panel, the KPI reference, a date range and a module's detail worked. No GitCoin adjustment or feature-flag change was submitted.

Evidence: Figures 68 to 72.

3.29.4 Test Procedure

1. Sign in as staff and open each admin view.
2. Search for the test learner and open its KPI panel.
3. Expand the KPI reference and a module, and apply a date range.

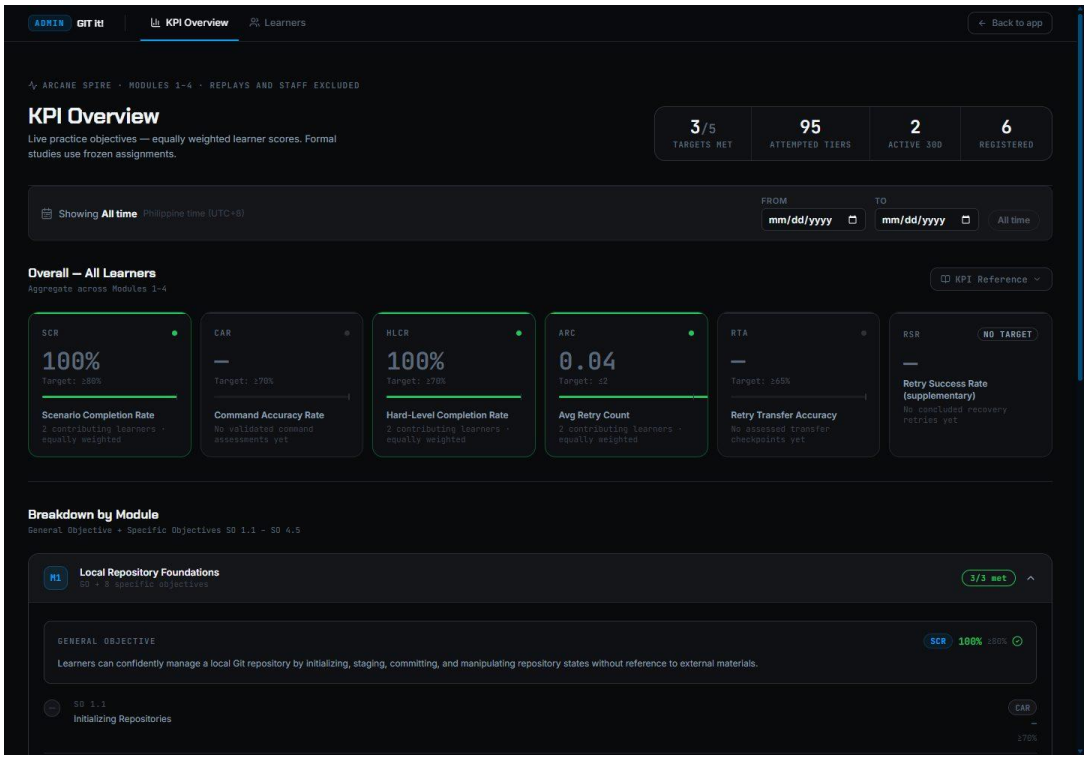


Figure 68. KPI Overview.

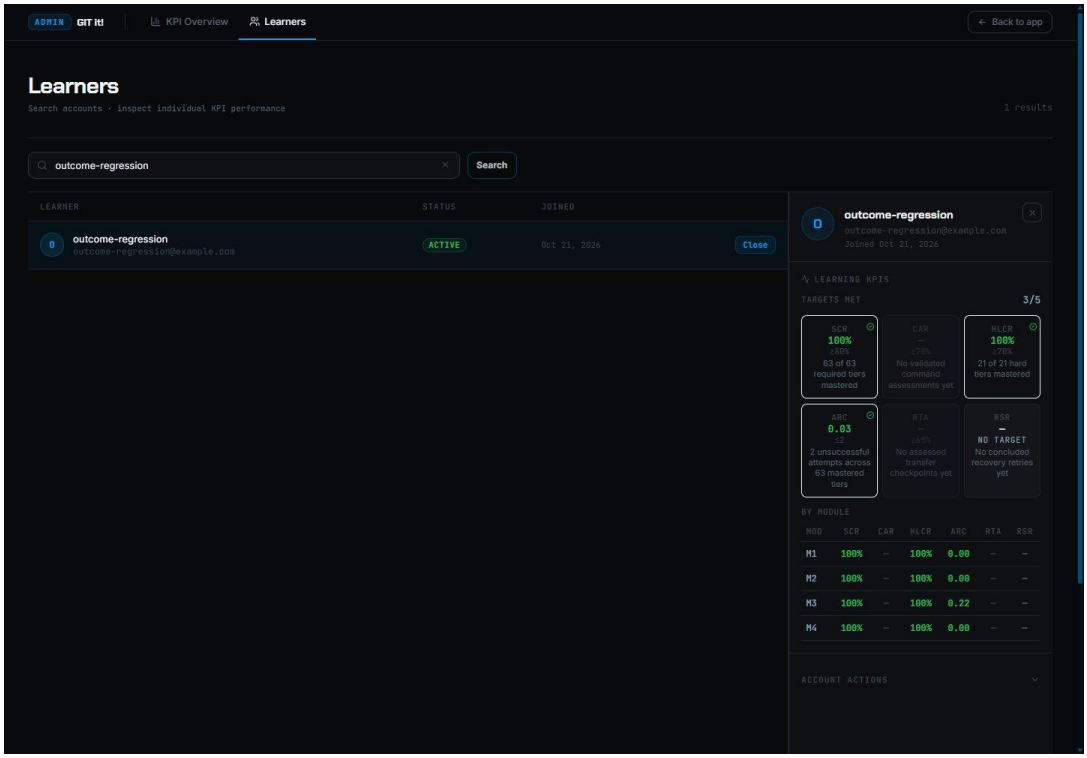


Figure 69. A learner's KPI panel.

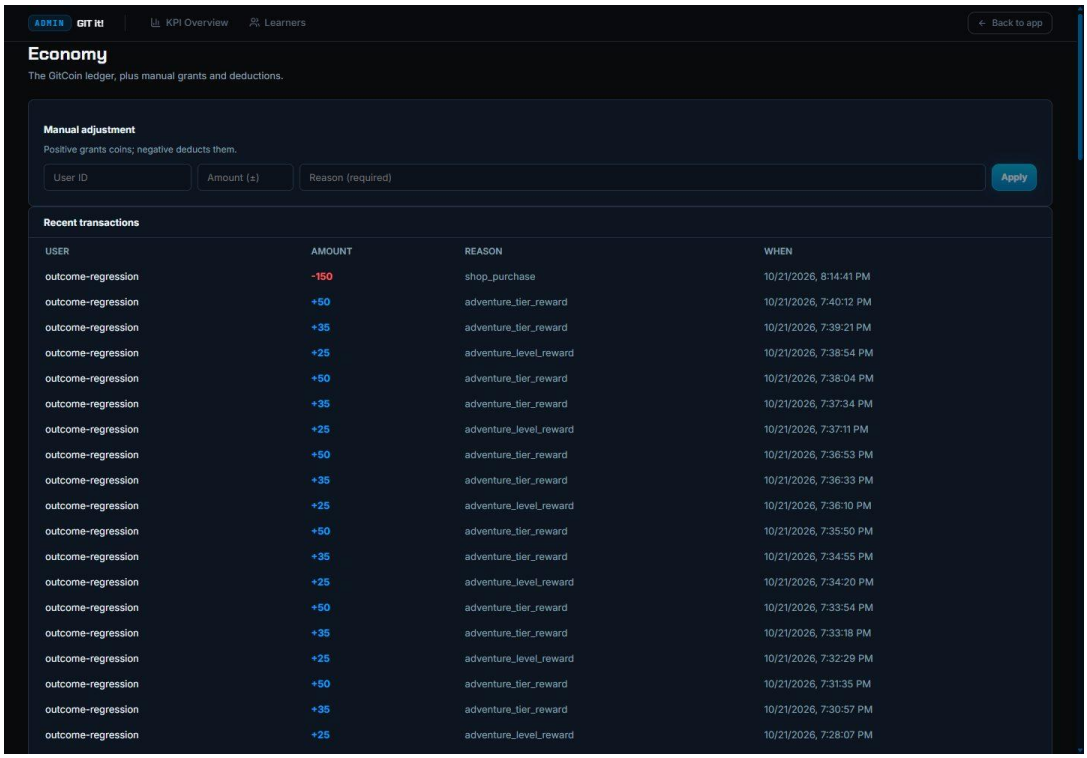


Figure 70. The Economy ledger.

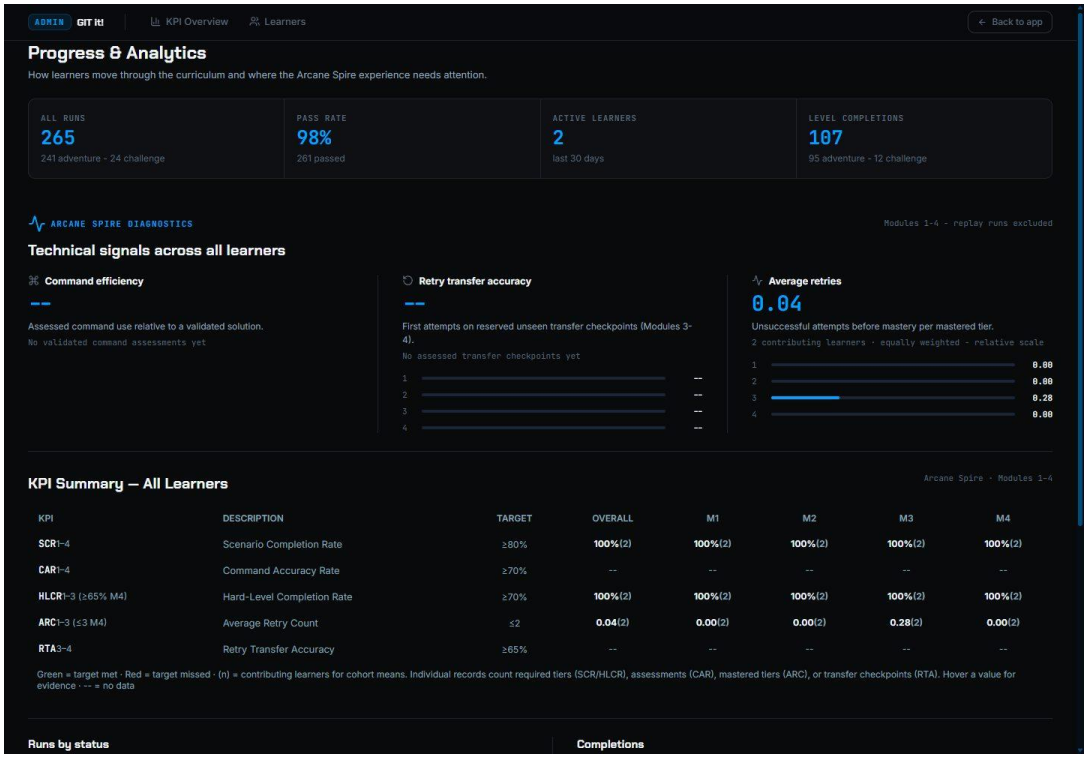


Figure 71. Progress & Analytics.

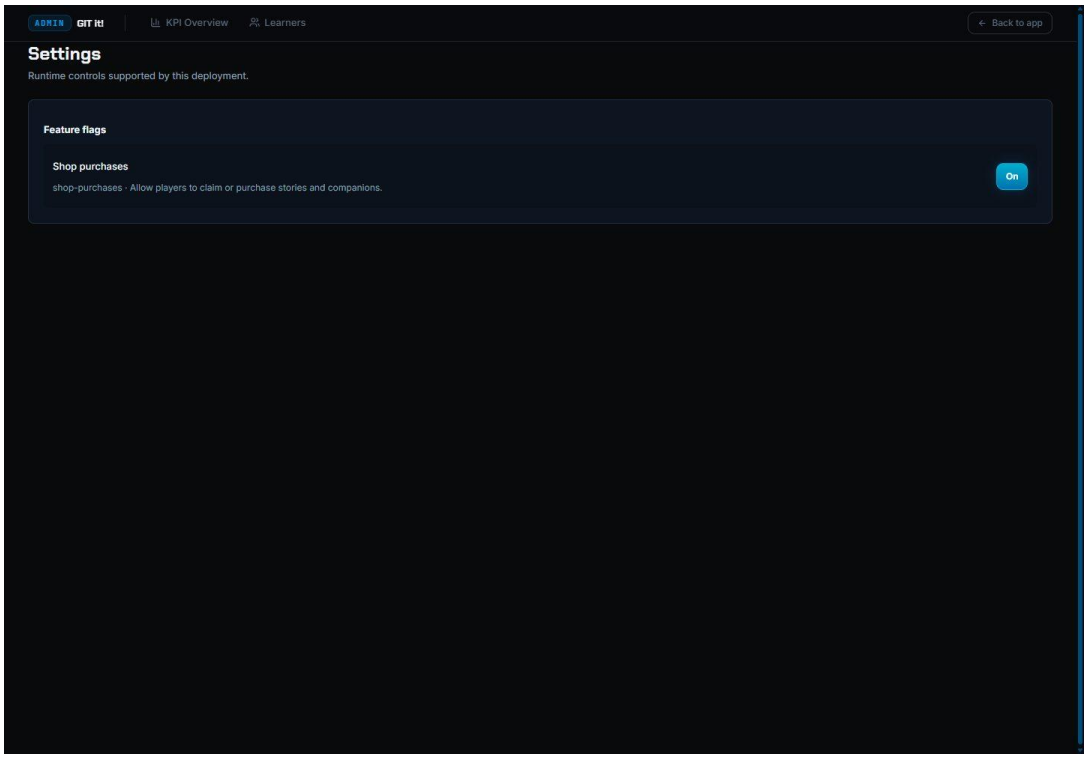


Figure 72. Admin Settings with the feature flags, unchanged.

3.30 Keyboard Access

3.30.1 Purpose

Confirm that the tested learner flows work from the keyboard.

3.30.2 Inputs

- The skip link, view selector, account menu, dialogs and the drill.

3.30.3 Expected Outputs & Pass/Fail Criteria

- Keyboard actions activate controls and close overlays.
- Focus returns to a useful place without trapping the player.

Actual Result: PASS. The skip link moved focus to the main content, the view selector responded to the keyboard, and Escape closed the account panel and returned focus to its button. The exit dialog and the drill also worked from the keyboard.

Evidence: Figures 73 to 74.

3.30.4 Test Procedure

1. Press Tab to reach Skip to content, then Enter.
2. Change the view with the keyboard.
3. Open the account menu and press Escape.
4. Cancel a dialog and exit the drill with Escape.

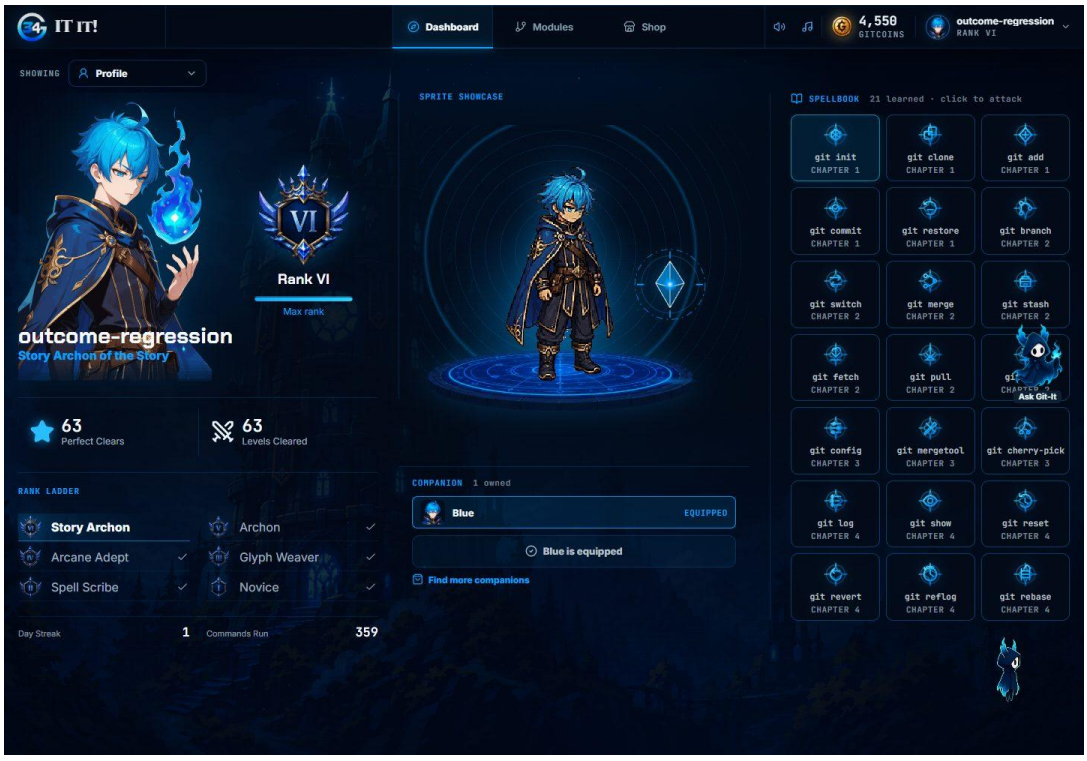


Figure 73. Skip to content moves focus to the main content.

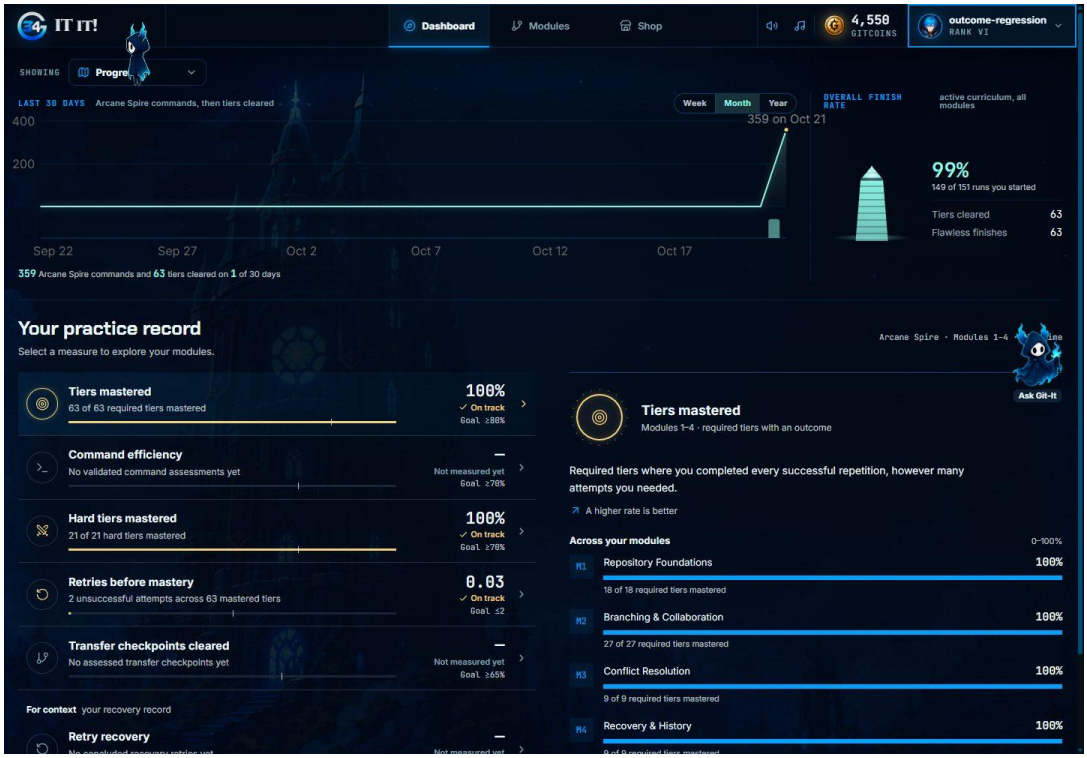


Figure 74. The view selector changed from the keyboard.

3.31 Command Confirmation Time

3.31.1 Purpose

Measure how long the server takes to verify and record a submitted command and return its result, as seen by the client (NFR-PERF-01).

3.31.2 Inputs

- git status submitted in a started Level 1 Easy run. It is a free diagnostic that travels the full submit path (request validation, transition check, step log and run payload) without spending the command budget.
- The Run A and Run B workloads of case 3.32.

3.31.3 Expected Outputs & Pass/Fail Criteria

- 95th percentile of command confirmation time ≤ 1.5 s with up to 20 learners and ≤ 4 s with 40 learners under the NFR-PERF-04 workload (SRS 1.4). Runs 4 and 5 were judged against SRS 1.3: ≤ 1 s.

Actual Result: Run 4: FAIL. Even with only 4 to 5 learners playing, the 95th percentile was 1.35 to 1.47 s and no submit took less than 1.01 s (median 1.08 s). It rose to 4.3 s at 16 learners and 7.7 s at 28. The database spent about 1.2 ms on each submit; the rest is the network between the API in Singapore and the database in Tokyo, crossed about 13 times per submit, plus queueing on the server's two threads (INC-10).

Actual Result (Run 5): FAIL, improved. Up to 16 learners the 95th percentile was 0.39 to 1.36 s (0.85 s at 16; Run 4: 4.26 s). It rose to 2.2 s at 24 learners and 2.9 s with 40 learners playing

(median 0.89 s), then fell to 1.5 s in the last minute, after every learner had signed in. Moving the database to the API's region removed most of the delay per query; the remaining time is processing on the 0.1 CPU instance, where sign-ins compete with submits (INC-10).

Actual Result (Run 6): PASS. With up to 20 learners the 95th percentile was 0.74 s (median 0.24 s). With all 40 learners playing during the 10-minute hold it was 1.83 s (median 0.48 s), and 0.96 to 2.27 s in any one minute of the hold. The highest minute overall was 2.99 s, while the last learners were joining.

Evidence: Tables A-7 to A-9, Figure 79. Run 5: Tables A-12 and A-13, Figure 80. Run 6: Tables A-15 and A-16, Figure 81.

3.31.4 Test Procedure

1. Create the test accounts and take a pg_stat_statements snapshot.
2. Run loadtest.py against https://git-it-34.vercel.app (Run A, then Run B).
3. Compute the percentiles of submit latencies with HTTP 200 from the request logs.

3.32 Concurrent Learner Workload

3.32.1 Purpose

Check that 40 learners can play at once while NFR-PERF-01 holds and the API keeps pace with at least 4 requests per second (NFR-PERF-04), and that fewer than 1% of requests fail (NFR-PERF-07).

3.32.2 Inputs

- Run A: 40 virtual learners joining over 60 s, then 10 minutes of play: git status about every 10 s, profile and dashboard about every 60 s, 30 s timeout.
- Run B: the same learners joining one every 15 s to find the capacity. The script stops itself after more than 30 failed requests in one minute.
- Run 6: the SRS 1.4 workload: 40 learners joining one every 15 s, then 10 minutes at 40. The script refreshes an expired access token and retries, as the app does.

3.32.3 Expected Outputs & Pass/Fail Criteria

- 40 learners join one every 15 s, and all 40 then play together for 10 minutes (SRS 1.4). Runs 4 and 5 were judged against SRS 1.3, under which all 40 also had to sign in within 60 s (Run A).
- Throughput ≥ 4 requests/s while NFR-PERF-01 holds.
- Fewer than 1% of requests fail with HTTP 5xx, are rejected with HTTP 429 or time out (SRS 1.4 counts HTTP 429; SRS 1.3 did not).

Actual Result: Run 4: FAIL. Run A: sign-ins took 2 to 19 s each while they queued. After 25 s the hosting platform answered 503 and then 429 for about 30 s, so 21 of 40 learners could not sign in, 14 could not start the tier and only 5 played (0.58 requests/s). Run B: throughput levelled off at about 2 requests/s from 16 learners while latency kept rising, and at 28 learners the platform began rejecting requests, which stopped the test after 6 min 47 s. HTTP 5xx stayed under 1% (0.68% and 0.48%), but the platform rejected 10.7% (Run A) and 4.5% (Run B) of requests with HTTP 429, which learners see as failures. No request timed out.

Actual Result (Run 5): FAIL, improved. Run B: all 40 learners signed in and played. With 40 learners throughput was 4.75 requests/s, and none of the 2,151 requests failed or was rejected (Run 4: stopped at 28 learners). Command confirmation was 2.9 s at the 95th percentile, so NFR-PERF-

01 is not met with 40 learners. Run A: 40 sign-ins within one minute again overloaded the instance (sign-in median 17 s). After about 30 s the hosting edge answered 503 and 429, and the stopping rule, now applied to Run A as well, ended the run after 45 s, when 21 of the 30 learners who had joined had signed in.

Actual Result (Run 6): PASS. All 40 learners signed in, started Level 1 Easy and played. Throughput was 4.96 requests/s during the 10-minute hold. One of 4,541 requests failed (HTTP 502 in minute 8, 0.02%); none was rejected with HTTP 429 and none timed out. 20 requests got HTTP 401 when a learner's 15-minute access token expired; each was followed by a successful refresh and retry, as in the app, so they are not failures. A first attempt at 12:32 is not counted: its script did not yet refresh tokens, so from minute 16 its requests failed with HTTP 401 (Table A-1).

Evidence: Tables A-6 to A-9, Figure 79. Run 5: Tables A-12 and A-13, Figure 80. Run 6: Tables A-15 and A-16, Figure 81.

3.32.4 Test Procedure

1. Run A: loadtest.py with 40 learners, --ramp 60 --duration 600 --think 10 --home-every 60 --timeout 30.
2. Wait until /api/health/ready/ answers 200 again.
3. Run B: --ramp 600 --duration 120 with the stopping rule.
4. Summarize throughput, failed requests and latency, overall and by minute.

3.33 Immediate Terminal Feedback

3.33.1 Purpose

Confirm that the terminal shows a command's output from the local simulation before the server confirms it (NFR-PERF-02).

3.33.2 Inputs

- git status typed and submitted with real Enter key presses, 10 times, in Level 1 Easy on production.
- Run 6: the same, with the commands 6 s apart.

3.33.3 Expected Outputs & Pass/Fail Criteria

- The command line and its output are painted ≤ 200 ms after Enter.

Actual Result: Run 4: PASS. The output appeared 7 to 19 ms after Enter (median 11.5 ms) for all 10 commands, while the server's confirmation took about 1 s.

Actual Result (Run 6): PASS. 11.5 to 28.6 ms after Enter (median 16.4 ms) for all 10 commands (O-8).

Evidence: Table A-11. Run 6: Table A-18.

3.33.4 Test Procedure

1. Sign in as perf01 and start Level 1 Easy.
2. Install the timing listener (feedback_probe.js). It timestamps the Enter key press and the first frame painted after the command text appears again in the terminal.
3. Type git status and press Enter 10 times.

3.34 Home Load Time

3.34.1 Purpose

Measure how long Home takes to become usable, with the profile and progress shown (NFR-PERF-03).

3.34.2 Inputs

- /home reloaded 5 times while signed in, browser cache warm.
- Run 6: /home reloaded 5 times at 10 Mbps, browser cache warm.

3.34.3 Expected Outputs & Pass/Fail Criteria

- Profile and progress shown ≤ 3 s after the reload starts.

Actual Result: Run 4: PASS. Profile and progress data arrived 2.2 to 3.3 s after the reload (median 2.6 s); 4 of 5 loads were within 3 s. The rest of Home (statistics and performance panels) finished at 3.5 to 4.0 s. A first visit also downloads about 545 KB of compressed scripts and styles, about 0.45 s at 10 Mbps. The chapter map shown after sign-in is slower (O-6).

Actual Result (Run 6): PASS. At 10 Mbps, profile and progress data arrived 0.78 to 0.96 s after the reload (median 0.86 s), and every Home panel had loaded after 0.94 to 1.21 s.

Evidence: Table A-11. Run 6: Table A-18.

3.34.4 Test Procedure

1. Sign in as perf01.
2. Reload /home five times, waiting 9 s each time.
3. Read the browser's resource timing: Home is usable when /api/auth/me/ (profile) and /api/progress/dashboard/ (progress) have both returned.

3.35 Database Query Time

3.35.1 Purpose

Check that the queries with the highest total time stay fast under the workload (NFR-PERF-05).

3.35.2 Inputs

- pg_stat_statements snapshots at 03:30:28 and 03:38:57 (inside Run A's steady state), and before and after Run B.
- Run 6: snapshots before and after the Run 6 workload (12:53:58 and 13:14:14).

3.35.3 Expected Outputs & Pass/Fail Criteria

- Average ≤ 100 ms for each of the ten queries with the highest total time.

Actual Result: Run 4: PASS. In Run A the top ten queries averaged 0.06 to 1.69 ms. In Run B no application query averaged more than 1.7 ms. Two queries of Supabase's own backup job (44 ms and 126 ms, one call each) ran during Run B; they are not application queries.

Actual Result (Run 5): PASS. During Run B, 193 distinct statements ran 28,141 times in 2.65 s in total. The ten with the highest total time averaged 0.04 to 1.61 ms, and no application query averaged more than 10 ms.

Actual Result (Run 6): PASS. During the Run 6 workload, 138 distinct statements ran 56,445 times in 5.69 s in total. The ten application queries with the highest total time averaged 0.04 to 1.27 ms,

and no application query averaged more than 7.1 ms.

Evidence: Table A-10. Run 5: Table A-14. Run 6: Table A-17.

3.35.4 Test Procedure

1. Take a pg_stat_statements snapshot before the workload.
2. Take another after it.
3. Subtract calls and total time per query and rank the queries by total time.

3.36 Level Start Time

3.36.1 Purpose

Measure the time from starting or resuming a tier until the workspace is ready (NFR-PERF-06).

3.36.2 Inputs

- Level 1 Easy started once and resumed five times from the chapter map, signed in as perf01.
- Run 6: Level 1 Easy at 10 Mbps, signed in as perf01: one first visit with the HTTP cache cleared, then resumes from the chapter map.

3.36.3 Expected Outputs & Pass/Fail Criteria

- From the Start click until the terminal accepts input: ≤ 2 s at the 95th percentile when the level's art is cached, and ≤ 8 s when it must be downloaded, on a 10 Mbps connection (SRS 1.4). Runs 4 and 5 were judged against SRS 1.3: ≤ 2 s.

Actual Result: Run 4: FAIL. 4.0 to 4.8 s (median 4.1 s). Nearly all of it is the single start request (POST /api/adventure-level-tiers/43/runs/), which took about 3.9 s with no other load and 3.1 to 9.4 s during Run B. The cause is the one recorded in INC-10.

Actual Result (Run 5): FAIL, improved. The start request now takes 0.38 to 0.60 s (Run 4: about 3.9 s). Reopening the level took 0.93 to 1.45 s, within the target, but the first open took 4.55 s. The terminal became ready just after the level's battle images finished loading, and on a first visit the browser downloads 6.4 MB of level images and scripts; the chapter's battle background alone is 2.1 MB. A learner's first visit to a level therefore still exceeds 2 s (INC-10).

Actual Result (Run 6): PASS. At 10 Mbps, a first visit with the HTTP cache cleared took 6.15 s (≤ 8 s): the start request took 1.14 s and the level downloaded 3.9 MB of battle art. Resumes with the art cached took 0.57 to 0.77 s (8 samples, 95th percentile 0.77 s; ≤ 2 s). Two resumes whose encounter included a monster not met before downloaded its art first and took 2.10 s and 5.66 s (≤ 8 s). One sample (12.1 s) was discarded: Start was clicked while the chapter map was still downloading its own art after the cache was cleared.

Evidence: Table A-11. Run 5: Table A-13. Run 6: Table A-18.

3.36.4 Test Procedure

1. Open the chapter map and the Level 1 panel.
2. Arm a timer on the Easy tier button.
3. Click Start and record when the Git command input is enabled.
4. Return to the chapter map and repeat.

3.37 Cold Start Recovery

3.37.1 Purpose

Measure the first request after the free API host has slept (NFR-PERF-08).

3.37.2 Inputs

- The first request after the API had been idle for more than 15 minutes, on two occasions.

3.37.3 Expected Outputs & Pass/Fail Criteria

- The first request completes in ≤ 120 s, and the app shows Restoring session rather than an error.

Actual Result: PASS. The first requests took 97.6 s and 86.3 s and returned HTTP 200. Later requests took about 0.4 s (median of 10). Run 3 confirmed the Restoring session screen (O-2).

Evidence: Table A-11.

3.37.4 Test Procedure

1. Leave the API idle for more than 15 minutes.
2. Request /api/health/live/ and time the response.
3. Request /api/health/ready/ ten times.

3.38 AI Tutor Response Time

3.38.1 Purpose

Measure how long the AI tutor takes to answer (NFR-PERF-09).

3.38.2 Inputs

- Six general Git questions asked with the Home context, one every 6 s, within the limit of 12 per minute.
- Run 6: six questions not asked before, so that no reply came from the answer cache.

3.38.3 Expected Outputs & Pass/Fail Criteria

- A reply, a decline or the recovery message arrives within 30 s.

Actual Result: Run 4: PASS. All six questions were answered (HTTP 200) in 2.5 to 4.4 s (median 3.3 s).

Actual Result (Run 6): PASS. Six questions not asked before were answered (HTTP 200) in 1.46 to 4.60 s (median 2.05 s). Questions already asked in Run 4 came back from the answer cache in about 0.3 s, so they were not reused.

Evidence: Table A-11. Run 6: Table A-18.

3.38.4 Test Procedure

1. Sign in through the API as perf01.
2. Send each question to /api/tutoring/chat/ with the Home context.
3. Record the time until the complete reply arrives.

Appendix (Test Logs)

A.1 Test Execution Log

Table A-1 lists each execution in Runs 3 to 6. Run 3 used the local build described in section 2.3; Run 4 used the production deployment in Table 2-2, Run 5 the changes in Table 2-3, and Run 6 the changes in Table 2-4.

Test	Method	Outcome
3.1 to 3.4	pytest; makemigrations --check; Ruff; Vitest; ESLint; Knip; npm run build; guard scripts	All passed: 987 backend and 1,210 frontend tests
3.5 to 3.12, 3.14 to 3.19	Scripted Chromium session on the local build	All passed
3.13	Chromium session on https://git-it-34.vercel.app, read-only except one test learner	Passed
3.20 and 3.21	All 63 Adventure tiers played with authored solutions; saved records checked	63 tiers and 149 scenarios passed, three stars each
3.22	Interactive scenarios within the curriculum run	Passed
3.23	All 12 Challenge trials after the Adventure tiers	12 trials passed, three stars each
3.24 to 3.30	Scripted Chromium session, local learner and staff accounts	All passed
Generated targets	Replay of the 399 authored variant solutions	All matched
3.31, 3.32 (Run A)	loadtest.py on production: 40 virtual learners, 60 s ramp, 10 minutes	Failed: 5 of 40 learners played; p95 1.47 s; 10.7% rejected
3.31, 3.32 (Run B)	loadtest.py on production: one learner joining every 15 s	Failed: about 2 requests/s at most; stopped at 28 learners
3.33, 3.34, 3.36	Chromium on production, account perf01	3.33 and 3.34 passed; 3.36 failed
3.35	pg_stat_statements snapshots around Runs A and B	Passed
3.37	Timing of the first request after the API slept	Passed
3.38	Six tutor questions through the API	Passed
INC-08, INC-09 retests	pytest, Redis outage probe, CI run and production version check	Passed
3.31, 3.32 (Run 5, Run	loadtest.py on production: 40 virtual learners, 60 s	Failed: stopped after 45 s;

Test	Method	Outcome
A)	ramp	21 of the 30 joined learners signed in
3.31, 3.32 (Run 5, Run B)	loadtest.py on production: one learner joining every 15 s, 2 minutes at 40	Failed: 40 of 40 played at 4.75 requests/s with no failed requests, but p95 2.9 s at 40
3.35 (Run 5)	pg_stat_statements snapshots around Runs A and B	Passed
3.36 (Run 5)	Chromium on production, account perf01	Failed: first open 4.55 s; reopened 0.93 to 1.45 s
3.32 (Run 6, first attempt)	loadtest.py on production: one learner every 15 s, then 10 minutes at 40, without token refresh	Not counted: from minute 16 requests failed with HTTP 401 because the script did not refresh expired tokens
3.31, 3.32 (Run 6)	loadtest.py on production: one learner every 15 s, then 10 minutes at 40, with token refresh	Passed: 40 of 40 played at 4.96 requests/s; p95 1.83 s at 40; 1 of 4,541 requests failed
3.33, 3.34, 3.36 (Run 6)	Chromium on production at 10 Mbps, account perf01	Passed
3.35 (Run 6)	pg_stat_statements snapshots around the Run 6 workload	Passed
3.38 (Run 6)	Six new tutor questions through the API	Passed
INC-10 retest	Cases 3.31, 3.32 and 3.36 against SRS 1.4	Passed; INC-10 closed

Table A-1. Test execution log.

A.2 Test Results

Run 3: all 30 functional test cases (3.1 to 3.30) passed. Run 4: of the eight performance test cases (3.31 to 3.38), five passed and three failed. Run 5: after two corrective actions for INC-10, 3.35 passed again and 3.31, 3.32 and 3.36 improved but still failed. Run 6: after the remaining corrective actions, and against the targets of SRS 1.4, all seven performance cases it repeated (3.31 to 3.36 and 3.38) passed; 3.37 passed in Run 4 and was not repeated, because the API's host settings and startup did not change. Table A-2 lists the result of each case, Table A-3 shows the completion screen of every Adventure tier, and Table A-4 shows the completion screen of every Challenge Gate trial. A.3 records the defects found in runs 1, 2 and 4, with their corrective actions and retest results, and A.4 holds the performance measurements.

ID	Test case	Actual Result	Evidence
3.1	Backend Automated Regression Suite	PASS	Test logs
3.2	Backend Code Quality Gates	PASS	Test logs
3.3	Frontend Automated Suite and Build	PASS	Test logs
3.4	Architecture, Contract and Deployment Guards	PASS	Test logs
3.5	Sign In and Sign Out	PASS	Figures 1 to 3
3.6	Account Registration	PASS	Figures 4 to 5
3.7	Password Reset	PASS	Figures 6 to 8
3.8	Onboarding and Companion Purchase	PASS	Figures 9 to 10
3.9	Adventure Terminal Objectives and Exit	PASS	Figures 11 to 13
3.10	Progress Persistence	PASS	Figure 14
3.11	Access Control and Routing	PASS	Figures 15 to 16
3.12	Responsive Layout at Phone Width	PASS	Figure 17
3.13	Production Smoke Test	PASS	Figures 18 to 23
3.14	Ask Git-It: General Git Help	PASS	Figures 24 to 27
3.15	Ask Git-It: Scenario Answer Protection	PASS	Figures 28 to 29
3.16	Repository Diagrams	PASS	Figures 30 to 33
3.17	Project Files Panel	PASS	Figures 34 to 35
3.18	Workspace Aids	PASS	Figures 36 to 37
3.19	Run Control: Failure, Retry and Replay	PASS	Figures 38 to 43
3.20	Easy Tier of Every Adventure Level	PASS	Table A-3
3.21	Medium and Hard Tiers of Every Adventure Level	PASS	Table A-3
3.22	Interactive Staging, Conflicts and Merge Tool	PASS	Figures 44 to 45
3.23	Challenge Gates	PASS	Table A-4
3.24	Squire's Drill	PASS	Figures 46 to 49
3.25	Chapter Features	PASS	Figures 50 to 54
3.26	Dashboard Views	PASS	Figures 55 to 58

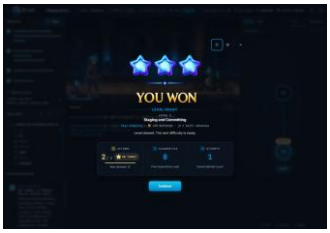
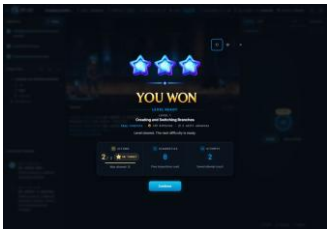
ID	Test case	Actual Result	Evidence
3.27	Shop and Loadout	PASS	Figures 59 to 63
3.28	Account Settings	PASS	Figures 64 to 67
3.29	Admin Console	PASS	Figures 68 to 72
3.30	Keyboard Access	PASS	Figures 73 to 74
3.31	Command Confirmation Time	FAIL	Tables A-7 to A-9, Figure 79
3.32	Concurrent Learner Workload	FAIL	Tables A-6 to A-9, Figure 79
3.33	Immediate Terminal Feedback	PASS	Table A-11
3.34	Home Load Time	PASS	Table A-11
3.35	Database Query Time	PASS	Table A-10
3.36	Level Start Time	FAIL	Table A-11
3.37	Cold Start Recovery	PASS	Table A-11
3.38	AI Tutor Response Time	PASS	Table A-11
3.31 (Run 5)	Command Confirmation Time	FAIL	Tables A-12 and A-13, Figure 80
3.32 (Run 5)	Concurrent Learner Workload	FAIL	Tables A-12 and A-13, Figure 80
3.35 (Run 5)	Database Query Time	PASS	Table A-14
3.36 (Run 5)	Level Start Time	FAIL	Table A-13
3.31 (Run 6)	Command Confirmation Time	PASS	Tables A-15 and A-16, Figure 81
3.32 (Run 6)	Concurrent Learner Workload	PASS	Tables A-15 and A-16, Figure 81
3.33	Immediate Terminal Feedback	PASS	Table A-18

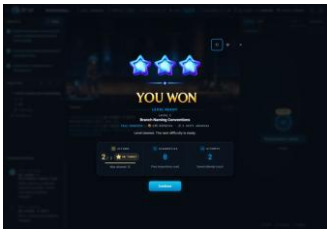
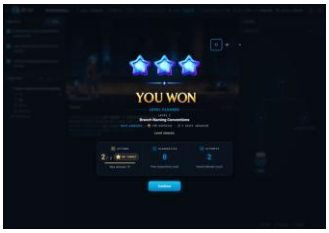
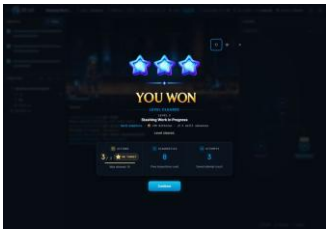
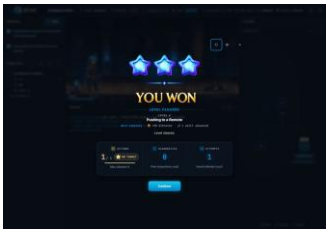
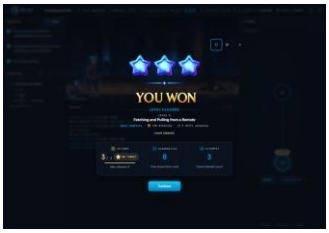
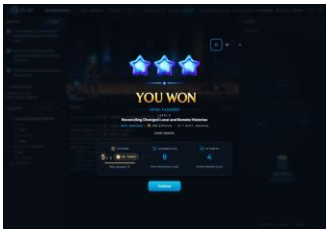
ID	Test case	Actual Result	Evidence
(Run 6)			
3.34 (Run 6)	Home Load Time	PASS	Table A-18
3.35 (Run 6)	Database Query Time	PASS	Table A-17
3.36 (Run 6)	Level Start Time	PASS	Table A-18
3.38 (Run 6)	AI Tutor Response Time	PASS	Table A-18

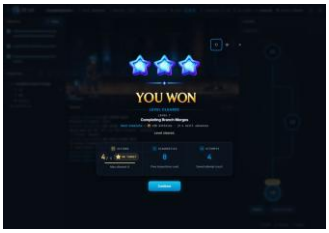
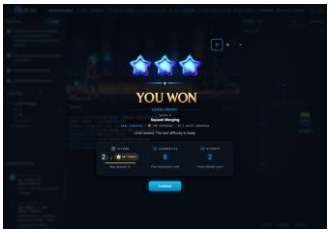
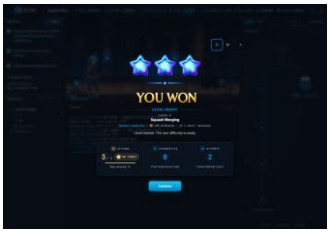
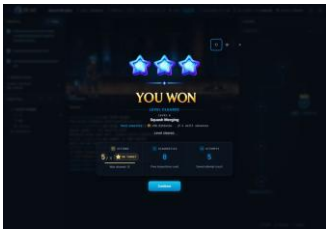
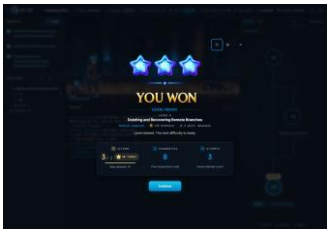
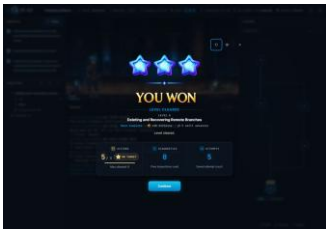
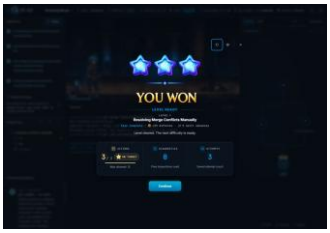
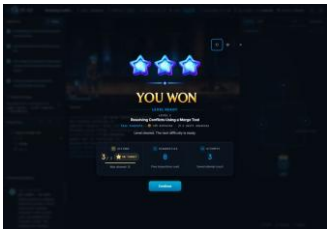
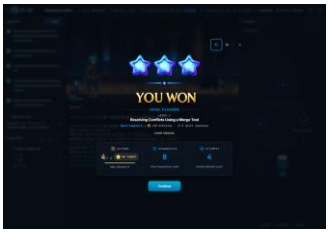
Table A-2. Results by test case.

Table A-3 lists every Adventure level by chapter and level number, with the completion screen of each tier. The number after PASS is the count of ordered scenarios in that tier; every scenario earned three stars.

Level	Easy	Medium	Hard
Chapter 1, Level 1: Initializing Repositories	 *PASS 4*	 *PASS 2*	 *PASS 2*
Chapter 1, Level 2: Cloning Remote Repositories	 *PASS 4*	 *PASS 2*	 *PASS 2*

Level	Easy	Medium	Hard
Chapter 1, Level 3: Staging and Committing	 *PASS 3*	 *PASS 2*	 *PASS 2*
Chapter 1, Level 4: Partial Staging	 *PASS 2*	 *PASS 2*	 *PASS 2*
Chapter 1, Level 5: Amending Commits	 *PASS 2*	 *PASS 2*	 *PASS 2*
Chapter 1, Level 6: Unstaging and Discarding Changes	 *PASS 3*	 *PASS 2*	 *PASS 2*
Chapter 2, Level 1: Creating and Switching Branches	 *PASS 3*	 *PASS 2*	 *PASS 2*

Level	Easy	Medium	Hard
Chapter 2, Level 2: Branch Naming Conventions	 *PASS 3*	 *PASS 2*	 *PASS 2*
Chapter 2, Level 3: Stashing Work in Progress	 *PASS 3*	 *PASS 2*	 *PASS 2*
Chapter 2, Level 4: Pushing to a Remote	 *PASS 3*	 *PASS 2*	 *PASS 2*
Chapter 2, Level 5: Fetching and Pulling from a Remote	 *PASS 4*	 *PASS 2*	 *PASS 2*
Chapter 2, Level 6: Reconciling Diverged Local and Remote Histories	 *PASS 3*	 *PASS 2*	 *PASS 2*

Level	Easy	Medium	Hard
Chapter 2, Level 7: Completing Branch Merges	 *PASS 3*	 *PASS 2*	 *PASS 2*
Chapter 2, Level 8: Squash Merging	 *PASS 3*	 *PASS 2*	 *PASS 2*
Chapter 2, Level 9: Deleting and Recovering Remote Branches	 *PASS 3*	 *PASS 2*	 *PASS 2*
Chapter 3, Level 1: Resolving Merge Conflicts Manually	 *PASS 2*	 *PASS 3*	 *PASS 2*
Chapter 3, Level 2: Resolving Conflicts Using a Merge Tool	 *PASS 2*	 *PASS 2*	 *PASS 2*

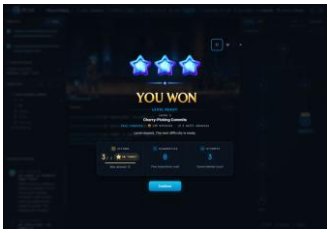
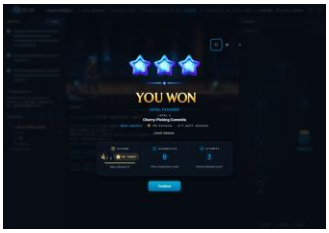
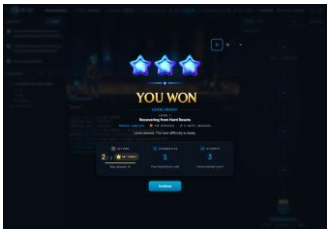
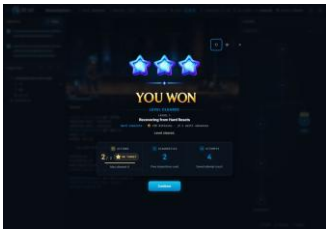
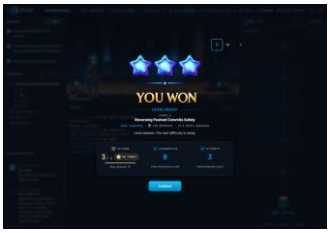
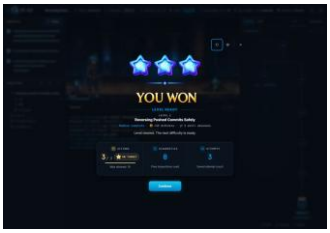
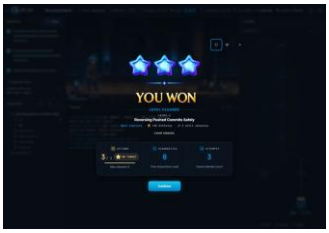
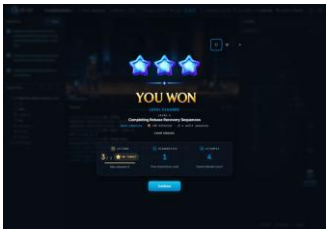
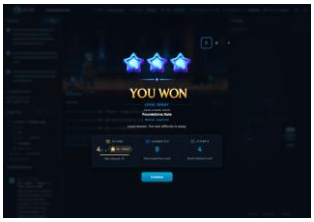
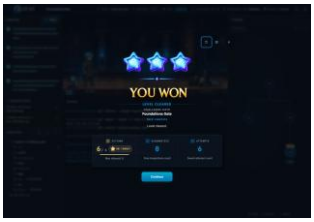
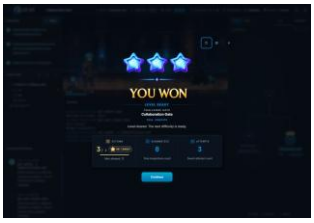
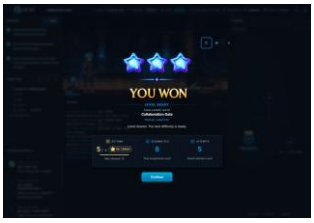
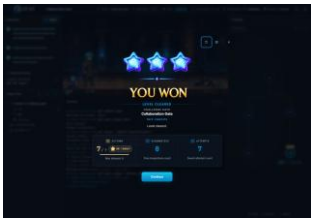
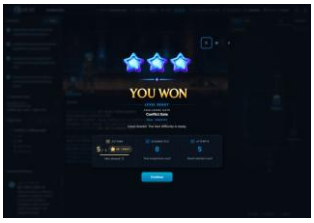
Level	Easy	Medium	Hard
Chapter 3, Level 3: Cherry-Picking Commits	 *PASS 3*	 *PASS 3*	 *PASS 3*
Chapter 4, Level 1: Recovering from Hard Resets	 *PASS 2*	 *PASS 2*	 *PASS 2*
Chapter 4, Level 2: Reversing Pushed Commits Safely	 *PASS 2*	 *PASS 2*	 *PASS 2*
Chapter 4, Level 3: Completing Rebase Recovery Sequences	 *PASS 3*	 *PASS 3*	 *PASS 3*

Table A-3. Adventure coverage: completion screen of every tier.

Table A-4 lists the 12 Challenge Gate trials. They were played after the Adventure tiers, in order, as normal first clears.

Trial	Gate and difficulty	Actual Result	Completion screen
1	Foundations Gate, Easy	PASS, 3 stars	

Trial	Gate and difficulty	Actual Result	Completion screen
2	Foundations Gate, Medium	PASS, 3 stars	
3	Foundations Gate, Hard	PASS, 3 stars	
4	Collaboration Gate, Easy	PASS, 3 stars	
5	Collaboration Gate, Medium	PASS, 3 stars	
6	Collaboration Gate, Hard	PASS, 3 stars	
7	Conflict Gate, Easy	PASS, 3 stars	

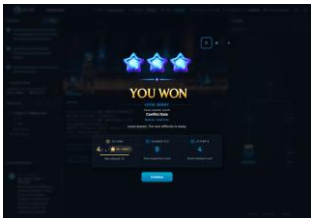
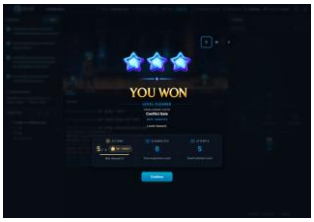
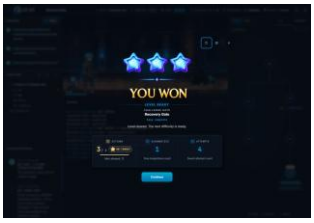
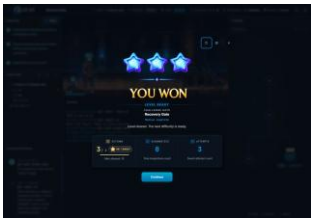
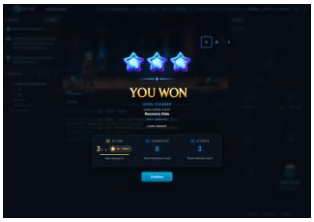
Trial	Gate and difficulty	Actual Result	Completion screen
8	Conflict Gate, Medium	PASS, 3 stars	
9	Conflict Gate, Hard	PASS, 3 stars	
10	Recovery Gate, Easy	PASS, 3 stars	
11	Recovery Gate, Medium	PASS, 3 stars	
12	Recovery Gate, Hard	PASS, 3 stars	

Table A-4. Challenge Gate coverage: completion screen of every trial.

A.3 Incident Report

Runs 1 and 2 (versions 1.0 to 1.2) found seven incidents, INC-01 to INC-07. Each was corrected and retested, and Run 3 retested every affected case again as part of the full regression. Version 3.0 adds INC-08 and INC-09, found while preparing Run 4 and closed after retest, and INC-10, the performance shortfall found by Run 4, retested in Runs 5 and 6 and closed in version 3.2. Table A-5 is the defect log, and the full report of each incident follows it.

ID	Defect	Severity	Found in	Corrective action	Retest result	Status
INC-01	Wrong password shows a session-expired message	Medium	Run 1, case 3.5	Sign-in and registration requests skip the token refresh; guard and unit test added (commit 7b4c86f)	Run 2: PASS. Run 3: PASS (3.5)	Closed
INC-02	Level title overlaps the mode label in the workspace header	Low	Run 1, case 3.9	Title shrinks to an ellipsis before the stats strip (commit 7b4c86f)	Run 2: PASS, measured (3.9). Run 3: case 3.9 passed	Closed
INC-03	Backend fails ruff lint and format checks	Medium	Run 1, case 3.2	3 import blocks sorted and 21 files formatted with ruff (commit 7b4c86f)	Run 2: PASS. Run 3: PASS (3.2)	Closed
INC-04	Ask Git-It chat covers page content and required controls	Medium	Run 1, cases 3.9 and 3.12	Close button added; chat starts closed on phones and inside levels and remembers its state (commit 7b4c86f)	Run 2: PASS. Run 3: PASS (3.12)	Closed
INC-05	Unknown addresses show a developer error screen	Low	Run 1, cases 3.11 and 3.13	Page not found error page added to the router (commit 7b4c86f)	Run 2: PASS locally. Run 3: PASS locally (3.11) and on production (3.13)	Closed
INC-06	Deployment settings guard fails under the CI environment	Medium	Run 1, case 3.4	Guard runs its checks with config.settings (commit 7b4c86f)	Run 2: PASS. Run 3: PASS (3.4)	Closed
INC-07	Ask Git-It labels every Home question as a scenario	Medium	Run 2, case 3.14	Scenario index rebuilt, label limited to confident matches, stale-index test added (commit 97337be)	Version 1.2: retested. Run 3: PASS (3.14)	Closed
INC-08	API returns HTML server errors when Redis is unreachable	High	Run 4 preparation, NFR-REL-02 check	Cache errors ignored with short timeouts; JSON not-found and server-error responses for /api/ (commit 66da969)	PASS: 7 new tests, backend suite 1,045 passed, outage probe	Closed

ID	Defect	Severity	Found in	Corrective action	Retest result	Status
INC-09	Stale frontend test blocks CI and production deploys	Medium	Run 4 preparation, production version check	Test fixture and expected label updated (commit efe4059)	PASS: CI green; production reports efe4059	Closed
INC-10	Command confirmation, level start and concurrent learners miss their targets	High	Run 4, cases 3.31, 3.32 and 3.36	Database moved to the API's region and 8 server threads (commit 6e9ed3c); smaller battle images (commit d65fe22); NFR-PERF-01, 04, 06 and 07 revised in SRS 1.4	Run 5: improved, still FAIL. Run 6: PASS (3.31, 3.32, 3.36)	Closed

Table A-5. Defect log.

INC-01 Wrong password shows a session-expired message

Severity: Medium. Players cannot tell that their password is wrong.

Found in: Run 1, case 3.5 Sign In and Sign Out (local build).

Steps: Open /login, enter an unregistered username and any password, and select Sign in.

Expected: An inline message says the username or password is incorrect, and the username stays filled in.

Actual: A toast said “Your session has expired. Please log in again.” and both fields were cleared. The server log showed POST /api/auth/login/ 401, then POST /api/auth/refresh/ 401 twice (Figure 75).

Cause: The login call in frontend/src/shared/auth/authApi.ts did not set skipAuthRefresh, so httpClient.ts treated the 401 as an expired access token, tried a token refresh, and on failure cleared the session and showed the expiry toast.

Corrective action: Sign-in and registration requests now skip the token refresh (frontend/src/shared/auth/authApi.ts). The auth contract guard requires this, and a unit test covers it (authApi.test.ts). Commit 7b4c86f.

Fix commit: 7b4c86f (21 October 2026), “feat: improve tutor responses, scenario handling, and workspace UI”.

Retest: Run 2: PASS (case 3.5). Run 3: PASS. A wrong password showed “Incorrect username/email or password” under the form and kept the user on the sign-in page (case 3.5, Figure 1).

Status: Closed.

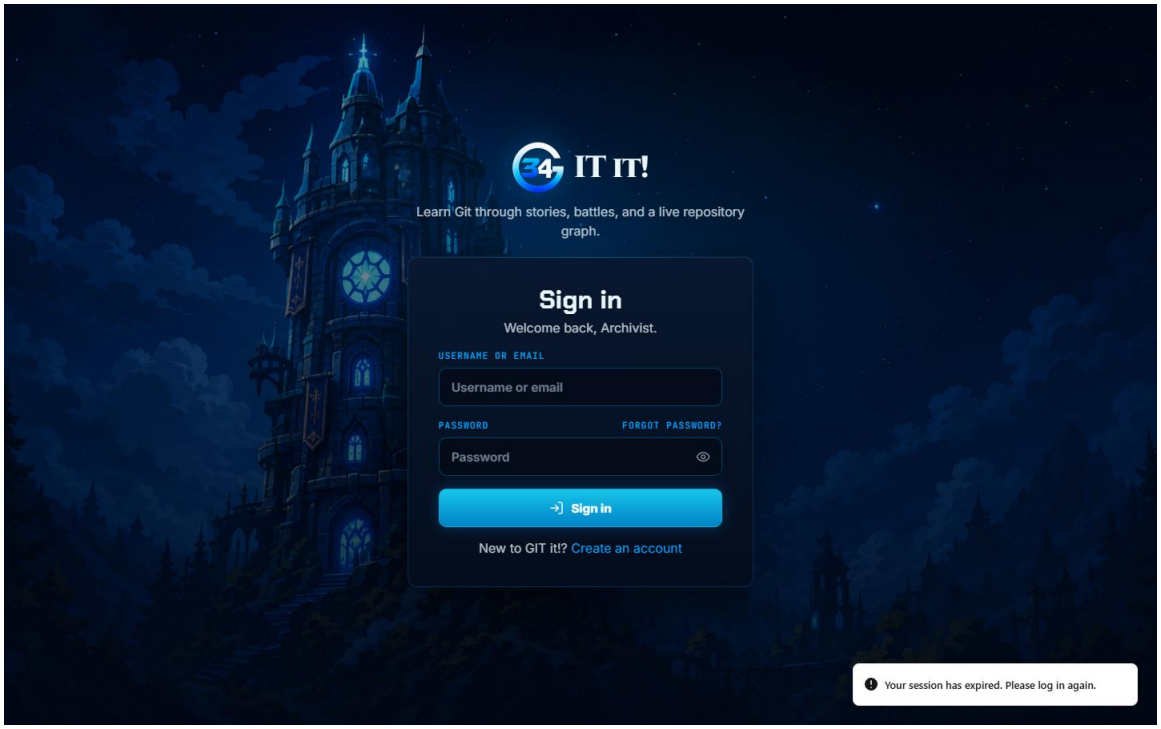


Figure 75. INC-01 in run 1: a wrong password produced a session-expired message.

INC-02 Level title overlaps the mode label in the workspace header

Severity: Low. Cosmetic.

Found in: Run 1, case 3.9 Adventure Play (1440 x 900).

Steps: Start Level 1, Initializing Repositories, in any tier.

Expected: A long title is cut off with an ellipsis before the next header item.

Actual: The title ran under the “Mode Adventure” label and the two texts overlapped (Figure 76).

Cause: In the header layout the stats strip could shrink while the title could not, so a long title ran under the strip instead of being cut off.

Corrective action: The stats strip keeps its width and the title shrinks to an ellipsis, with the full title shown on hover (WorkspaceStatusHeader.tsx, workspace-shell.css and tier-workspace.css). Commit 7b4c86f.

Fix commit: 7b4c86f (21 October 2026), “feat: improve tutor responses, scenario handling, and workspace UI”.

Retest: Run 2: PASS. The title ended at x = 239 px and the stats began at x = 252 px (case 3.9). Run 3: case 3.9 passed; the header was not measured again.

Status: Closed.

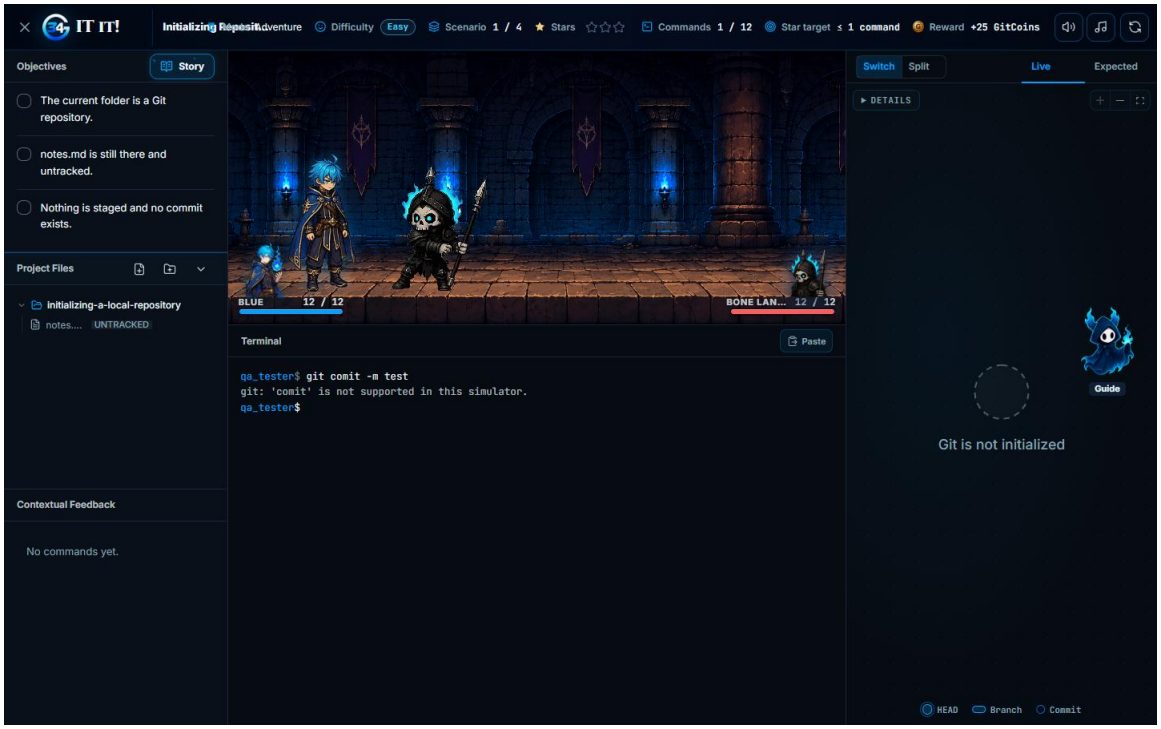


Figure 76. INC-02 in run 1: the level title overlapped the mode label.

INC-03 Backend fails ruff lint and format checks

Severity: Medium. The CI quality-gates job fails on these checks.

Found in: Run 1, case 3.2 Backend Code Quality Gates.

Steps: In backend/, run ruff check and ruff format --check on the whole directory.

Expected: No lint errors are reported, and no file would be reformatted.

Actual: ruff check reported I001 (unsorted imports) in authoring/compiler.py, authoring/validators.py and curriculum/tests/test_scenario_plan_invariants.py. ruff format --check listed 21 files, including adventures/services/tier_runs.py, adventures/tier_payloads.py and tutoring/tests/test_forms.py. All were files changed in the uncommitted working tree.

Cause: The changed files had not been linted and formatted before the test run.

Corrective action: ruff check --fix and ruff format were run in backend/: 3 import blocks were sorted and 21 files formatted. Commit 7b4c86f. The two authoring files were later deleted when the content authoring feature was removed (commit 8b44499).

Fix commit: 7b4c86f (21 October 2026), “feat: improve tutor responses, scenario handling, and workspace UI”. The two authoring files were later deleted in 8b44499 (21 October 2026), “refactor: remove obsolete content authoring feature”.

Retest: Run 2: PASS (case 3.2). Run 3: PASS. All checks passed, and the format checks covered 472 backend files and 84 repository scripts (case 3.2).

Status: Closed.

INC-04 Ask Git-It chat covers page content and required controls

Severity: Medium. On phones the chat hid most of the page on every load.

Found in: Run 1, cases 3.9 Adventure Play and 3.12 Responsive Layout.

Steps: Open /home or the story map at 390 x 844, or clear a scenario at 1440 x 900 so that the command guide opens.

Expected: The chat starts collapsed or remembers being closed, has a visible close button, and never covers controls the player must use.

Actual: The chat opened on every page load and covered about 85% of the phone screen. In the level workspace it covered the command guide's Got it button (Figure 77). It had no visible close control; only Escape or a click outside closed it, and it opened again on the next page load.

Cause: The chat started open on every page load, did not remember being closed and had no close button.

Corrective action: The chat has a visible close button. Outside levels it remembers being closed and starts closed on phones (720 px wide or less). Inside a level it starts as an Ask Git-It launcher, so it never covers the command guide. Commit 7b4c86f. Later launcher changes (27 and 28 October) kept this behavior; it was confirmed in the current source on 30 October 2026.

Fix commit: 7b4c86f (21 October 2026), "feat: improve tutor responses, scenario handling, and workspace UI".

Retest: Run 2: PASS (cases 3.9 and 3.12). Run 3: PASS. At 390 x 844 the dashboard, map, shop, Settings and level workspace fit the phone width (case 3.12, Figure 17).

Status: Closed.

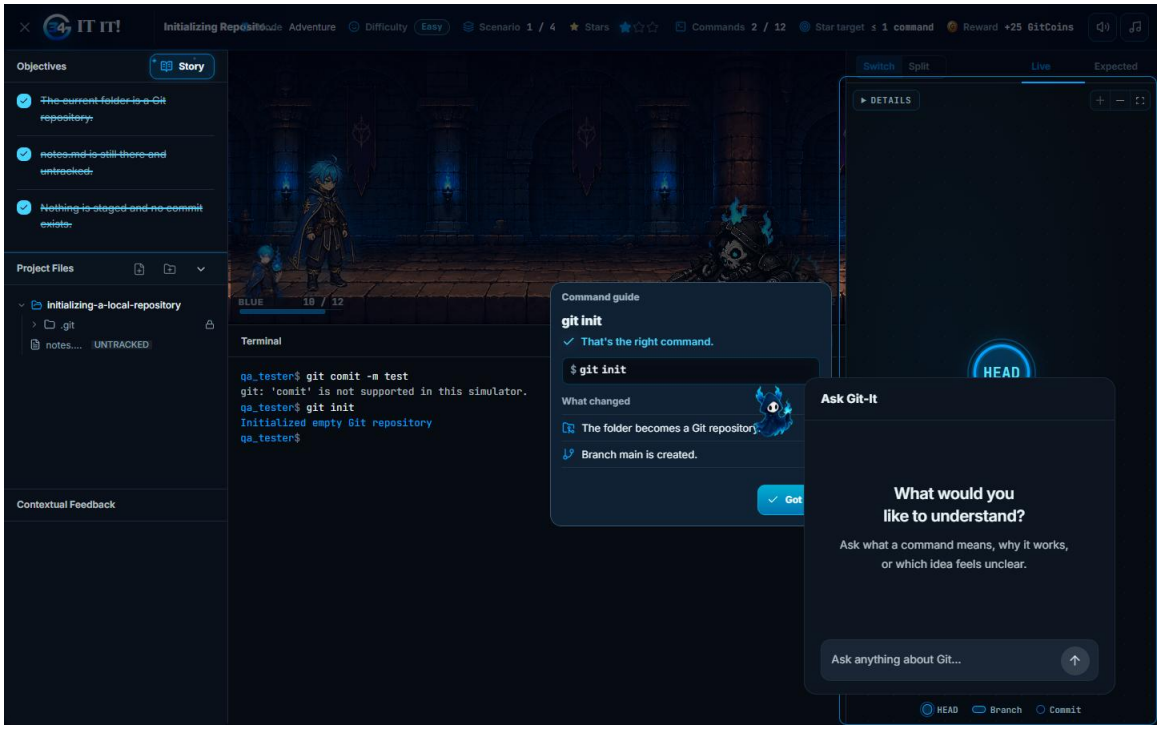


Figure 77. INC-04 in run 1: Ask Git-It covered the command guide's Got it button.

INC-05 Unknown addresses show a developer error screen

Severity: Low. Players who follow a broken or mistyped link see developer text and no way back.

Found in: Run 1, cases 3.11 Access Control and Routing and 3.13 Production Smoke Test.

Steps: Open /this-route-does-not-exist, locally or on https://git-it-34.vercel.app.

Expected: A not-found page in the app's style with a link back to the app.

Actual: React Router's default screen, "Unexpected Application Error! 404 Not Found", followed by a message addressed to the developer (Figure 78).

Cause: The router in frontend/src/app/router.tsx had no catch-all route and no errorElement.

Corrective action: The router's root route now has an error page (frontend/src/app/RouteErrorPage.tsx) that shows "Page not found" with a way back. Commit 7b4c86f.

Fix commit: 7b4c86f (21 October 2026), "feat: improve tutor responses, scenario handling, and workspace UI".

Retest: Run 2: PASS locally (case 3.11). Run 3: PASS. The unknown address showed "Page not found" with a link back to the app, locally (case 3.11, Figure 16) and on production (case 3.13, Figure 19).

Status: Closed.

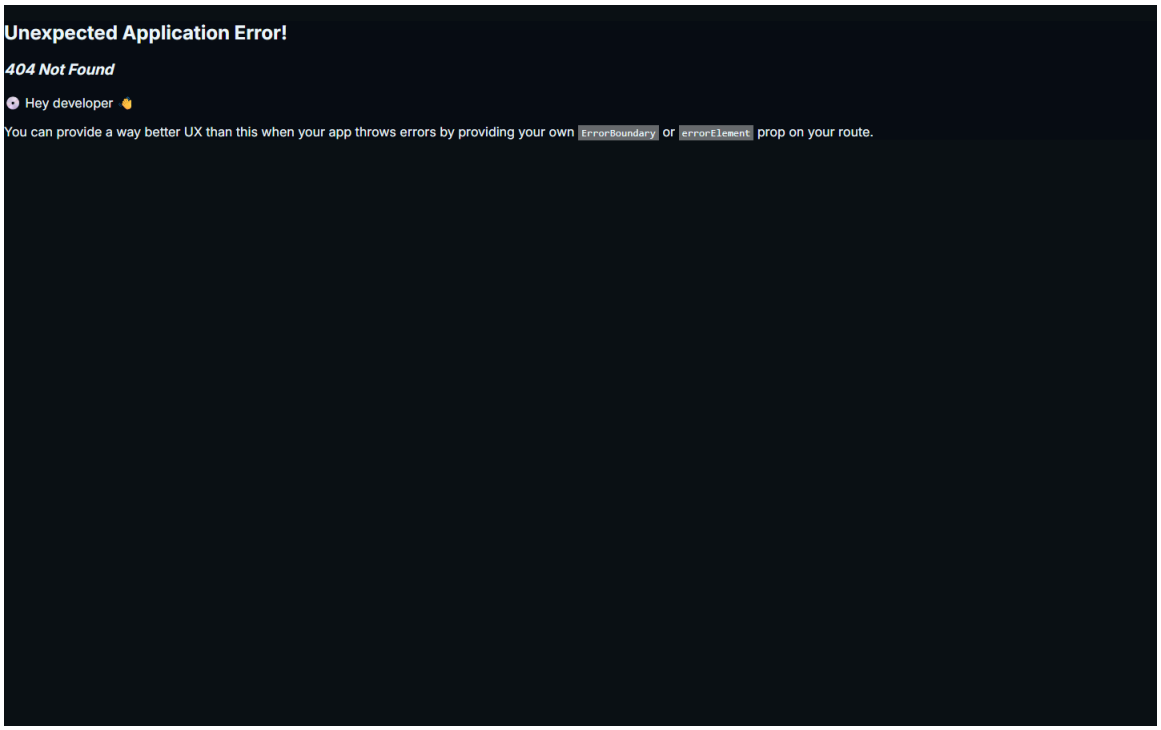


Figure 78. INC-05 in run 1: an unknown address showed the developer error screen.

INC-06 Deployment settings guard fails under the CI environment

Severity: Medium. The CI quality-gates job is expected to fail on this step.

Found in: Run 1, case 3.4 Architecture, Contract and Deployment Guards.

Steps: From frontend/, with DJANGO_SETTINGS_MODULE=config.test_settings as set in ci.yml, run python ../scripts/check_django_deploy.py.

Expected: The production-settings check passes.

Actual: Runtime configuration reported debug=True, and Django's deploy check failed with W008, W012, W016 and W018. Without DJANGO_SETTINGS_MODULE the same script passed (debug=False, no issues).

Cause: The script passed the parent environment to manage.py, so the inherited config.test_settings, which sets DEBUG = True, overrode the production values that the script supplies.

Corrective action: scripts/checks/check_django_deploy.py now sets DJANGO_SETTINGS_MODULE to config.settings for its own checks. Commit 7b4c86f.

Fix commit: 7b4c86f (21 October 2026), "feat: improve tutor responses, scenario handling, and workspace UI".

Retest: Run 2: PASS with the CI environment (case 3.4). Run 3: PASS. Every guard script exited with code 0 (case 3.4).

Status: Closed.

INC-07 Ask Git-It labels every Home question as a scenario

Severity: Medium. General Git questions got the scenario label and placeholder-only answers, and the label appeared even on off-topic replies.

Found in: Run 2, case 3.14 Ask Git-It: General Git Help.

Steps: On Home, ask Ask Git-It a general question, such as "What is the difference between git merge and git rebase?"

Expected: General questions are answered without the scenario label; only questions that match a GIT it! scenario switch to protected teaching.

Actual: Every reply showed "Scenario spotted: explaining, not solving", and the follow-up used only placeholders such as <target-branch>. Asked directly, the recognizer rated even "What is the capital of France?" as uncertain.

Cause: The tutor's scenario index (backend/tutoring/data/scenario-index.json) was stale. Scenario ids include the scenario bank's file name, and the bank file had been renamed, so the tutor ignored the index. Without it, recognition fell back to "uncertain", which protects every conversation by design.

Corrective action: A first rebuild during run 2 stopped at 296 of 399 scenarios when the embedding API quota ran out. The index was then rebuilt for all 399 scenarios. A match now counts as confident only from a score of 0.75, so general Git questions, which score up to about 0.74, no longer get the label. "Scenario spotted" appears only for a confident match or inside a level, and never on the "AI is unavailable" message. A new test fails whenever the committed index no longer matches the scenario bank. Commit 97337be.

Fix commit: 97337be (21 October 2026), "fix(tutor): rebuild scenario index and label only confident matches".

Retest: Version 1.2 retested cases 3.14 and 3.15 after the fix. Run 3: PASS. General answers carried no scenario label, the follow-up used the earlier context, and the unrelated and override requests were refused (case 3.14, Figures 24 to 27).

Status: Closed.

INC-08 API returns HTML server errors when Redis is unreachable

Severity: High. When the cache is down, every request that applies a rate limit fails, including token refresh, registration and command submission, so learners are signed out and cannot play.

Found in: Run 4 preparation, while checking NFR-REL-02 (the API keeps serving when the cache is unavailable) with a local probe on 30 October 2026.

Steps: Point the cache at a closed Redis port, then call POST /api/auth/refresh/ and POST /api/auth/register/ through the application client.

Expected: The API keeps serving; /api/health/ready/ reports degraded, and any error is returned as JSON.

Actual: /api/health/ready/ correctly answered 503 degraded, but refresh and register answered HTTP 500 with the HTML page Server Error (500) after about 0.5 s.

Cause: DRF's rate limiting reads and writes the cache, and the Redis cache backend raised a connection error. Django's default not-found and server-error handlers also return HTML, which the frontend cannot read as an API error.

Corrective action: Commit 66da969: the Redis cache ignores connection errors with short connect and socket timeouts, so rate limits fail open while the error is logged, and /api/ addresses get JSON not-found and server-error responses (backend/config/settings.py, backend/common/api.py, backend/config/urls.py). DEPLOYMENT.md describes the outage behavior.

Fix commit: 66da969 (30 October 2026), “fix(api): keep serving when Redis is down and answer errors in JSON”.

Retest: PASS. Seven new tests (test_cache_outage.py and test_error_responses.py) pass; five of them fail without the fix. The full backend suite passed (1,045 tests). The probe now gets 200 for login and refresh with Redis down, and /api/health/ready/ still reports degraded. The fix is deployed: production runs efe4059, which includes 66da969.

Status: Closed.

INC-09 Stale frontend test blocks CI and production deploys

Severity: Medium. Fixes merged to main did not reach production.

Found in: Run 4 preparation: production reported version ad4e9d1 although main already contained 6d6d1f2 and 66da969.

Steps: Compare the version in /api/health/ready/ with the latest commit on main, then open the CI run of that commit.

Expected: Every commit on main passes CI and is deployed.

Actual: The frontend CI job had failed since commit 6d6d1f2. Render deploys only after checks pass (autoDeployTrigger: checksPass), so production stayed on ad4e9d1.

Cause: Commit 6d6d1f2 added star scoring and changed the star label, but TierStatusHeader.test.tsx still used a run fixture without star_breakdown and expected the old label.

Corrective action: Commit efe4059 adds star_breakdown to the fixture and expects the new label, Star rules: 2 of 3 stars available on clear.

Fix commit: efe4059 (30 October 2026), “test(frontend): expect the available-on-clear star label for started runs”.

Retest: PASS. All five CI jobs succeeded, and production reported version efe4059 before Run 4 started.

Status: Closed.

INC-10 Command confirmation, level start and concurrent learners miss their targets

Severity: High. Every command waits about 1 s for confirmation and starting a level takes about 4 s. The free deployment slows sharply beyond about 16 learners, and at 28 the hosting platform starts rejecting requests, against a target of 40.

Found in: Run 4, cases 3.31, 3.32 and 3.36 (production, 30 October 2026).

Steps: Run loadtest.py with 40 virtual learners (case 3.32), or start Level 1 Easy in the browser (case 3.36).

Expected: With 40 learners, command confirmation ≤ 1 s at the 95th percentile and at least 4 requests/s; level start ≤ 2 s.

Actual: Command confirmation p95 was 1.35 to 1.47 s with 4 to 5 learners and 7.7 s with 28. Throughput peaked at 2.3 requests/s. During a 60 s ramp, 21 of 40 learners could not sign in. Level start took 4.0 to 4.8 s (Figure 79).

Cause: Three causes. (1) Distance: the API runs in Singapore and the database pooler in Tokyo. A git status submit runs about 13 queries one after another, and the database executes them in about 1.2 ms in total, so nearly all of the 1 s is spent on round trips between the two regions. Level start and the dashboard run more queries. (2) Capacity: the free instance runs one Gunicorn worker with two threads on 0.1 CPU. A thread is held for the whole of a request, including those round trips, so the instance completes about 2 requests per second. Sign-in also hashes the password, which takes seconds on 0.1 CPU. (3) When the queue grows, Render's edge answers 503 and then 429 before requests reach the application.

Corrective action: Applied on 30 October 2026: (a) the database moved to a new Supabase Free project in Singapore (ap-southeast-1), the API's region, with the curriculum seeded again; (b) GUNICORN_THREADS raised from 2 to 8 in render.yaml; (c) battle backgrounds served as WebP and skill effect sheets palette-optimized, so a first visit to Level 1 downloads 2.96 MB of battle art instead of 6.3 MB; (d) NFR-PERF-01, NFR-PERF-04, NFR-PERF-06 and NFR-PERF-07 revised in SRS version 1.4 to the capacity of the free hosting tier: its 0.1 CPU cannot sign in 40 learners within one minute, and the level's loading screen deliberately waits for all of the encounter's art so that battles never flicker (Table A-20). Fewer queries per request was not needed for the revised targets.

Fix commit: 6e9ed3c (30 October 2026), “perf(deploy): run 8 Gunicorn threads on the Render API”; d65fe22 (30 October 2026), “perf(assets): shrink battle backgrounds and effect sheets for level start”. The database move is a hosting configuration change and the target revision is SRS version 1.4; neither has a commit.

Retest: Run 5 (commit 6e9ed3c): partly corrected; 3.31, 3.32 and 3.36 still failed against SRS 1.3 (Appendix A.5). Run 6 (API 6e9ed3c, frontend d65fe22): PASS against SRS 1.4. 40 learners joining one every 15 s played for 10 minutes at 4.96 requests/s with 1 failed request of 4,541; command confirmation p95 was 0.74 s with up to 20 learners and 1.83 s with 40; at 10 Mbps, level start took

0.57 to 0.77 s with the art cached and 6.15 s on a first visit (Appendix A.6).

Status: Closed.

Observations

These were noted during Runs 3 to 6. They are not separate defects; O-6 shares the cause recorded in INC-10. O-4 and O-5 of version 1.2 were retired in version 2.0, and their numbers are not reused. They describe behavior that works as designed but is worth knowing when reading the results.

- **O-1:** Inside a one-step level, the tutor may explain the command that the level's command guide already teaches, such as git init, while still refusing to solve the task (case 3.15). This is teaching behavior, not a leak of the answer, but it means that generic commands are not always withheld.
- **O-2:** The production API host sleeps after 15 minutes without traffic. The first visit afterwards shows “Restoring session” until the host wakes: about 30 seconds in Run 3, and 86 to 98 seconds for the first request in Run 4 (case 3.37). NFR-REL-01 adds an uptime monitor to keep the host awake.
- **O-3:** The working tree tested in Run 3 was not deployed. The production results in case 3.13 cover the version that was live on 21 October 2026.
- **O-6:** The chapter map shown after sign-in requests 14 API endpoints at once. On the two-thread free instance its chapter progress arrived 4.2 to 7.6 s after a reload, compared with 2.2 to 3.3 s for Home (case 3.34).
- **O-7:** Run B's stopping rule ended it at 28 learners, so steady play between 28 and 40 learners was not observed. Run A shows that 40 learners signing in within one minute exceed what the platform accepts.
- **O-8:** While the battle animation of a command plays (about 4 s), pressing Enter does not submit the next command, and the typed text stays in the input. In Run 6, commands typed 2.5 s apart were therefore joined into one unsupported command (git statusgit status), which cost companion health. Case 3.33 was repeated with the commands 6 s apart. A learner who presses Enter during an animation gets no sign that it was ignored, which is worth reviewing.

A.4 Performance Measurements

Run 4 ran against https://git-it-34.vercel.app on 30 October 2026, in the environment of Table 2-2. Latencies are measured at the client from sending a request to receiving the whole response, so they include the network path from the Philippines through Vercel to the API. Percentiles use successful responses only; failed requests are counted separately. The raw request logs (prod_40_requests.csv with 441 requests and prod_step_requests.csv with 621 requests) and their summaries are kept with the test evidence. Table A-6 defines the two workloads.

Item	**Run A**	**Run B**
Purpose	40 learners at once (NFR-PERF-04)	Capacity: learners added gradually
Start (Philippine time)	03:27:56	03:40:40
Virtual learners	40	40 planned; 28 joined before the stop
Joining	Over 60 s, one every 1.5 s	One every 15 s

Item	Run A	Run B
Duration	60 s ramp, then 600 s steady state	Planned 600 s ramp and 120 s hold; stopped after 6 min 47 s
Each learner	Sign in, start or resume Level 1 Easy (buying a companion first when the API asks for one), then git status about every 10 s (±20%) and profile and dashboard about every 60 s	Same as Run A
Offered load at 40 learners	About 4 commands/s plus 1.3 Home requests/s	Same as Run A
Timeout and stop rule	30 s timeout; no stop rule	30 s timeout; stop after more than 30 failed requests in 60 s

Table A-6. Workload definition.

Table A-7 lists Run A by request type. Times are in milliseconds.

Request	Count	HTTP status	p50	p95	p99	Max
Command submit, steady state	258	200 ×258	1,076	1,468	2,070	4,250
Profile, steady state	44	200 ×44	250	604	915	1,143
Dashboard, steady state	44	200 ×44	1,295	1,394	1,914	2,095
Sign in, whole run	40	200 ×19, 429 ×19, 503 ×2	9,757	18,030	18,738	18,915
Companion purchase, whole run	12	201 ×8, 429 ×4	7,873	16,495	16,739	16,800
Tier start, whole run	31	201 ×5, 423 ×12, 429 ×13, 503 ×1	12,963	17,931	18,322	18,419
Command submit, whole run	268	200 ×259, 429 ×9	1,076	1,468	2,067	4,250

Table A-7. Run A: latency by request type (ms).

HTTP 423 is the expected first answer to a tier start by a new account without a companion; the learner then buys one and starts again. HTTP 429 and 503 came from the hosting platform's edge within about 0.1 s; the application has no rate limit on sign-in, purchase or tier start.

Table A-8 lists Run B by minute, and Figure 79 plots it. Times are in milliseconds.

Minute	Learners joined	Requests/s	Submit p50	Submit p95	Failed requests
1	4	0.42	1,076	1,345	0
2	8	0.95	1,084	1,988	0
3	12	1.45	1,233	2,362	0
4	16	1.83	2,253	4,260	0
5	20	1.88	3,406	4,770	0
6	24	2.02	5,344	7,167	0
7 (46 s)	28	2.33	6,501	7,688	31

Table A-8. Run B: throughput and command confirmation by minute.

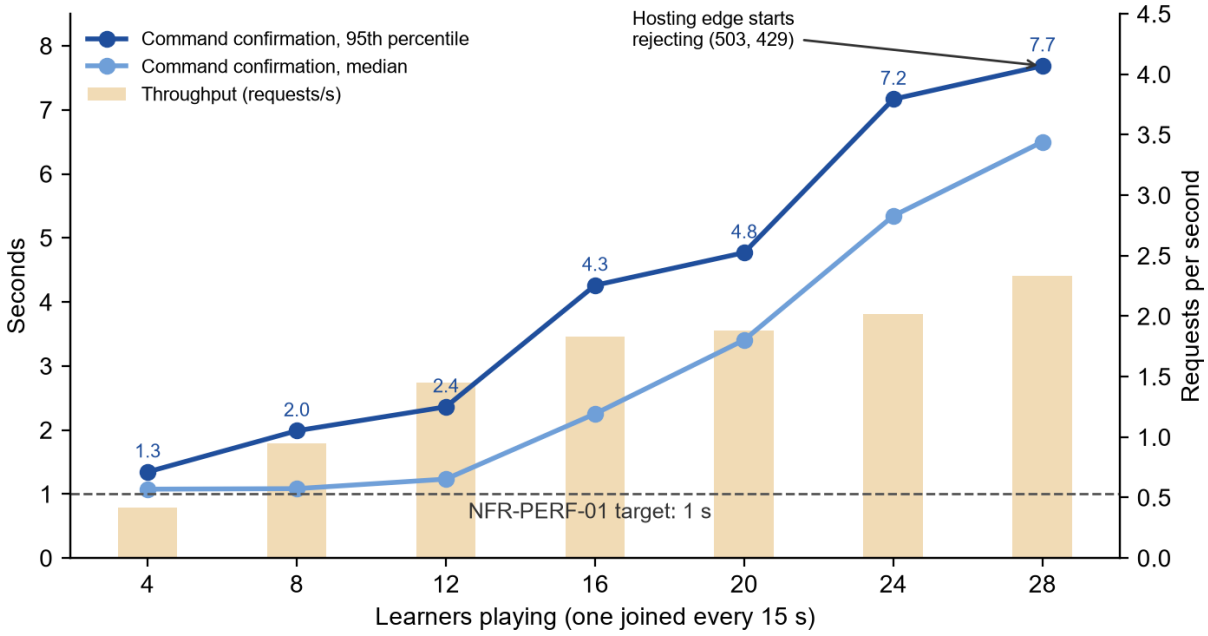


Figure 79. Run B: command confirmation time and throughput as learners join.

Table A-9 compares both runs with the targets of NFR-PERF-01, NFR-PERF-04 and NFR-PERF-07.

Metric	Run A	Run B	Target
Learners joined	40	28 (stopped)	40
Learners signed in	19	27	All
Learners playing (tier started)	5	26	All

Metric	Run A	Run B	Target
Requests	441	621	—
Throughput	0.58 requests/s (steady state)	0.42 to 2.33 requests/s	≥ 4 requests/s
Command confirmation p50 / p95	1.08 s / 1.47 s	2.94 s / 6.91 s	p95 ≤ 1 s
HTTP 5xx or timeout	3 (0.68%)	3 (0.48%)	< 1%
HTTP 429 from the hosting edge	47 (10.7%)	28 (4.5%)	—
Actual Result	FAIL	FAIL	

Table A-9. Workload results against the targets.

Table A-10 lists the ten queries with the highest total execution time during Run A (NFR-PERF-05).

Rank	Query	Calls	Total ms	Average ms
1	Load the tier run for a submit (adventures_adventureleveltierrun)	218	69.0	0.32
2	Command forms of the curriculum, for the dashboard	38	64.2	1.69
3	Record the command step (INSERT practice_commandstep)	218	42.2	0.19
4	Foreign-key check on the player row	219	34.0	0.16
5	Update the run's attempt counters	218	23.9	0.11
6	Look up the command form of the submitted command	218	22.7	0.10
7	Count scenarios per command form, for the dashboard	38	19.2	0.51
8	Command guide lookup for the scenario	218	18.1	0.08
9	Load the signed-in user (auth_user)	294	18.0	0.06
10	Tier runs of the player, for the dashboard	38	14.8	0.39

Table A-10. Top ten queries by total time, Run A steady state.

Run A window 03:30:28 to 03:38:57: 31 distinct statements, 3,556 calls and 434 ms of execution time in total. In Run B the application's highest average was 1.70 ms; two Supabase backup-job queries (126 ms and 44 ms, one call each) are excluded.

Table A-11 lists the single-user measurements on production.

Measurement	Samples	Actual Result	Target
3.33 Terminal feedback after Enter	7.4, 7.7, 7.8, 8.1, 8.9, 14.0, 14.1, 17.1, 17.1, 18.7 ms	Median 11.5 ms; max 18.7 ms	≤ 200 ms
3.34 Home: profile and progress shown	2.17, 2.20, 2.56, 2.85, 3.34 s	Median 2.56 s	≤ 3 s
3.34 Home: all panels loaded	3.45, 3.46, 3.49, 3.78, 3.97 s	Median 3.49 s	—
3.36 Level start until the terminal is ready	4.85 (new run), 4.02, 4.07, 4.09, 4.13, 4.21 s	Median 4.11 s	≤ 2 s (p95)
3.37 First request after sleep	97.6, 86.3 s; then a median of 0.39 s over 10 requests	Max 97.6 s	≤ 120 s
3.38 AI tutor reply	2.47, 2.79, 3.24, 3.34, 4.36, 4.40 s	Median 3.29 s; max 4.40 s	≤ 30 s

Table A-11. Single-user measurements.

A.5 Retest Measurements (Run 5)

Run 5 repeated the workloads of Table A-6 on 30 October 2026 between 11:25 and 11:41 Philippine time, in the environment of Table 2-3, and then repeated the level start timing. Times are measured as in A.4. The raw request logs (r5_40_requests.csv with 81 requests and r5_step_requests.csv with 2,151 requests) and their summaries are kept with the test evidence. Table A-12 lists Run B by minute, and Figure 80 compares it with Run 4. Times are in milliseconds.

Minute	Learners joined	Requests/s	Submit p50	Submit p95	Failed requests
1	4	0.42	233	390	0
2	8	1.10	244	721	0
3	12	1.58	418	1,356	0
4	16	2.15	306	848	0
5	20	2.55	390	1,069	0
6	24	2.92	374	2,190	0
7	28	3.43	500	1,763	0
8	32	3.72	1,607	3,382	0
9	36	3.98	1,088	3,227	0
10	40	4.42	2,011	3,646	0

Minute	Learners joined	Requests/s	Submit p50	Submit p95	Failed requests
11	40	4.67	1,154	3,267	0
12	40	4.82	670	1,523	0
13 (2 s)	40	3.92	1,134	1,379	0

Table A-12. Run 5, Run B: throughput and command confirmation by minute.

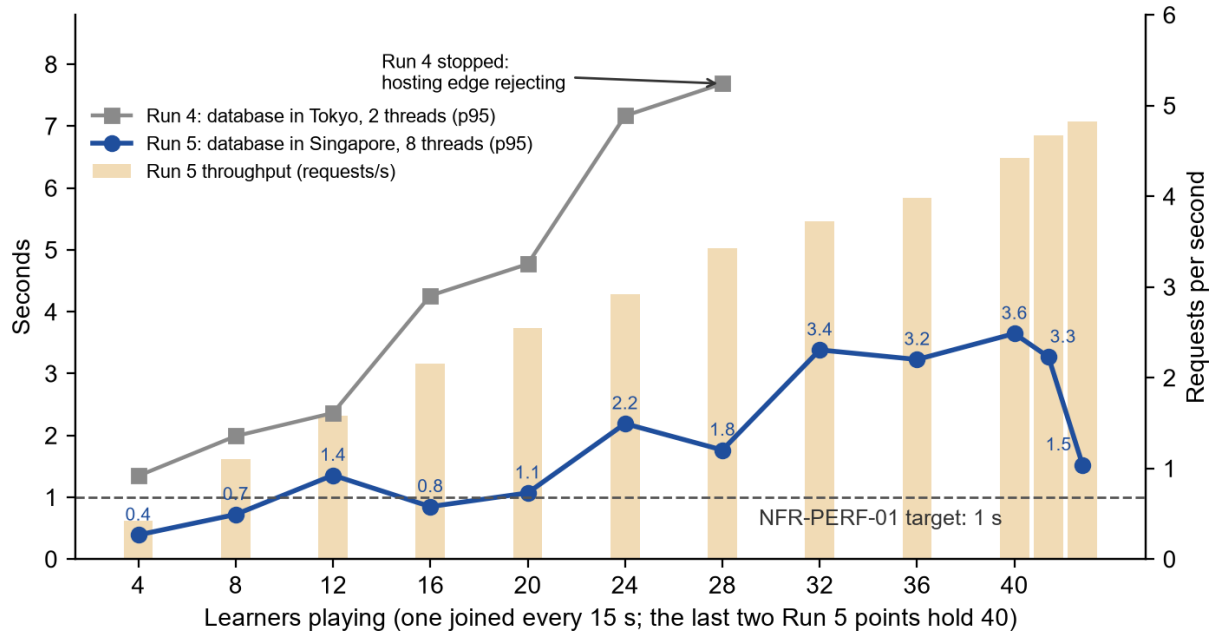


Figure 80. Command confirmation time as learners join: Run 4 compared with Run 5.

Table A-13 compares Run 5 with Run 4 and the targets of NFR-PERF-01, NFR-PERF-04, NFR-PERF-06 and NFR-PERF-07.

Metric	Run 4	Run 5	Target
Run A: 40 learners joining within 60 s	5 of 40 played; 10.7% of requests rejected by the edge	Stopped after 45 s; 21 of the 30 joined learners signed in	All 40 play
Run B: learners joined / playing	28 / 26 (stopped)	40 / 40	40
Run B: requests	621	2,151	—
Throughput with the most learners	2.33 requests/s (28 learners)	4.75 requests/s (40 learners, 2-minute hold)	≥ 4 requests/s
Command confirmation p95, 4 learners	1.35 s	0.39 s	≤ 1 s

Metric	Run 4	Run 5	Target
Command confirmation p95, 16 learners	4.26 s	0.85 s	≤ 1 s
Command confirmation p95, 28 learners	7.69 s	1.76 s	≤ 1 s
Command confirmation p50 / p95, 40 learners	Not reached	0.89 s / 2.90 s	p95 ≤ 1 s
Sign-in p50 / p95 (Run B)	3.03 s / 7.16 s	3.29 s / 5.28 s	—
Run B failed requests: 5xx or timeout / edge 429	3 (0.48%) / 28 (4.5%)	0 / 0	< 1%
Level start until the terminal is ready (3.36)	4.85 (new run), 4.02, 4.07, 4.09, 4.13, 4.21 s	4.55 (first open), 1.45, 1.02, 0.97, 0.94, 0.93 s	≤ 2 s (p95)
Level start request alone	About 3.9 s	0.38 to 0.60 s	—
Actual result: 3.31 / 3.32 / 3.36	FAIL / FAIL / FAIL	FAIL / FAIL / FAIL	

Table A-13. Run 5 compared with Run 4 and the targets.

Table A-14 lists the ten queries with the highest total execution time during Run 5's Run B (NFR-PERF-05).

Rank	Query	Calls	Total ms	Average ms
1	Load the tier run for a submit (adventures_adventureleveltierrun)	1,503	462.8	0.31
2	Command forms of the curriculum, for the dashboard	252	314.8	1.25
3	Command forms per scenario, for the dashboard	252	218.9	0.87
4	Record the command step (INSERT practice_commandstep)	1,503	203.7	0.14
5	Update the run's attempt counters and verified techniques	1,503	177.3	0.12
6	Look up the command form of the submitted command	1,544	126.1	0.08
7	Load the signed-in user (auth_user)	2,164	123.9	0.06
8	Command guide lookup for the scenario	1,544	101.6	0.07
9	Command form of the scenario's taught command	1,544	66.5	0.04
10	Load the player's run for a tier start	41	66.1	1.61

Table A-14. Top ten queries by total time, Run 5, Run B.

Run B window 11:28:01 to 11:40:17: 193 distinct statements, 28,141 calls and 2.65 s of execution time in total, including the activity of real learners at the same time. The highest application average was 9.98 ms (library entries, 3 calls).

A.6 Final Measurements (Run 6)

Run 6 ran the SRS 1.4 workload on 30 October 2026 from 12:53:58 to 13:14 Philippine time in the environment of Table 2-4, followed by the browser and tutor measurements. Times are measured as in A.4, and the browser was limited to 10 Mbps. The raw request log (r6_step_requests.csv with 4,541 requests) and its summary are kept with the test evidence, together with the discarded first attempt (r6a_*). Table A-15 lists the run by minute and Figure 81 plots it. Times are in milliseconds; failed requests are HTTP 5xx, HTTP 429 or no response, and token refreshes are expired access tokens renewed before a retry.

Minute	Learners joined	Requests/s	Submit p50	Submit p95	Failed requests	Token refreshes
1	4	0.47	270	459	0	0
2	8	1.00	229	524	0	0
3	12	1.52	231	624	0	0
4	16	1.97	232	599	0	0
5	20	2.42	256	987	0	0
6	24	2.97	336	978	0	0
7	28	3.32	467	2,172	0	0
8	32	3.75	583	2,224	1	0
9	36	4.05	1,404	2,507	0	0
10	40	4.37	1,865	2,994	0	0
11	40	4.80	990	2,269	0	0
12	40	4.80	509	1,869	0	0
13	40	4.92	451	1,858	0	0
14	40	4.87	364	957	0	0
15	40	4.97	516	1,936	0	0
16	40	4.95	447	1,534	0	4
17	40	5.32	400	1,111	0	4
18	40	4.87	495	1,487	0	4

Minute	Learners joined	Requests/s	Submit p50	Submit p95	Failed requests	Token refreshes
19	40	5.20	484	1,599	0	4
20	40	5.08	691	2,031	0	4
21 (2 s)	40	3.22	1,320	1,632	0	0

Table A-15. Run 6: throughput and command confirmation by minute.

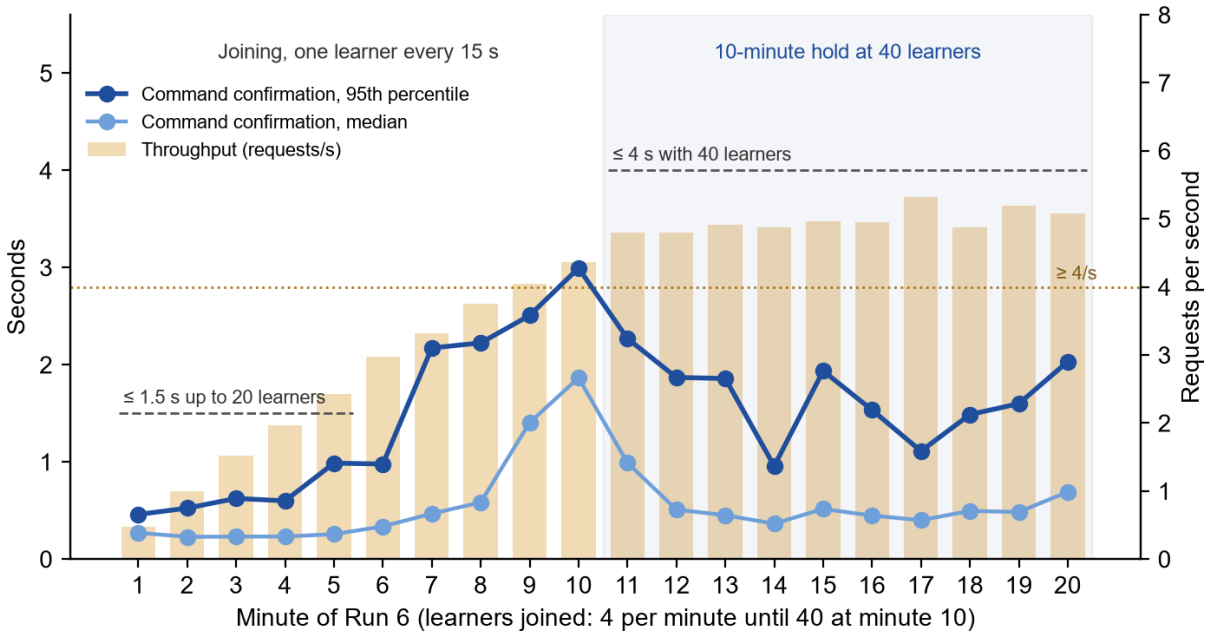


Figure 81. Run 6: command confirmation time and throughput against the SRS 1.4 targets.

Table A-16 compares Run 6 with the targets of NFR-PERF-01, NFR-PERF-04 and NFR-PERF-07 in SRS 1.4.

Metric	Run 6	Target (SRS 1.4)
Learners joined / playing	40 / 40	40, joining one every 15 s
Requests (whole run)	4,541	—
Throughput, 10-minute hold at 40	4.96 requests/s	≥ 4 requests/s
Command confirmation p50 / p95, up to 20 learners	0.24 s / 0.74 s	p95 ≤ 1.5 s
Command confirmation p50 / p95, hold at 40	0.48 s / 1.83 s	p95 ≤ 4 s
Command confirmation p95 / max, whole run	2.09 s / 3.83 s	—

Metric	Run 6	Target (SRS 1.4)
Sign-in p50 / p95	2.60 s / 4.93 s	—
Failed requests: 5xx / 429 / timeout	1 (HTTP 502) / 0 / 0; 0.02%	< 1%
Expired access tokens refreshed and retried	20	—
Actual result: 3.31 / 3.32	PASS / PASS	

Table A-16. Run 6 workload results against the targets.

Table A-17 lists the ten application queries with the highest total execution time during Run 6 (NFR-PERF-05).

Rank	Query	Calls	Total ms	Average ms
1	Load the tier run for a submit (adventures_adventureleveltierrun)	3,330	903.6	0.27
2	Command forms of the curriculum, for the dashboard	547	694.0	1.27
3	Record the command step (INSERT practice_commandstep)	3,330	481.2	0.14
4	Command forms per scenario, for the dashboard	547	462.4	0.85
5	Update the run's attempt counters and verified techniques	3,330	403.1	0.12
6	Tier runs of the player, for the dashboard	547	278.3	0.51
7	Look up the command form of the submitted command	3,370	270.1	0.08
8	Load the signed-in user (auth_user)	4,481	239.6	0.05
9	Command guide lookup for the scenario	3,370	231.4	0.07
10	Command form of the scenario's taught command	3,370	147.7	0.04

Table A-17. Top ten application queries by total time, Run 6.

Window 12:53:58 to 13:14:14: 138 distinct statements, 56,445 calls and 5.69 s of execution time in total. The connection pooler's DISCARD ALL resets (152 calls, 1.13 ms average) are not application queries and are left out of the ranking. The highest application average was 7.11 ms (challenge trial variants, 4 calls), from real learners using production at the same time.

Table A-18 lists the single-user measurements of Run 6.

Measurement	Samples	Actual Result	Target
3.33 Terminal feedback after Enter	11.5, 13.6, 14.8, 15.6, 16.2, 16.5, 17.0, 20.8, 23.7, 28.6 ms	Median 16.4 ms; max 28.6 ms	≤ 200 ms

Measurement	Samples	Actual Result	Target
3.34 Home at 10 Mbps: profile and progress shown	0.78, 0.81, 0.86, 0.92, 0.96 s	Median 0.86 s	≤ 3 s
3.34 Home at 10 Mbps: all panels loaded	0.94, 1.08, 1.10, 1.19, 1.21 s	Median 1.10 s	—
3.36 Level start at 10 Mbps, art cached	0.57, 0.60, 0.61, 0.68, 0.74, 0.74, 0.76, 0.77 s	p95 0.77 s	≤ 2 s (p95)
3.36 Level start at 10 Mbps, art downloaded	6.15 s (first visit, 3.9 MB); 5.66 and 2.10 s (resumes meeting a new monster)	Max 6.15 s	≤ 8 s
3.38 AI tutor reply, new questions	1.46, 1.55, 1.92, 2.18, 2.49, 4.60 s	Median 2.05 s; max 4.60 s	≤ 30 s

Table A-18. Run 6 single-user measurements.

Table A-19 gives the final status of every performance requirement of SRS 1.4, with the run that measured it.

Requirement	Target (SRS 1.4)	Actual Result	Run	Status
NFR-PERF-01	p95 ≤ 1.5 s up to 20 learners; ≤ 4 s with 40	0.74 s; 1.83 s	6	PASS
NFR-PERF-02	≤ 200 ms	Max 28.6 ms	6	PASS
NFR-PERF-03	≤ 3 s at 10 Mbps	Max 0.96 s	6	PASS
NFR-PERF-04	40 learners joining one every 15 s; ≥ 4 requests/s	40 learners; 4.96 requests/s	6	PASS
NFR-PERF-05	Average ≤ 100 ms (top ten queries)	Max 1.27 ms (top ten); 7.11 ms (any)	6	PASS
NFR-PERF-06	p95 ≤ 2 s with art cached; ≤ 8 s downloading at 10 Mbps	0.77 s; max 6.15 s	6	PASS
NFR-PERF-07	< 1% failed (5xx, 429 or timeout)	0.02%	6	PASS
NFR-PERF-08	≤ 120 s	Max 97.6 s	4	PASS
NFR-PERF-09	≤ 30 s	Max 4.60 s	6	PASS

Table A-19. Performance requirements: final status.

Table A-20 lists the performance targets that SRS 1.4 revised after Runs 4 and 5, and why.

Requirement	SRS 1.3 target	SRS 1.4 target	Reason for the change	Evidence
NFR-PERF-01	≤ 1 s at the 95th percentile with 40 learners	≤ 1.5 s at the 95th percentile with up to 20 learners; ≤ 4 s with 40	The free instance (0.1 CPU) could not confirm within 1 s under load, even after the database moved to the API's region and the server threads were raised. The terminal already shows the simulated result at once (NFR-PERF-02), so confirmation time does not hold up the learner. The new targets are the Run 5 measurements (1.02 s and 2.90 s) with a margin.	Tables A-9, A-13 and A-16
NFR-PERF-04	40 learners playing at once, all signing in within one minute; ≥ 4 requests/s	40 learners joining one every 15 s, then 10 minutes at 40; ≥ 4 requests/s	Each sign-in hashes the password, which takes seconds on 0.1 CPU. 40 sign-ins within one minute overloaded the instance and the hosting platform rejected requests. A class arriving over about 10 minutes is within capacity.	Tables A-7, A-9, A-13 and A-16
NFR-PERF-06	≤ 2 s at the 95th percentile	≤ 2 s at the 95th percentile when the art is cached; ≤ 8 s when it must be downloaded, at 10 Mbps	The loading screen deliberately waits for all of the encounter's battle art so that battles never flicker. A first visit downloads up to about 4 MB, about 3 s at 10 Mbps on its own, after the battle images had already been made smaller.	Tables A-11, A-13 and A-18
NFR-PERF-07	< 1% of requests fail with HTTP 5xx or time out	< 1%, also counting HTTP 429 rejections by the hosting platform	Made stricter. In Run 4 the hosting platform rejected 4.5% to 10.7% of requests with HTTP 429; learners see these as failures, but SRS 1.3 did not count them.	Table A-9

Table A-20. Performance target revisions in SRS 1.4.

The project runs on free hosting tiers by team decision, so SRS 1.4 set these targets to the capacity measured on that tier. A larger server would allow the SRS 1.3 targets (DEPLOYMENT.md).